FINFLOCK

Trading Engine

Technical and Conceptual Documentation

System design, market-data streaming, automatic strategy execution for stocks and options, broker connectivity, research and portfolio construction.

Repository	`trading-engine` (Django backend and IB Gateway child image)
Branch / commit	`backend` @ `170012f`
Document date	26 September 2026
Audience	Engineers, quants and operators working on the platform

How to read this document

Part I explains *what* the engine is and the ideas it is built on. Parts II–V describe each subsystem in technical depth: data models, state machines, formulas, thresholds and defaults, with references to source files. Part VI covers operations, configuration and the API. Chapter 15 collects known limitations found while writing this document. File references are relative to the repository root.

Contents

I Concepts and Architecture **1**

1 Introduction and Core Concepts **2**

1.1 What the trading engine is . . . 2
1.2 Design principles . . . 2
1.3 Core vocabulary . . . 3
1.4 The life of an automatic trade in one page . . . 4
1.5 Repository layout . . . 5

2 System Architecture **7**

2.1 Deployment topology . . . 7
2.2 Logical architecture . . . 9
2.3 Why these technologies . . . 9
2.4 Startup sequence . . . 9

II Market Data and Streaming **11**

3 Instruments and Market-Data Ingestion **12**

3.1 Instrument identity . . . 12
3.2 Market-data subscriptions and demand . . . 13
3.3 How the gateway produces market data . . . 14
3.4 The provider gate . . . 14
3.5 Fallback to Finnhub and failback . . . 15
3.6 Delayed data and paper trading . . . 16

4 The Streaming Pipeline **17**

4.1 Kafka topics . . . 17
4.2 The transactional outbox . . . 18
4.3 The Flink job . . . 18
4.4 Deterministic identity . . . 20
4.5 Persistence into PostgreSQL . . . 21
4.6 Dead letters and replay . . . 21

III Automatic Strategy Execution **23**

5 The Strategy Framework **24**

5.1 Definitions, instances and versions . . . 24
5.2 The plugin contract . . . 25
5.3 Input identity and shared requirements . . . 26
5.4 Activation lifecycle . . . 26
5.5 Evaluation jobs . . . 28
5.6 Reporting . . . 29

6 Automatic Stock Strategies **31**

6.1 Conventions shared by all stock plugins . . . 31
6.2 RSI mean reversion . . . 31
6.3 SMA crossover . . . 32
6.4 Donchian breakout . . . 32
6.5 Volatility-target momentum . . . 33
6.6 Fixed-weight rebalance . . . 33
6.7 From target weight to shares . . . 34
6.8 Catalogue strategies from research . . . 34
6.9 Stock strategy parameters at a glance . . . 34

7 Automatic Options Strategies **35**

7.1 Concepts . . . 35
7.2 Structures (templates) . . . 35
7.3 The decision funnel . . . 36
7.4 Stage 0: preconditions and data . . . 36
7.5 Stage 1: the RPS V2 score . . . 38
7.6 Stage 3: the IV and breach entry gate . . . 39
7.7 Stage 4: payoff, POP and expected value . . . 40
7.8 Stages 5 and 6: gates, TQS and the winner . . . 41
7.9 Stage 7: freezing the decision . . . 42
7.10 The options plugin . . . 42
7.11 Exits . . . 43
7.12 Executing a structure . . . 43
7.13 Options parameters reference . . . 45
7.14 Creating an options strategy . . . 46

8 Execution: From Target to Fill **47**
8.1 Target coordination . . . 47
8.2 The portfolio target snapshot . . . 48
8.3 Rebalance planning . . . 49
8.4 Position sizing . . . 50
8.5 Pre-trade risk . . . 50
8.6 Paper and live . . . 51
8.7 The order management system . . . 51
8.8 Broker command dispatch . . . 52
8.9 The return path: status, fills and ledgers . . . 53
8.10 Execution readiness . . . 54

9 Worked Walkthroughs **55**
9.1 A stock trade: SMA crossover on AAPL . . . 55
9.2 An options trade: iron condor on NIFTY . . . 56

IV Broker Connectivity **59**

10 Broker Connectivity **60**
10.1 Managed sessions . . . 60
10.2 Inside the IB Gateway child . . . 62
10.3 Event synchronisation . . . 63
10.4 Reconciliation . . . 64
10.5 Remote desktop (noVNC) . . . 64
10.6 Accounts and portfolios . . . 64

V Portfolio Intelligence **65**

11 Research and Recommendations **66**
11.1 Pipeline overview . . . 66
11.2 The research bundle . . . 66
11.3 Strategy implementations . . . 67
11.4 Backtest engines . . . 67
11.5 Experiments . . . 68
11.6 Scoring . . . 69
11.7 Goals and the recommendation cache . . . 69
11.8 Recommendation batches . . . 69

12 Portfolio Construction, Optimisation and Capital Flows **71**
12.1 The Portfolio Builder . . . 71
12.2 The Markowitz optimiser . . . 72
12.3 Capital flows . . . 73

VI Operations **74**

13 Platform and Operations **75**
13.1 Technology stack . . . 75
13.2 Scheduled work (Celery beat) . . . 75
13.3 API conventions . . . 76
13.4 Health, readiness and observability . . . 77
13.5 Dashboard analytics . . . 77
13.6 Security model . . . 77
13.7 Configuration . . . 78
13.8 Local validation stack . . . 78
13.9 Testing . . . 78

14 Data Model **79**
14.1 The relational spine . . . 79
14.2 Operational events and history . . . 79
14.3 Models by application . . . 79
14.4 Uniqueness guarantees that make the system idempotent . . . 81

15 Known Limitations and Review Notes **82**
15.1 Options . . . 82
15.2 Stock strategies and streaming . . . 83
15.3 Execution . . . 83
15.4 Broker connectivity . . . 83
15.5 Research . . . 84
15.6 Platform . . . 84

A REST API Catalogue **85**

B Environment Variables **88**

C Status Glossary **90**

PART I

Concepts and Architecture

CHAPTER 1

Introduction and Core Concepts

1.1 What the trading engine is

The Finflock Trading Engine is the execution core of the Finflock platform. It is a Django/ASGI backend that takes a portfolio connected to an Interactive Brokers (IBKR) account and runs *automatic trading strategies* against it: it streams market data, evaluates strategies on every completed bar, converts their decisions into portfolio targets, sizes and risk-checks orders, sends them to IBKR, and books fills back into positions, cash ledgers and per-strategy attribution. It supports two instrument families:

- **Stocks (equities)** — directional or weight-based strategies (RSI mean reversion, SMA crossover, Donchian breakout, volatility-targeted momentum, fixed-weight rebalance) that output a *target weight* of the portfolio.
- **Options** — defined-risk multi-leg structures (put and call credit spreads, iron condors, iron butterflies and others) selected by a quantitative decision engine (RPS V2 score, IV/breach entry gate, POP/EV pre-filters, a 14-factor Trade Quality Score) that outputs *exact contract quantities* per leg.

Around this core sit a **broker connectivity** layer (one isolated IB Gateway container per IBKR login), a **streaming** layer (Kafka and a PyFlink job that turns raw ticks into bars and indicators), a **research and recommendation** system that backtests a universe of 97 strategy hypotheses over 500 US large-caps, and a **portfolio construction** layer that turns user goals into strategy instances.

1.2 Design principles

The codebase follows a small number of principles consistently. Understanding them makes most of the detailed behaviour in later chapters predictable.

Fail closed.
When something is uncertain, the system stops rather than guesses. Automatic execution requires a green readiness report (Kafka, Flink, workers, gateway, reconciliation, fresh market data). Missing market data, a stale quote, a missing remote feature source or an unreconciled account all block trading instead of falling back to a neutral assumption.

The broker session is the source of truth for mode.
A portfolio is bound to a broker gateway session, and the session's mode (`paper` or `live`) defines the portfolio's execution mode (`PAPER` or `LIVE`). No API accepts an execution mode that disagrees with the session, and every hop re-checks the chain intent mode = command mode = session mode = gateway health mode.

Everything is idempotent.
Every externally triggered operation carries an idempotency key and a canonical request hash. Every internal event has a deterministic identity. Replays, retries, duplicate Kafka deliveries and worker crashes produce the same result exactly once.

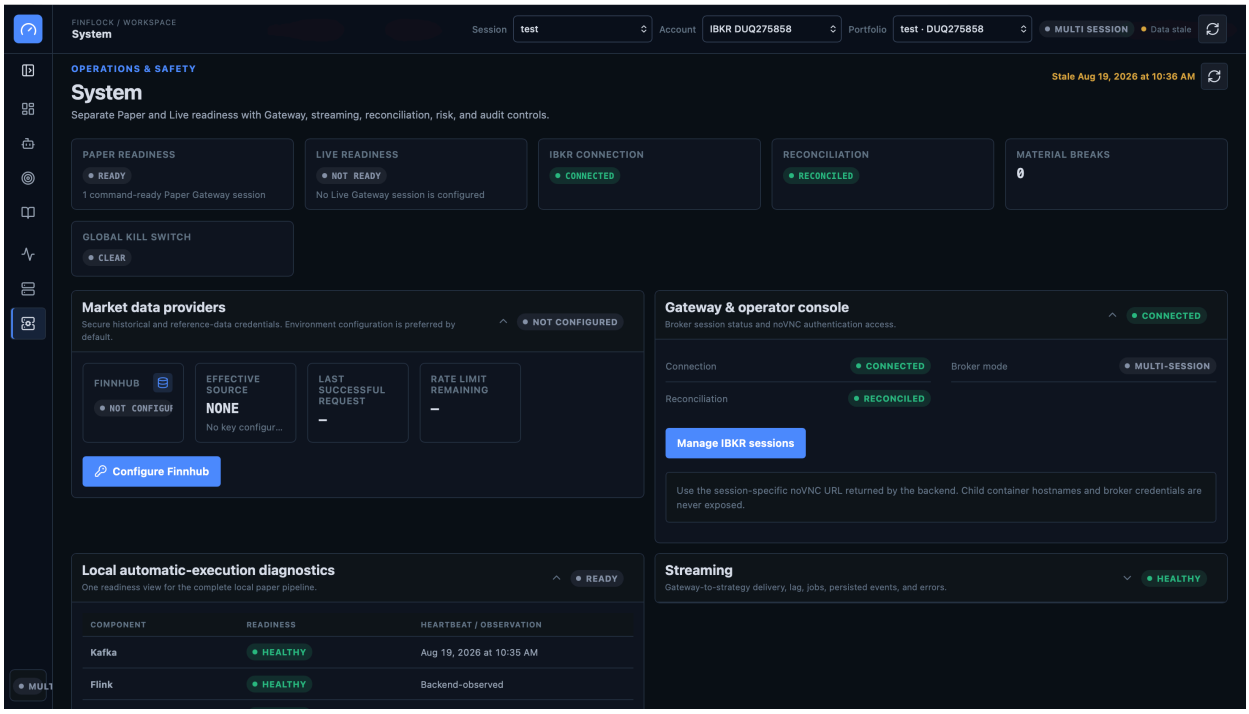


Figure 1.1: The operator's System page. Paper and Live readiness are reported separately, together with the IBKR connection, reconciliation state, kill switch, market data providers and streaming health. All of these come from the backend described here.

Durable state machines, not in-memory workflows.
Strategy activation, evaluation jobs, target coordination, rebalances, order intents, orders and broker commands are all rows in PostgreSQL with explicit states. Celery workers only *advance* these rows; if a worker dies, a recovery task re-claims the work.

Transactional outbox.
Domain state changes and the events describing them are written in the same database transaction; a publisher later ships the events to Kafka.

Evaluate only on facts.
Strategies are evaluated from persisted, final, versioned bars and indicators, never from a live buffer. Warm-up bars never trade. Delayed data may only drive `PAPER` execution.

Never resend an order blindly.
If the backend cannot prove whether an order reached IBKR, the command becomes **UNCERTAIN** and is reconciled against the gateway before any resend.

1.3 Core vocabulary

Concept	Meaning
Broker account	An IBKR account (`BrokerAccount`) with broker-projected NAV, cash, buying power and daily P&L, a reconciliation flag and a kill switch.
Gateway session	A managed IBKR login (`BrokerGatewaySession`) backed by its own IB Gateway child container. Has a mode (paper/live) and a lifecycle status.

Concept	Meaning
Trading portfolio	The unit the engine trades (`TradingPortfolio`). Bound to one account and one gateway session. Holds positions, ledgers, buffers and a kill switch.
Instrument / contract	A canonical instrument (`Instrument`) and its exact IBKR identity (`BrokerContract`, keyed by `conid`); options also have an `OptionContract`.
Strategy definition	A plugin class and its metadata (`StrategyDefinition`): declared inputs, parameter schema, supported timeframes and asset types.
Strategy instance	A definition bound to a portfolio, an instrument, a timeframe and parameters (`StrategyInstance`), with a lifecycle state.
Strategy version	An immutable snapshot of an instance's configuration. Each material change creates a new version.
Input requirement	A content-addressed demand for one stream input (a bar series or an indicator with parameters), shared between instances.
Market bar / indicator	Final, versioned OHLCV bars and indicator values produced by Flink and persisted by the backend.
Evaluation job	A durable unit of work: "evaluate instance X, version v, on bar b".
Strategy run / signal / target	The record of one evaluation, the decision it produced, and (if the desired position changed) the target weight or quantity.
Target coordination	A debounced per-portfolio request to net all current strategy targets into one portfolio target snapshot.
Rebalance run	The plan that converts a target snapshot into per-instrument trades.
Order intent	A request to trade one instrument. Passes position sizing and pre-trade risk before it becomes an order.
Order	The OMS record of an intent sent to the broker, with a strict status machine.
Broker command	A durable PLACE/MODIFY/CANCEL command to the gateway, with delivery semantics that tolerate lost responses.
Fill	A broker execution, applied exactly once to the order, position, ledgers and strategy attribution.
Reconciliation	A periodic comparison of broker positions and executions with internal records; a material break blocks trading on the account.

1.4 The life of an automatic trade in one page

From market tick to booked fill

1. **Activate.** An operator enables a strategy instance. Preflight checks readiness; the instance registers its inputs on Kafka topic `strategy.inputs.v1` and a market-data subscription is opened on the portfolio's IB Gateway.
2. **Warm up.** The gateway sends historical bars (`WARMUP`) followed by live 5-second bars. Flink aggregates them to the strategy's timeframe and computes the declared indicators. The backend persists bars and indicators; once enough warm-up bars with all indicators exist, the instance waits for its first live bar.

3. **Evaluate.** Each final `LIVE` bar creates an evaluation job. When all indicators for that bar have arrived, a worker runs the plugin's `evaluate()`. The decision is stored as a run and signal and, if the target changed, a new target.
4. **Coordinate.** The portfolio is marked for coordination. After a 2-second debounce, all strategies' latest targets are netted into one immutable snapshot.
5. **Plan.** A rebalance run converts target weights (or exact option quantities) into per-instrument trade deltas, respecting drift thresholds, cash, turnover and sell-before-buy (or hedge-before-short for options).
6. **Size and risk-check.** Each order intent is sized (weight cap, ADV participation, cash) and passes pre-trade risk (kill switches, mode, reconciliation, lot size, fresh price, limits, available position or option coverage, cash reservation).
7. **Dispatch.** An order and a PLACE broker command are created; a dedicated worker sends the command to the gateway, which places the order at IBKR.
8. **Book.** The backend pulls gateway events every 5 seconds: order status changes update the order; executions create fills, update positions and cash ledgers, and attribute the fill back to each contributing strategy.

Chapter 9 traces this path concretely for an SMA crossover on a stock and for an iron condor on NIFTY.

1.5 Repository layout

Path	Responsibility
`config/`	Django settings, Celery app and beat schedule, URL routing, ASGI entry.
`apps/core/`	Response envelope, middleware, idempotency, throttling, validation, system/health/order/dashboard views, analytics.
`apps/instruments/`	Instruments, IBKR contracts, option contracts, provider mappings, contract search and qualification, instrument registry publication.
`apps/broker_gateway/`	Managed IBKR sessions, QCH client, gateway HTTP client, event sync, noVNC proxy, secret encryption.
`apps/accounts/`, `apps/portfolios/`	Broker accounts; portfolios, positions and ledgers.
`apps/market_data/`	Finnhub provider, IBKR/Dhan option chains, remote feature database, provider fallback state machine, pricing helpers.
`apps/market_streams/`	Market-data subscriptions and leases, persisted bars, indicators and market state, evaluation jobs, the Kafka persistence consumer.
`apps/strategies/`	Strategy framework, plugins (stocks, options, catalogue), evaluation jobs, option analytics and execution, reporting, deletion.
`apps/allocation/`, `apps/rebalancing/`	Capital flows, target coordination, target snapshots, rebalance planning and advancement, attribution.
`apps/position_sizing/`, `apps/risk/`	Sizing policies and decisions; pre-trade risk, kill switches, capital reservations.

Path	Responsibility
apps/oms/, apps/execution/	Order intents, orders, status history; broker commands, dispatch, execution modes, readiness, Flink diagnostics.
apps/reconciliation/	Reconciliation runs and breaks.
apps/audit/, apps/event_bus/	Audit events, outbox, operation attempts; consumed events, dead letters, replay, stream health.
apps/research/	Research bundle import, backtest engines, 97 strategy implementations, experiments, scoring, recommendation cache.
apps/portfolio_construction/, apps/portfolio_optimization/	Goal-based Portfolio Builder; Markowitz optimiser.
streaming/flink/jobs/	The PyFlink market pipeline (normalisation, bars, indicators, stale detection, stream health).
gateway/IB_gateway/	The IB Gateway child image: IBC, IB Gateway, a small Django API, the broker worker, noVNC and nginx.
scripts/	Database, Kafka topic and Flink job bootstrap; Flink Python runtime build.
research_bundle/	The vendored research catalogue (stocks, strategies, GICS, backtest protocol, compatibility rules).
tests/	54 test modules, about 537 tests.

CHAPTER 2

System Architecture

2.1 Deployment topology

The backend ships as a single container image that runs many processes under Supervisor. It depends on PostgreSQL, Redis, Kafka and an external Flink cluster, and it creates one private IB Gateway container per IBKR login through QCH, the hosting platform's container broker. Figure 2.1 shows the runtime topology.

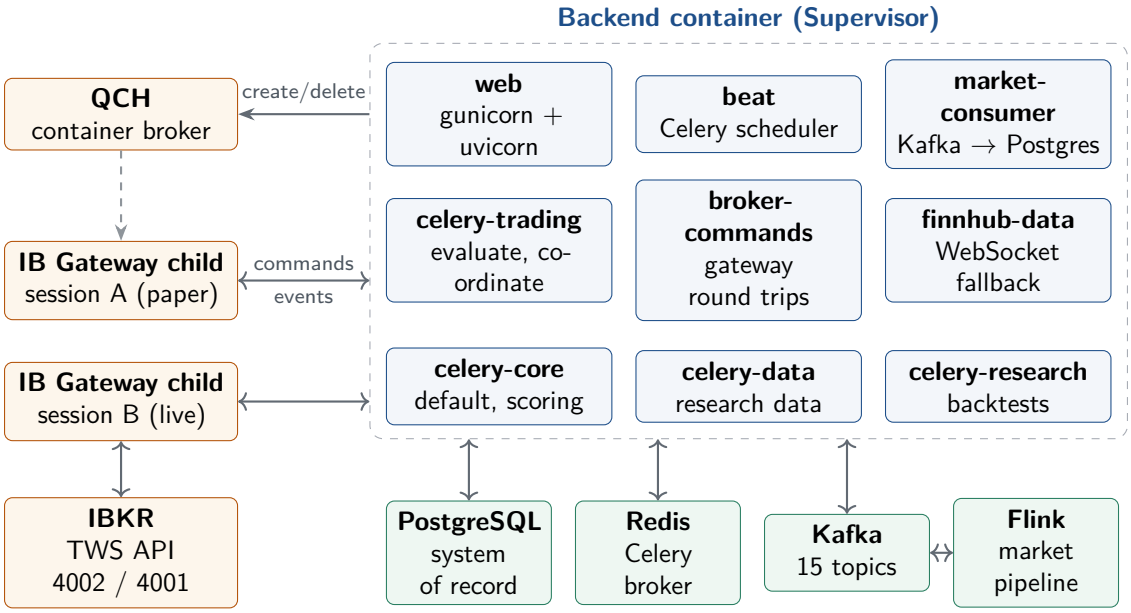


Figure 2.1: Runtime topology. All backend processes share one image and one database. Each IBKR login runs in its own gateway child container, created through QCH and addressed by container name on a shared network.

2.1.1 Processes in the backend container

Program	Queues / role	Notes
`web`	Gunicorn with `uvicorn_worker.UvicornWorker`, 2 workers	Serves the REST API and the noVNC reverse proxy (HTTP and WebSocket).
`celery-core`	`celery`, `research_scoring`, `recommendation_cache`, `recommendations`; prefork 2	Default queue: broker sync, outbox publishing, reconciliation, construction and optimisation tasks.
`celery-data`	six `research_*` data queues; threads, 8	Research data refresh and feature computation.
`celery-research`	five `research_*` backtest queues; threads, 8	Backtest experiments per engine role.

Program	Queues / role	Notes
`celery-trading`	`strategy_evaluation`, `target_coordination`, `intent_execution`; prefork 3, prefetch 1	Three child slots shared by the three trading queues. Prefetch 1 stops one queue from hoarding slots.
`celery-broker-commands`	`broker_commands`; prefork 1	Isolated so that a slow evaluation can never delay an order acknowledgement. Commands are serialised.
`beat`	Celery beat	Fires the polling tasks (1 s on the order path).
`market-consumer`	`manage.py consume_market_streams`	Persists Flink output (bars, indicators, market quality).
`finnhub-market-data`	`manage.py consume_finnhub_market_data`	Finnhub WebSocket fallback feed; idle unless fallback is enabled.

All Celery workers run with `-without-gossip -without-mingle -without-heartbeat`; nothing in the system consumes worker events, and these options remove fan-out traffic on the Redis broker. Worker liveness is instead proven by heartbeat rows in `StreamHealthMetric`, which execution readiness checks.

2.1.2 External services

PostgreSQL
The system of record for everything: configuration, strategy state, market bars and indicators, OMS, ledgers, outbox, audit. Row locks (`SELECT ... FOR UPDATE SKIP LOCKED`) implement all work queues.

Redis
Celery broker. Task results are ignored by default (`CELERY_TASK_IGNORE_RESULT=true`); only failures are stored, via `django_celery_results`.

Kafka
Event backbone for market data (raw, canonical, bars, indicators, quality), the instrument registry, strategy input registration, and audit streams (targets, risk, orders, executions, reconciliation).

Flink 1.20
Runs one PyFlink job, `market-pipeline-v1`, containing five stages. The backend submits it at startup and ships a self-contained Python runtime to the TaskManagers.

QCH
The hosting platform's sub-container broker. The backend asks it to create, list and delete IB Gateway children from a pinned image.

IB Gateway children
One per session. Each runs IBC, IB Gateway, a small Django API and a single broker worker that owns the TWS API connection (Chapter 10).

Finnhub
Optional REST and WebSocket fallback market data, and the primary source of research daily data.

Remote feature database
Optional read-only PostgreSQL source of daily index candles and IV-rank features used by the options RPS model.

2.2 Logical architecture

Figure 2.2 shows how data and control flow between subsystems. Three loops are visible: the *market-data loop* (gateway → Kafka → Flink → Kafka → Postgres), the *decision loop* (bars → evaluation → targets → coordination → rebalance) and the *execution loop* (intents → risk → commands → gateway → events → fills).

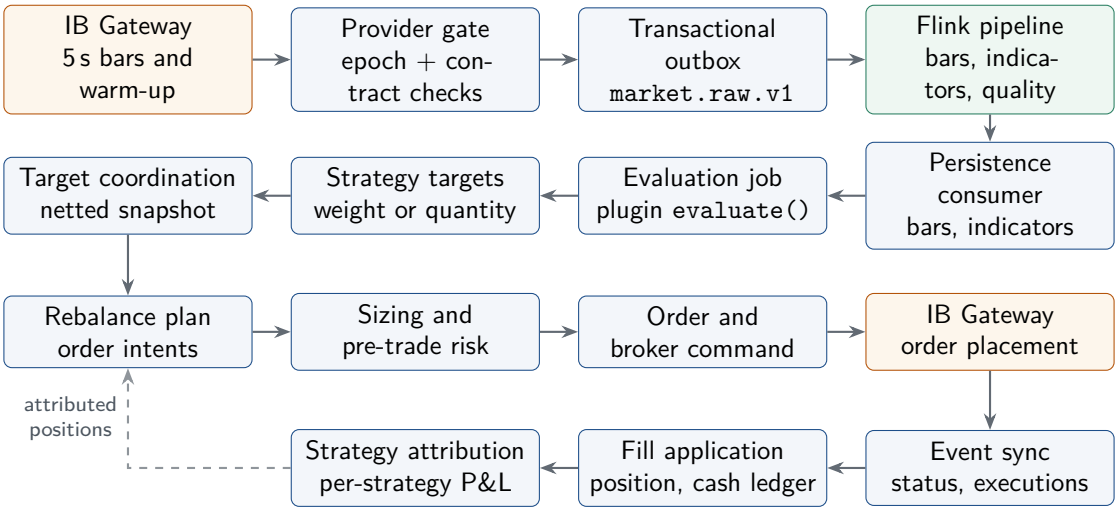


Figure 2.2: Logical data flow. Market data flows left to right along the top; decisions flow right to left through the middle; execution flows left to right along the bottom and returns to the gateway.

2.3 Why these technologies

- **Kafka and Flink for market data.** Bars and indicators must be computed once, deterministically, and shared between all strategies that need them. A stream processor with checkpointed keyed state is a natural fit; strategies declare their inputs and the pipeline computes exactly those.
- **PostgreSQL row queues for trading work.** Order-path work needs transactional guarantees with the domain state it changes ("claim this intent and create its order atomically"). Using database rows as the queue, with Celery beat as the clock, gives exactly that without a second source of truth.
- **One gateway container per login.** IB Gateway is a stateful Java desktop application with one API connection per login. Isolating each login in its own container, with its own database and credentials, prevents one session from ever executing another session's commands and lets sessions be created and destroyed on demand.
- **Celery beat at 1 s on the order path.** The four order-path pollers (evaluation, coordination, intents, broker commands) run every second. That second is the maximum added latency per hop; housekeeping tasks run far less often.

2.4 Startup sequence

The container entrypoint (`entrypoint.sh`) performs these steps before Supervisor starts any process:

1. Wait for PostgreSQL and create the application database if missing (`scripts/ensure_database.py`; 30 attempts, 2 s apart).

2. Apply Django migrations.
3. If `CLEANUP_DATABASE_ON_START=true`: cancel the required Flink jobs, delete all records from the required Kafka topics (keeping the topics), flush every database row, and re-seed the built-in strategy definitions and default policies. *This is destructive and repeats on every start while the flag is true.*
4. Create any missing Kafka topics (`scripts/ensure_kafka_topics.py`).
5. Ensure the Flink job is running (`scripts/ensure_flink_jobs.py`): under a PostgreSQL advisory lock, cancel superseded per-stage jobs, submit missing or terminal jobs (restoring the latest retained checkpoint unless cleanup mode is on), wait for **RUNNING** and a first completed checkpoint, and auto-repair a known Python-worker failure. A failure here is logged but does not stop the backend, so diagnostics remain available.
6. `exec supervisord`.

PART II

Market Data and Streaming

CHAPTER 3

Instruments and Market-Data Ingestion

Every trading decision in the engine rests on two things: knowing *exactly* which contract is being traded, and trusting the price data about it. This chapter covers instrument identity, how market-data subscriptions are demanded and opened, the single gate every raw market event passes through, the IBKR-to-Finnhub fallback state machine, and the rules that make delayed data safe for paper trading only.

3.1 Instrument identity

3.1.1 Models

Model	Purpose and key fields
`Issuer`	A stable company identity (unique `cik`, legal and display name, headquarters). Tickers identify instruments, not issuers.
`Instrument`	The canonical instrument: `symbol`, `asset_class` (default `STK`), `exchange` (default `SMART`), `primary_exchange`, `currency`, `multiplier`, `lot_size`, `min_tick`, `fractional_support`, `trading_calendar`, `active`, `tradable`. Unique on (symbol, asset class, exchange, primary exchange, currency).
`BrokerContract`	One-to-one with an instrument. `conid` (unique big integer) is the exact IBKR contract identity. *Everything downstream is keyed by conid*: gateway subscriptions, raw events, stream normalisation and the contract-mismatch gate.
`OptionContract`	One-to-one with an option instrument: `underlying`, `underlying_conid`, `expiration`, `strike`, `right` (C/P), `trading_class`, `multiplier`. Unique on (underlying conid, expiration, strike, right, trading class, multiplier).
`InstrumentProviderMapping`	Maps an instrument to a data provider symbol (for example Finnhub) with a status **PENDING**, **VERIFIED**, **AMBIGUOUS**, **UNSUPPORTED** or **ERROR**. A provider symbol can be **VERIFIED** for only one instrument.

3.1.2 Search, resolution and qualification

Contract search (GET `/instruments/search/`) forwards to the gateway's `reqMatchingSymbols`, ranks exact stock matches first, and filters by asset class (`STK`, `IND`, `OPT`), currency and, for India, exchange (NSE, NFO, BSE, BFO, SMART).

`resolve_instrument` turns operator input into a canonical instrument with a qualified contract. If a conid is supplied, the contract is re-qualified *exactly*: the returned conid, security type, currency and, for options, expiration, strike, right, multiplier and trading class must all match. Otherwise

the instrument is found or created (an ambiguous ticker requires an exchange; the trading calendar is derived from the exchange or currency: NSE → XNSE, BSE → XBOM, otherwise XNYS) and qualified asynchronously through a gateway `QUALIFY` command whose completion event creates the contract.

`record_qualified_contract` atomically upserts the `BrokerContract`, creates the `OptionContract` for options (lot size = minimum size rounded up to the size increment; minimum tick from contract details), publishes the instrument to the Kafka registry and, if the contract identity changed, resets the Finnhub mapping to **PENDING**.

3.1.3 The instrument registry

Flink must map each raw event's conid to an instrument before it can do anything with it. The backend publishes each contract to `instrument.registry.v1` (key = conid) with its instrument id, symbols, asset class, exchanges, currency and `active = instrument.active and instrument.tradable`. A beat task re-publishes every contract every 60 seconds, so a recreated topic heals within a minute instead of causing every event to dead-letter as an unknown conid.

3.2 Market-data subscriptions and demand

Market data is *demand-driven and reference-counted*. A `MarketDataSubscription` exists per (gateway session, instrument, timeframe). Its `consumer_count` is the number of enabled strategy instances that need that series plus the number of unexpired *consumer leases*.

Strategy demand
An enabled instance counts directly. The required history is the maximum warm-up of its active input bindings plus `WARMUP_SAFETY_BARS` (5).

Leases
`MarketDataConsumerLease` rows carry time-bounded, non-strategy demand of type `MANUAL`, `WATCHLIST`, `MONITORING`, `RISK` or `STRATEGY`. The manual order ticket takes a 120-second `MANUAL` lease that the frontend refreshes while the ticket is open; option strategies take a 24-hour `STRATEGY` lease per leg.

`reconcile_market_subscription` is the single function that turns demand into gateway commands:

- **Count > 0.** If the subscription is currently served by the Finnhub fallback it is left alone. If it is already **SUBSCRIBING** or **ACTIVE** on the same gateway connection generation, it is reused (no command). Otherwise a new request id and a new *provider generation* (a UUID epoch) are minted, the state becomes **SUBSCRIBING**, and a `SUBSCRIBE_MARKET_DATA` command is sent with `historical_bars` equal to the required history.
- **Count $= 0$.** The subscription becomes **CANCELLING** and a cancel command is sent.
- **Gateway error.** The subscription becomes **ERROR**, and for subscribes the fallback logic is invoked.

A beat task (`restore_active_market_subscriptions`, every 15 s) re-issues subscriptions whose recorded gateway `connection_generation` differs from the current one — this is how the engine re-subscribes after an IB Gateway reconnect.

3.3 How the gateway produces market data

When a subscribe command reaches the gateway's broker worker:

1. The contract is qualified and the market-data type is set: `reqMarketDataType(3)` (delayed) in a paper session, `reqMarketDataType(1)` (real time) in a live session. Delayed data is refused outright in a live session.
2. **Warm-up.** Unless the request is a probe, historical bars are requested for the subscription timeframe, sized from the requested bar count (about 6.5 trading hours per day, converted to calendar days; requests over 365 days use IBKR's year unit). The last N bars are emitted with `source=ibkr_historical` and `processing_mode=WARMUP`.
3. **Live data.** In a live session `reqRealTimeBars` delivers 5-second bars (`source=ibkr_live`). In a paper session delayed ticks from `reqMktData` are bucketed locally into 5-second bars, stamped about 15 minutes behind (`source=ibkr_delayed`, with `received_at` and `delay_seconds`).
4. Every bar carries a stable `source_event_id` (conid, subscription key, timeframe, window start, provider generation). Errors on a market request remove the subscription and emit a `market.error` event.

The backend pulls these events from the gateway every 5 seconds (Chapter 10) and hands each `market.raw` event to the provider gate.

3.4 The provider gate

All raw market data, from IBKR or Finnhub, passes through one function, `apps/market_data/fallback.py::publish_provider_event`. It decides whether an event is trustworthy enough to enter the stream. Under a row lock on the subscription it drops the event with a counted reason when any of these hold:

Drop reason	Condition
`SUBSCRIPTION_MISSING`	No subscription for the session/instrument/timeframe key
`INSTRUMENT_MISMATCH`, `CONID_MISMATCH`	Exact contract identity differs
`GENERATION_MISSING`, `GENERATION_INVALID`	Missing or malformed provider epoch
`PROCESSING_MODE_INVALID`, `MARKET_DATA_TYPE_INVALID`	Unknown values
`TIMESTAMP_INVALID`	Unparseable time
`PROVIDER_MISMATCH`	Provider is not the subscription's active provider
`GENERATION_MISMATCH`	Epoch is not the subscription's current generation

Accepted events are normalised (historical sources become `WARMUP`; `ibkr_delayed` forces `DELAYED`; `received_at` becomes the backend ingress time, the trusted freshness clock) and written as an `OutboxEvent` on `market.raw.v1` with a deterministic event id. The outbox idempotency key for a live bar is derived from (instrument, timeframe, window start, mode), so a live window is published once regardless of which provider delivers it: the first writer wins.

Provider generation: the epoch gate

Each (re)subscription and each provider switch mints a new `provider_generation`. Only events stamped with the current generation of the current active provider are accepted. This makes it impossible for a late event from a cancelled IBKR request, or from Finnhub after failback, to leak into the stream.

3.5 Fallback to Finnhub and failback

Finnhub fallback is **disabled by default**: four switches (`MARKET_DATA_FALLBACK_ENABLED`, `FINNHUB_HISTORICAL_FALLBACK_ENABLED`, `FINNHUB_LIVE_FALLBACK_ENABLED`, `FINNHUB_AUTO_FAILBACK_ENABLED`) must be turned on. When enabled, the subscription's `fallback_state` follows the machine in Figure 3.1.

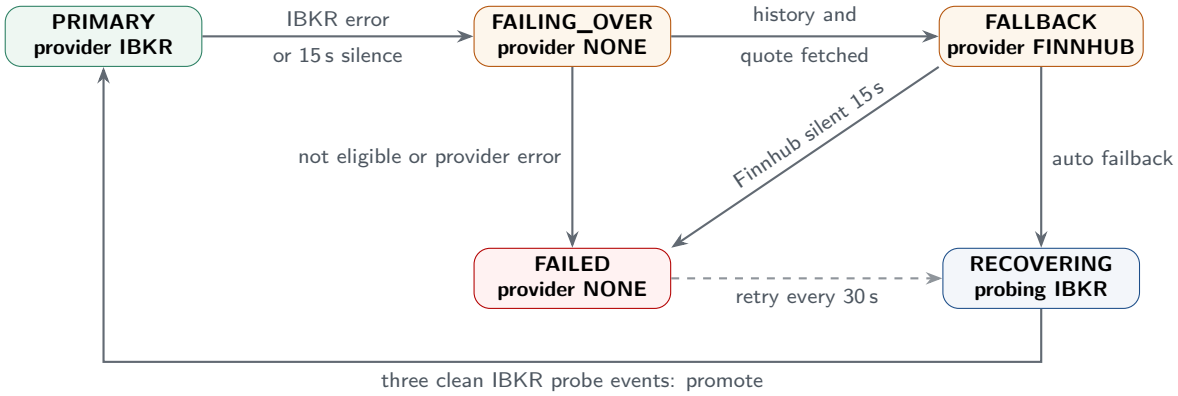


Figure 3.1: Market-data provider state machine for one subscription. Every transition is recorded in `MarketDataProviderTransition`.

- **Error classification.** IBKR errors are classified as disconnected (1100, 2110), entitlement (354, 10167, 10168), pacing (420, 100), timeout, no data, unsupported instrument (200, 321) or temporary. Unsupported instruments are marked unusable immediately; the other categories trigger failover.
- **Eligibility.** Failover requires the switches, an active tradable stock, a **VERIFIED** Finnhub mapping, and a subscription conid equal to the contract conid.
- **Failover.** A new generation starts with provider `NONE`. Outside the lock, Finnhub historical candles (for warm-up) and a quote are fetched. Back under the lock, and only if nothing changed meanwhile, warm-up outbox rows are written, the provider becomes `FINNHUB`, state **FALLBACK**, the market state is seeded from the quote, and strategies blocked for market-data reasons are released.
- **Live fallback feed.** The `finnhub-market-data` process subscribes to Finnhub trades over WebSocket for every symbol in fallback, aggregates them into UTC-aligned 5-second bars (deduplicating trades, counting late ones), and publishes them through the same provider gate.
- **Failback.** With auto failback enabled, a probe subscription is opened on IBKR with a new probe generation. When three live IBKR events with that generation arrive within the grace window, on a clean 5-second boundary that does not overlap the last published window, the subscription is promoted back to `IBKR`/**PRIMARY**.
- **Both providers down.** The subscription becomes **FAILED**; matching enabled strategies are **BLOCKED** with their prior state saved so they can resume later.

3.6 Delayed data and paper trading

Paper sessions read IBKR's delayed feed, so a paper account needs no real-time entitlement. The engine confines delayed data to paper execution with these rules:

1. The gateway allows delayed data only in paper mode, at configuration time and at subscription time.
2. Source `ibkr_delayed` always forces `market_data_type=DELAYED`, at backend ingress and again in Flink.
3. Freshness is measured from `received_at` (backend ingress), not from the delayed event time.
4. `InstrumentMarketState.is_execution_usable(at, mode)` requires a **FRESH** status, a positive price and an age within `stale_after_seconds` (300 s); rejects historical, warm-up, backfill and replay sources; and accepts `DELAYED` data only when the execution mode is `PAPER` and the source is `ibkr_delayed`.
5. Delayed bars are only routed to `PAPER` strategy instances for evaluation.

CHAPTER 4

The Streaming Pipeline

This chapter describes how raw 5-second bars become final, versioned bars at each strategy's timeframe, how indicators are computed, and how the results come back into PostgreSQL exactly once. It also covers the transactional outbox, dead letters and replay.

4.1 Kafka topics

Topics are created at startup by `scripts/ensure_kafka_topics.py` with `KAFKA_TOPIC_PARTITIONS` partitions (default 3) and replication factor 1. No per-topic retention or compaction is configured, so broker defaults apply.

Topic	Producer	Consumer	Key
`market.raw.v1`	Backend outbox (provider gate)	Flink normalisation	instrument id
`instrument.registry.v1`	Backend outbox	Flink normalisation (from earliest)	conid
`market.canonical.v1`	Flink normalisation	Flink bars and stale detection	none
`market.bars.v1`	Flink bar aggregation	Flink indicators; backend consumer	none
`market.indicators.v1`	Flink indicators	Backend consumer	none
`market.quality.v1`	Flink stale detection	Flink stream health; backend consumer	none
`strategy.inputs.v1`	Backend outbox (activation, pause, deletion)	Flink bars and indicators (broadcast, from earliest)	instrument id
`system.health.v1`	Flink stream health	(none)	none
`dead-letter.v1`	Flink normalisation	(inspection only)	none
`strategy.targets.v1`	Backend outbox	(audit stream)	aggregate id
`portfolio.rebalance.planned.v1`	Backend outbox	(audit stream)	aggregate id
`risk.decisions.v1`	Backend outbox	(audit stream)	aggregate id
`orders.events.v1`	Backend outbox	(audit stream)	aggregate id
`executions.events.v1`	Backend outbox	(audit stream)	aggregate id
`reconciliation.events.v1`	Backend outbox	(audit stream)	aggregate id

4.1.1 The event envelope

Every message is a JSON envelope (`assets/kafka/schemas/event-envelope-v1.json`) with required fields `event_id` (UUID), `event_type`, `schema_version` (1), `occurred_at`, `produced_at`,

`producer`, `aggregate_type`, `aggregate_id`, `correlation_id`, `causation_id`, `idempotency_key` and `payload`. The envelope is validated with JSON Schema on publish and on consume. Flink derives `event_id` deterministically as uuid5(NS, `event_type:stable_key`) and sets the causation id to the source event's id, so every derived fact can be traced to the raw event that caused it.

4.2 The transactional outbox

Domain code never talks to Kafka directly. It writes an `OutboxEvent` row (topic, partition key, type, aggregate, payload, correlation/causation ids, unique idempotency key) in the same transaction as the state change. The beat task `publish_outbox_events` (every 2 s):

1. Claims up to 100 rows in **PENDING**, **FAILED** or stale **PUBLISHING** status with `FOR UPDATE SKIP LOCKED`, marking them **PUBLISHING** with a 60-second lease.
2. Produces them with an idempotent producer (`enable.idempotence=true`, `acks=all`), keyed by partition key, and flushes once.
3. Marks confirmed rows **PUBLISHED**; failed rows become **FAILED** with exponential backoff $\min(300, 2^{\min(n,8)})$ seconds.
4. Writes the `outbox-publisher` heartbeat: HEALTHY, or DEGRADED if any row has failed. Execution readiness depends on this heartbeat.

Delivery is at-least-once; consumers deduplicate. Published rows are deleted after 30 days.

4.3 The Flink job

A single PyFlink job, `market-pipeline-v1` (`streaming/flink/jobs/market_pipeline.py`), contains five stages. The stages are still connected through Kafka topics rather than in-memory streams, so each stage keeps its own consumer group and can still be submitted on its own for debugging; running them as one job saves four TaskManager slots and four Python harnesses (about 1.26 GB of resident memory measured).

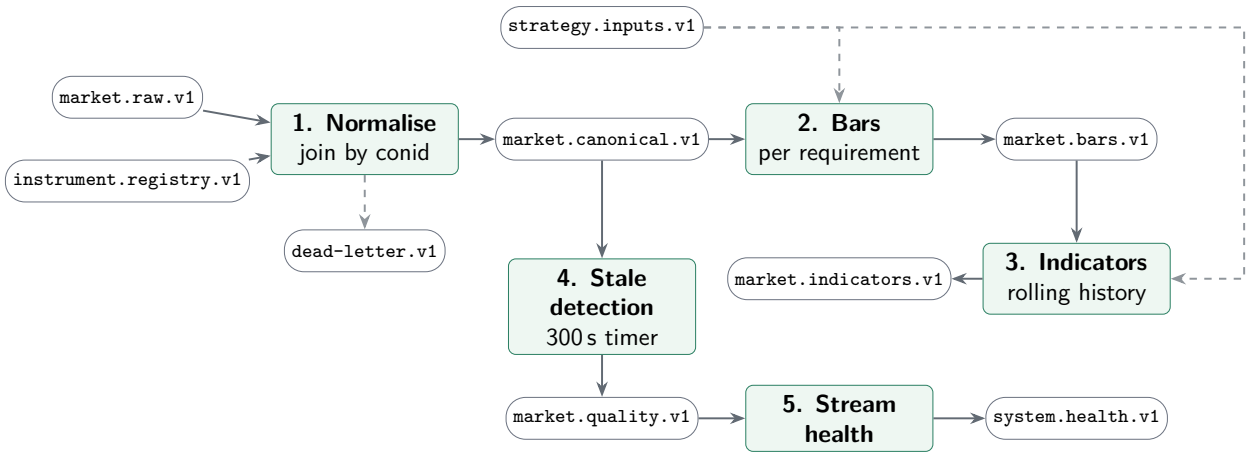


Figure 4.1: Stages of `market-pipeline-v1`. Dashed arrows are broadcast registry inputs or side outputs. The backend consumes bars, indicators and quality.

4.3.1 Stage 1: normalisation

Raw events are keyed by conid and joined with the instrument registry in a keyed co-process function.

- If the registry entry for the conid is known and active, the event is normalised immediately: the registry's instrument id overrides the raw one; required fields are checked; prices must be positive, volume non-negative and low $\leq$ high; times become UTC; `ibkr_delayed` forces `DELAYED`.
- If the conid is unknown, the event is buffered (up to 1000 per conid, deduplicated) for up to 30 seconds. A registry arrival flushes the buffer; a timeout dead-letters the events as `UNKNOWN_CONID_TIMEOUT`. This covers the startup race where market data arrives before its registry row.
- Output is deduplicated by event id (24-hour state TTL) and written to `market.canonical.v1`; invalid events go to `dead-letter.v1` with a reason code.

4.3.2 Stage 2: bar aggregation

Bar aggregation only produces what strategies asked for. It consumes `market.canonical.v1` together with the broadcast `strategy.inputs.v1` registry; each `BAR` requirement adds an (instrument, timeframe) pair and removals delete it.

- **Windows** are epoch-aligned in UTC: $start = t - (t \bmod s)$ for timeframe s seconds (`1m`, `5m`, `15m`, `1h`, `1d`). A daily window is UTC midnight to UTC midnight.
- **Pass-through.** A provider bar whose timeframe already equals the requirement (for example IBKR warm-up bars) becomes a final bar directly.
- **Aggregation.** Otherwise ticks are bucketed and an event-time timer fires at the window end. Watermarks allow 30 seconds of lateness, with 15 seconds of source idleness. Open = first, high = max, low = min, close = last, volume = sum; provenance comes from the last tick.
- **Versioning.** Each bar has a deterministic id bar_id = SHA256($instrument, timeframe, window_start$). A content fingerprint is kept; a late tick that changes the content produces version 2, 3, ... of the same bar. Unchanged content emits nothing.
- **Finality.** Only final bars are emitted. A window closes only when the watermark passes its end, i.e. when an event at least 30 seconds after the window end has arrived.

4.3.3 Stage 3: indicator computation

Indicators are computed per (instrument, interval) over a rolling history of final bars (up to 10,000, sorted by window end). `LIVE` and `WARMUP` bars share one history, so warm-up history feeds live indicators; `REPLAY` and `BACKFILL` have separate histories. A live ordering guard drops a bar if a higher version of the same bar was already accepted, or if it sorts before the newest accepted bar.

For each registered `INDICATOR` requirement the stage *recomputes the requirement's identity hash* and skips it unless it matches the registry key (an integrity check shared with the backend; see Section 4.4). It emits the value and the *previous value* (the same indicator computed on the history without the last bar), so that crossover rules can be evaluated from a single event.

Indicator	**Definition (N = `window`, default 20)**	**Null until**
`sma`	$\frac{1}{N}\sum_{i=0}^{N-1} C_{t-i}$	fewer than N bars
`rsi`	Simple-average (Cutler) RSI over the last N changes Δ_i: $G = \frac{1}{N}\sum \max(\Delta_i, 0)$, $L = \frac{1}{N}\sum \max(-\Delta_i, 0)$, RSI $= 100 - \frac{100}{1+G/L}$ (100 if $L = 0$). Not Wilder-smoothed.	$\leq N$ closes

Indicator	Definition (N = `window`, default 20)	Null until
`donchian`	Prior bars only (excludes the current bar). Role `entry`/upper: $\max(H_{t-N..t-1})$; role `exit`/lower: $\min(L_{t-N..t-1})$	fewer than N prior bars
`momentum`	$C_t/C_{t-N} - 1$	$\leq N$ closes
`realized_volatility`	Sample standard deviation of simple returns over the last N intervals, $\times\sqrt{252}$ (applied whatever the timeframe)	fewer than 2 returns
`average_volume`	$\frac{1}{N}\sum V$ over the last N bars (used for ADV sizing)	fewer than N bars
`weekly_close`	Close of the last completed ISO week	—
`weekly_atr`	Wilder ATR over completed weeks, $\text{ATR} \leftarrow \alpha\,\text{TR} + (1-\alpha)\text{ATR}$, $\alpha = 1/N$	fewer than N weeks
`reference_price`	C_t	—

Indicator output names are `name_role` when a role is set, except Donchian, which maps to `donchian_upper` (role `entry`) or `donchian_lower`. EMA, ADX, Bollinger Bands and MACD are *not* part of the streaming catalogue (the options model computes its own daily ADX and Bollinger width in Python; see Chapter 7).

4.3.4 Stages 4 and 5: stale detection and stream health

Every canonical event emits a `market.quality` event with status **FRESH**, the reference price and `latest_event_at` (the backend ingress time for delayed data, the event time otherwise). For live processing a processing-time timer is reset to 300 seconds on each event; if it fires, a **STALE** event is emitted. The backend upserts `InstrumentMarketState` from these events. Stage 5 maps each quality event to a `system.health.v1` record.

4.3.5 Checkpoints, runtime and bootstrap

The job checkpoints every 30 seconds (timeout 120 s). Because the backend and the Flink TaskManagers share no filesystem, the backend image builds a relocatable CPython 3.11 with PyFlink 1.20.1 into a reproducible ZIP (`/opt/flink-python-runtime/flink-python-runtime.zip`, SHA-256 pinned) and ships it with every job graph. `scripts/ensure_flink_jobs.py` keeps the job running: it takes a PostgreSQL advisory lock, cancels superseded per-stage jobs, waits for transitional jobs, resubmits terminal jobs from their latest retained checkpoint, waits for the first completed checkpoint, and repairs only one known failure cause (a missing Python worker executable); for any unknown restart cause it refuses to act.

4.4 Deterministic identity

Backend and Flink share one UUID namespace and the same canonical-JSON SHA-256 helpers, so both sides compute identical identities:

Identity	Hashed fields
raw event key	provider, provider generation, source event id
canonical event key	raw event id, instrument id
bar id	instrument id, timeframe, window start
bar event key	bar id, version
requirement identity hash	implementation version, indicator name and role, input type, instrument id, parameters, timeframe
indicator event key (`source_key`)	bar id, bar version, implementation version, requirement identity hash
quality event key	source event id, status

Re-emitting the same fact therefore yields the same event id, which the backend's consumed-event ledger turns into a no-op. A change to any indicator parameter, role or implementation version produces a new identity, so old values are never reinterpreted.

4.5 Persistence into PostgreSQL

The `consume_market_streams` process reads `market.bars.v1`, `market.indicators.v1` and `market.quality.v1` with auto-commit disabled. Each message is processed through `consume_once`, which inserts a `ConsumedEvent(consumer_name, event_id)` row in the same transaction as the handler; a duplicate returns immediately. The Kafka offset is committed only after the PostgreSQL commit.

- **Bars** (`persist_bar`) are inserted by (bar id, version). An existing row must match every field exactly, or the event is dead-lettered as a conflicting fact. A live delivery of a previously warm-up or replayed bar promotes it to `LIVE`. New final bars advance strategy warm-up; live bars schedule strategy evaluation.
- **Indicators** (`persist_indicator`) are inserted by `source_key`, linked to their bar, and may promote a waiting evaluation job once the last expected indicator lands.
- **Quality** (`persist_quality`) upserts `InstrumentMarketState` (live only) and resumes option evaluation jobs that were waiting for quotes.

Error policy. Transient errors (database or network) are raised without committing, so the process restarts and re-reads the message. Deterministic errors (bad JSON, schema, integrity conflicts) are written to the `DeadLetterEvent` table and then committed.

4.6 Dead letters and replay

- The Flink `dead-letter.v1` topic holds raw events that failed normalisation; it is for inspection only.
- The backend `DeadLetterEvent` table holds events the persistence consumer rejected. `manage.py replay_market_dlq` re-runs them in `REPLAY` mode.
- POST `/streaming/replay/` replays a time range of one of the three market output topics through a throwaway consumer group, forcing `REPLAY` mode.

> **Replay can never trade**
>
> `REPLAY`, `WARMUP` and `BACKFILL` facts are persisted but can never move live strategy state or

> create targets. Only final `LIVE` bars schedule evaluation.

PART III

Automatic Strategy Execution

CHAPTER 5

The Strategy Framework

The strategy framework is shared by stocks and options. It defines how a strategy declares what data it needs, how an instance is activated and warmed up, how a completed bar becomes a durable evaluation job, and how a decision becomes a target. Chapters 6 and 7 then describe the concrete stock and option strategies.

5.1 Definitions, instances and versions

Three layers

A **definition** is *what* a strategy is (a plugin class and its parameter schema). An **instance** is *where* it runs (portfolio, instrument, timeframe, parameters, lifecycle state). A **version** is an immutable snapshot of the instance's configuration; every material change creates a new version, and every run, target and job is tied to the version it was computed under.

Model	Key fields and semantics
`StrategyDefinition`	Unique upper-case `key`, `plugin_path`, JSON Schema `parameter_schema`, supported asset types, directions and timeframes, `enabled`. Fourteen are seeded: five stock plugins, eight option templates and the RPS options auto selector.
`StrategyInstance`	Definition, portfolio, instrument, timeframe, merged parameters, `target_configuration` (target weight, capital share, priority), risk and order policy, `execution_mode` (always derived from the portfolio's session), `allocated_capital`, `kill_switch`, `state`, `enabled`, `version`, activation stage timestamps, plugin-private `state_data`, `block_reason`, and a monotonic market cursor (last bar time, id and version) used to reject out-of-order bars.
`StrategyVersion`	(instance, version) unique; `configuration_snapshot` and its SHA-256 `parameter_hash`; `activated_at`, `retired_at`. Saving an existing row raises: versions are immutable.
`StrategyInputRequirement`	A shared, content-addressed demand for one input: identity hash (unique), instrument, timeframe, `BAR` or `INDICATOR`, name, role, parameters, implementation version, `warmup_bars`, `active_ref_count`.
`StrategyInputBinding`	Links an (instance, version) to a requirement, with an `active` flag.
`StrategyWarmupReadiness`	Immutable evidence that warm-up completed: provider, generation, requirement hashes, bar ids and timestamps, an evidence hash.

Model	Key fields and semantics
`StrategyRun`	One evaluation (or operator flatten). Unique `idempotency_key strategy:{instance}:v{version}:{instrument}:{timeframe}:{event}:{data_version}`, input hash, configuration and context snapshots, status.
`StrategySignal`	One per run: `ENTER_LONG`, `EXIT_LONG`, `HOLD`, `NO_ACTION`, `SET_TARGET`, `ENTER_OPTIONS`, `EXIT_OPTIONS`, `HOLD_OPTIONS`; reason, details.
`StrategyTarget`	`target_type` `WEIGHT`, `VALUE`, `QUANTITY` or `FLAT`; weight, value or quantity; direction; signal time; execution mode; metadata; status **ACTIVE**. Written only when the target actually changes.
`StrategyAttributedPosition`	The strategy's share of a portfolio position.
`StrategyAllocation`	Capital share, minimum/maximum share, capacity, priority.
`StrategyAction`	Durable, idempotent operator action (enable, pause, flatten).
`OrderPolicy`	Order type (default `LMT`), time in force (`DAY`), limit offset in basis points, and further replace/collar fields.
`StrategyRiskPolicy`	Maximum weight (0.10), notional (100,000), quantity (100,000), `allow_short` (false), and per-lifecycle behaviours (default HOLD).

5.2 The plugin contract

Every strategy is a subclass of `StrategyPlugin` (apps/strategies/plugins/base.py):

```
class StrategyPlugin:
    key, name, description
    supported_asset_types = ("STK",)
    supported_directions  = ("LONG",)
    supported_timeframes  = ("1m", "5m", "15m", "1h", "1d")
    parameter_schema      = {"type": "object"}      # JSON Schema 2020-12
    default_parameters    = {}
    execution_model       = "SINGLE_INSTRUMENT_WEIGHT"   # or MULTI_LEG_QUANTITY

    def validate_configuration(self, parameters) -> dict   # merge defaults, validate
    def required_stream_inputs(self, parameters) -> list[StreamInput]
    def evaluate(self, context: EvaluationContext) -> StrategyDecision
    def build_targets(self, context, decision) -> list[target]
```

- `StreamInput` declares an input: type (`BAR` or `INDICATOR`), name, parameters, role, implementation version and the number of warm-up bars it needs.
- `EvaluationContext` is immutable and built only from persisted facts: the bar, current indicator values, previous indicator values, previous state, plugin `state_data`, attributed positions, active orders, portfolio state and event metadata.
- `StrategyDecision` carries a signal type, a desired weight (`None` means "no new target"), direction, reason, next state, confidence, new `state_data`, and for multi-leg strategies a tuple of per-leg quantity targets.
- `build_targets`: a weight of `None` produces no target, 0 produces a `FLAT` target, anything else a `WEIGHT` target. Multi-leg decisions produce `QUANTITY` (or `FLAT`) targets per leg.

Plugins never compute indicators

A plugin only reads indicator values that the streaming pipeline computed and the backend persisted. This guarantees that two strategies needing "SMA(20) on AAPL 5m" see the same number, computed once.

5.3 Input identity and shared requirements

When an instance is activated, each declared input becomes a `StrategyInputRequirement` whose identity is

$$h = \text{SHA256}(\text{canonical JSON}(\text{requirement fields})),$$

where the fields are the implementation version, indicator name and role, input type, instrument id, parameters and timeframe. The same function exists in the Flink job, which recomputes the hash and ignores any registry entry that does not match. Two instances with identical inputs share one requirement (`active_ref_count` counts active bindings). On activation, the full requirement list is published on `strategy.inputs.v1`; on pause or deletion, the hashes whose reference count dropped to zero are published as removals so Flink stops computing them.

When `EXECUTION_REGISTER_ADV_INPUT` is true (the default), the framework also registers an `average_volume` indicator with window `EXECUTION_AVERAGE_VOLUME_WINDOW` (20) for every strategy. Position sizing uses it as the average daily volume (ADV); without a fresh value the liquidity limit is zero and orders are rejected. In practice this makes the minimum warm-up about 20 bars for every strategy.

5.4 Activation lifecycle

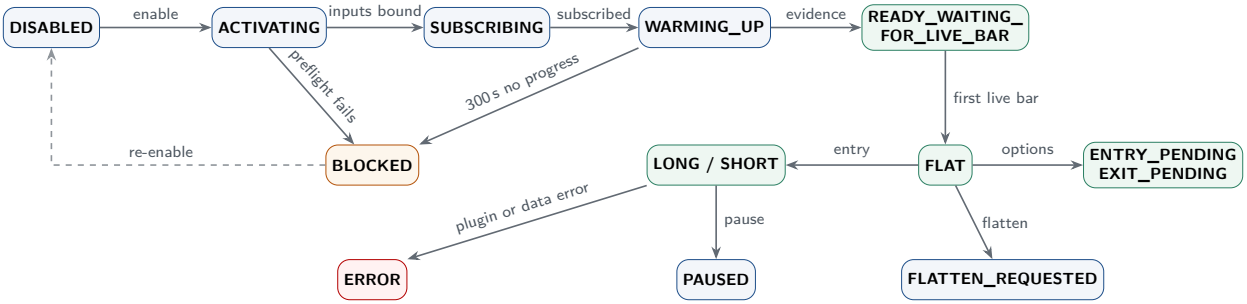


Figure 5.1: Strategy instance lifecycle. Green states are evaluation-ready: every final live bar creates an evaluation job. Warm-up timeouts are recoverable; plugin, configuration and data integrity failures move the instance to ERROR.

5.4.1 Create

`create_instance` validates that the timeframe is supported, merges and validates the parameters against the JSON Schema, optionally resolves and qualifies the instrument through the portfolio's gateway, derives the execution mode from the portfolio's session, and in one transaction creates the instance (**DISABLED**, not enabled), version 1, a `StrategyAllocation`, and inactive input bindings. Nothing is published and no market data is requested until activation. The API accepts `activate: true` to create and enable in one call.

5.4.2 Update

A change to the definition, instrument, timeframe, parameters, target configuration, policies or execution mode retires the current version (deactivating its inputs), increments the version, resets the activation stage timestamps and registers inputs for the new version. An enabled instance goes back to **WARMING_UP**.

5.4.3 Enable and preflight

POST `/strategy-instances/<id>/enable/` requires an `Idempotency-Key`. A repeated request returns the same `StrategyAction`: 202 while processing, the stored result once completed, and a failed action can only be retried with `Idempotency-Retry: true` if it was marked retryable. `enable_instance` runs `_validate_activation`:

1. The requested mode matches the portfolio session, and LIVE requires `ALLOW_LIVE_TRADING`.
2. Kafka is enabled (automatic execution needs streaming).
3. Execution readiness has no blockers (Section 8.10). The strategy's own `MARKET_DATA_STALE` blocker is excused, because otherwise a cold start would deadlock: enabling needs fresh data and only enabling produces data.
4. The instrument has a qualified contract with a conid.
5. The portfolio's session is **CONNECTED**, command-enabled, reports connected, and was checked within the last 30 seconds.
6. The gateway reports reconciled, the account is reconciled, and there are no unresolved material reconciliation breaks.
7. The account is available through the session, the definition is enabled, and the configuration re-validates.

Any failure puts the instance into **BLOCKED** with a reason and marks the action failed, with a retryable flag. On success the instance moves to **ACTIVATING** and the Celery task `activate_strategy_instance` is queued on commit.

5.4.4 Activate, subscribe, warm up

The worker re-validates, sets `enabled=true`, stamps a fresh `effective_from` on first activation, registers the inputs (publishing them on `strategy.inputs.v1`), checks that the number of active bindings matches the plugin's declared inputs, moves to **SUBSCRIBING**, and reconciles the market-data subscription. No database lock is held during gateway I/O.

When the gateway confirms the subscription, `refresh_strategy_warmup_state` runs the warm-up state machine:

- *required* is the maximum warm-up over the current version's active bindings.
- It selects up to *required* + 5 distinct final `WARMUP` bars from the current provider generation (or received within an hour of the request from the same provider).
- Warm-up is complete when at least *required* bars exist *and*, scanning from the newest, a bar has non-null warm-up values for every declared indicator.
- The instance then moves to **READY_WAITING_FOR_LIVE_BAR**, a `StrategyWarmupReadiness` evidence row is written, and any live bar that arrived during warm-up is scheduled immediately.

READY_WAITING_FOR_LIVE_BAR is a durable wait with no timeout: markets may be closed. By contrast, an instance stuck in **ACTIVATING**, **SUBSCRIBING** or **WARMING_UP** with no progress for 300 seconds is **BLOCKED** with a precise diagnosis (no subscription, command pending, subscription failed, Kafka publication failed, no raw event, no canonical event, no final bar, indicators missing). Warm-up timeouts are recoverable: later progress unblocks the instance.

The enable action stays **PROCESSING** until the *first persisted live evaluation*: subscribing is not the same as trading.

5.4.5 Pause, flatten, delete

Pause
Disables the instance, deactivates its inputs (publishing removals), marks the portfolio for target coordination, and decrements the subscription's consumer count. Existing positions are kept (the default lifecycle behaviour for a paused strategy is HOLD).

Flatten
Creates a run with a `FLAT` target (quantity 0, operator exit) for every attributed instrument, moves to **FLATTEN_REQUESTED**, and marks the portfolio for coordination. When a rebalance completes with no attributed position left, the state becomes **FLAT**.

Delete
A guarded hard delete that requires the exact strategy name. It is refused with `STRATEGY_DELETION_BLOCKED` while there are open orders, active executions, pending rebalances or non-zero attributed positions. On success, financial history (order intents, attributions, capital snapshots, allocation decisions) is preserved with a strategy identity snapshot; mutable records are deleted; input removals are published; and the subscription is reconciled.

5.5 Evaluation jobs

5.5.1 From bar to job

Each time a final `LIVE` bar or one of its indicators is persisted, `coordinate_bar_readiness` finds enabled, evaluation-ready instances on that instrument and timeframe (delayed bars only for `PAPER` instances) and calls `ensure_strategy_evaluation_job`. There is exactly one job per (instance, version, bar id, bar version):

- If every expected indicator identity for that bar is present, the job is **PENDING**; otherwise it is **WAITING_FOR_INPUT**. Whichever arrives last, bar or indicator, promotes the same job.
- The instance's market cursor must advance monotonically on (window end, bar id, version). An older or non-live bar turns a waiting job into **FAILED** with `STALE_INPUT`.
- A corrected bar (version 2) gets new indicator identities and a new job.

5.5.2 Job states and failures

Error code	Retryable	Effect
`INFRASTRUCTURE_RETRYABLE`	yes	**RETRY** with backoff $\min(300, 5 \cdot 2^{n-1})$ s; **FAILED** after 5 attempts
`MISSING_INPUT`	—	Back to **WAITING_FOR_INPUT**; a durable wait with no attempt limit (used by options while waiting for quotes)

Error code	Retryable	Effect
`STALE_INPUT`	no	**FAILED**; the instance is not harmed
`INVALID_CONFIGURATION`	no	**FAILED**; instance → **ERROR**
`PLUGIN_LOGIC_ERROR`	no	**FAILED**; instance → **ERROR**
`DATA_INTEGRITY_ERROR`	no	**FAILED**; instance → **ERROR**

A beat task (every second) claims up to 50 jobs in **PENDING** or due **RETRY** with `FOR UPDATE SKIP LOCKED`. Before calling the plugin, the worker re-checks everything from the database: the instance is enabled and evaluation-ready on the job's version; the bar is final, live and matches; delayed data only for paper; the active binding identities still equal the job's expected set; the configuration validates; and every indicator value exists with an event time equal to the bar's window end.

A job claimed for more than 300 seconds is recovered to **RETRY**. Because the run is keyed by its idempotency key, a crash after the run was committed but before the job was marked complete simply finds and reuses the existing run: exactly one run per bar.

5.5.3 From decision to target

`evaluate_instance` runs in one transaction under a row lock on the instance:

1. Refuse non-final bars, non-live processing modes and delayed data for non-paper instances.
2. Return an existing run with the same idempotency key, if any.
3. Create a `StrategyRun` with a snapshot of the bar, indicators, previous indicators and prior state (which enables a safe retry).
4. Build the `EvaluationContext` (attributed positions, active orders) and call the plugin.
5. Persist the `StrategySignal`. For each target from `build_targets`, persist a new `StrategyTarget` *only if* its type, weight or quantity differs from the latest active target for that instrument. A `HOLD` writes no target; the previous target stays authoritative.
6. Set the instance state to the decision's next state and store its `state_data`.
7. Write a `strategy.targets.v1` outbox event (audit) and mark the portfolio for target coordination (debounced, Chapter 8).

A database or connection error rolls everything back and the job retries. Any other exception marks the run and the instance **ERROR**.

5.6 Reporting

Everything shown in the UI is derived from persisted facts:

- **Ledger**: one row per run, newest first, with the bar close, signal, decision (ENTERED, EXITED, HOLDING, TARGET_SET, NO_TRADE, ORDERS_WORKING, ERROR), reasons, attached orders and, for options, the full decision evidence. Jobs without a run appear as "bar not evaluated".
- **P&L**: fills attributed to the strategy (*allocated qty* × *fill qty*/*order qty*, commission pro rata), weighted-average-cost accounting (realised $= (p - \bar{c}) \cdot q_{\text{closed}} \cdot \text{dir} \cdot m$), marks from the current market state, returning realised, unrealised, commissions and a running series.

- **Chart**: the last 500 final bars, indicators, signal/target/order/fill markers, entry and exit trade markers, and (options) the daily ATR bands.
- **Execution timeline**: a 16-stage read model from CREATED through ACTIVATION_QUEUED, SUBSCRIPTION_ACTIVE, WARMUP_COMPLETE, FIRST_EVALUATION_COMPLETED, TARGET_GENERATED, ORDER_INTENT_CREATED, RISK_DECISION, BROKER_COMMAND_SENT and ORDER_ACKNOWLEDGED to FILL_PROGRESS, with the blocker if one stage failed.

CHAPTER 6

Automatic Stock Strategies

Five built-in stock plugins are registered in `apps/strategies/plugins/builtin.py`. All of them support the timeframes `1m`, `5m`, `15m`, `1h` and `1d`, trade asset class `STK`, and produce a *target weight* of the portfolio (the `SINGLE_INSTRUMENT_WEIGHT` execution model). This chapter defines each one exactly.

6.1 Conventions shared by all stock plugins

Entry weight
The weight a plugin targets on entry is $w^* =$ `target_configuration.target_weight`, falling back to `parameters.target_weight`, then 0.

Held
A position is considered held when the previous state is **LONG**, **PARTIALLY_LONG** or **ENTRY_PENDING**.

Warm-up guard
If any indicator the rule needs is still null, the plugin returns `NO_ACTION` with next state **WARMING_UP**; the instance returns to **READY_WAITING_FOR_LIVE_BAR** automatically once the indicators are available.

Crossing rules
With p the previous and c the current value of an indicator and ℓ a level:

$$\textsc{CrossAbove} : p \le \ell < c, \quad \textsc{CrossBelow} : p \ge \ell > c, \quad \textsc{Above} : c > \ell, \quad \textsc{Below} : c < \ell.$$

Crossings are evaluated from one event, because every indicator value carries its previous value (Chapter 4).

Targets are written only on change
A `HOLD` decision produces no new target; the previous target remains in force for rebalancing.

Direction
RSI, SMA and Donchian accept `SHORT/BOTH` in their schemas but only emit long entries and exits. Volatility-target momentum can produce negative weights.

6.2 RSI mean reversion

Strategy key: `RSI_MEAN_REVERSION`.

> **Idea**
> Buy when momentum recovers from an oversold condition; sell when it has recovered enough.

Parameter	Default	Constraint
`window`	14	integer ≥ 2
`entry_threshold`	30	$[0, 100]$, strictly less than exit threshold
`exit_threshold`	65	$[0, 100]$
`entry_rule`, `exit_rule`	`CROSS_ABOVE`	CROSS_ABOVE, CROSS_BELOW, BELOW, ABOVE
`direction`	`LONG`	LONG, SHORT, BOTH (only long is traded)

Inputs. `BAR` (warm-up $n+1$) and `rsi(window=n)` (warm-up $n+1$), where RSI is the simple-average RSI of Chapter 4.

Rule. Let r_t and r_{t-1} be the current and previous RSI.

- Held and `exit_rule`$(r_{t-1}, r_t, \mathit{exit}) \Rightarrow$ `EXIT_LONG`, weight 0, next state **FLAT**.
- Not held and `entry_rule`$(r_{t-1}, r_t, \mathit{entry}) \Rightarrow$ `ENTER_LONG`, weight w^*, next state **LONG**.
- Otherwise `HOLD`.

With defaults, the strategy enters when RSI crosses up through 30 and exits when RSI crosses up through 65.

6.3 SMA crossover

Strategy key: `SMA_CROSSOVER`.

> **Idea**
> Follow the trend: be long while the fast moving average is above the slow one.

Parameter	Default	Constraint
`fast_window`	20	integer ≥ 2, strictly less than slow
`slow_window`	50	integer ≥ 3
`entry_rule` / `exit_rule`	CROSS_ABOVE / CROSS_BELOW	fixed

Inputs. `BAR` (warm-up = slow), `sma(window=fast, role=fast)` $\rightarrow$ `sma_fast`, `sma(window=slow, role=slow)` $\rightarrow$ `sma_slow`.

Rule. With f, s the current and f', s' the previous fast and slow SMAs:

$$\mathit{crossed_up} = (f' \leq s') \wedge (f > s), \qquad \mathit{crossed_down} = (f' \geq s') \wedge (f < s).$$

Held and crossed down $\Rightarrow$ `EXIT_LONG` (weight 0); not held and crossed up $\Rightarrow$ `ENTER_LONG` (weight w^*); otherwise `HOLD`. Because the previous slow SMA needs one more bar than the declared warm-up, the first bar after a minimal warm-up may return `NO_ACTION`.

6.4 Donchian breakout

Strategy key: `DONCHIAN_BREAKOUT`.

> **Idea**
> Buy a breakout above the highest high of the last N bars; exit on a break below the lowest low of the last M bars.

Parameters. `entry_window` 20, `exit_window` 10 (integers ≥ 2).

Inputs. `BAR`; `donchian(window=entry, role=entry)` → `donchian_upper`; `donchian(window=exit, role=exit)` → `donchian_lower`; warm-up $\max(entry, exit) + 1$.

Rule. The channel is computed over *prior* bars, excluding the current bar, so a close above it is a genuine breakout:

$$U_t = \max_{i=1..N} H_{t-i}, \qquad L_t = \min_{i=1..M} L_{t-i}.$$

Held and $C_t < L_t \Rightarrow$ `EXIT_LONG`; not held and $C_t > U_t \Rightarrow$ `ENTER_LONG`; otherwise `HOLD` ("price remains inside channel").

6.5 Volatility-target momentum

Strategy key: `VOLATILITY_TARGET_MOMENTUM`.

> **Idea**
> Size the position so that its volatility matches a target, in the direction of recent momentum.

Parameter	Default	Constraint
`momentum_window`	20	integer ≥ 2
`volatility_window`	20	integer ≥ 2
`target_volatility`	0.10	> 0
`maximum_weight`	0.20	$(0, 1]$
`direction`	`BOTH`	LONG, SHORT, BOTH

Inputs. `BAR`, `momentum(window=m)`, `realized_volatility(window=v)`; warm-up $\max(m, v) + 1$.

Rule (every bar). With momentum $\mu_t = C_t / C_{t-m} - 1$ and annualised realised volatility σ_t:

$$w_t = \begin{cases} 0 & \text{if } \sigma_t \leq 0 \text{ or } \mu_t = 0, \\ \mathrm{clip}\left(\mathrm{sign}(\mu_t)\,\dfrac{\sigma^{\text{target}}}{\sigma_t},\ -w_{\max},\ w_{\max}\right) & \text{otherwise,} \end{cases}$$

and $w_t \leftarrow \max(0, w_t)$ if the direction is LONG. The plugin returns `SET_TARGET` with weight w_t and next state **LONG**, **SHORT** or **FLAT** by the sign of w_t. Example: momentum 0.1, volatility 0.1 and target 0.2 give $w = 2.0$, capped to the maximum weight.

6.6 Fixed-weight rebalance

Strategy key: `FIXED_WEIGHT_REBALANCE`.

Every bar returns `SET_TARGET` with weight w^* (negative weights give a short direction). Because targets are only written when they change, this produces one target at activation and re-targets only when the configured weight changes; the rebalancer keeps the position at that weight as prices drift, subject to its drift threshold. Several fixed-weight strategies on one instrument net together (for example $+0.05$ and -0.02 net to 0.03).

6.7 From target weight to shares

A stock plugin's output is a weight, not a quantity. The conversion happens during target coordination and rebalance planning (Chapter 8):

$$w_i^{\text{eff}} = w_i \cdot \frac{K_s}{NAV}, \qquad q_i^{\text{desired}} = \frac{w_i^{\text{eff}} \cdot NAV}{P_i \cdot m_i},$$

where K_s is the strategy's capital (its `allocated_capital` or $NAV\times$ its allocation weight), P_i is a fresh, execution-usable reference price and m_i the multiplier. The rebalancer then rounds to the lot size, applies drift, minimum notional, cost and cash constraints, and sizing and risk cap the order.

6.8 Catalogue strategies from research

Strategies promoted from the research system (Chapter 11) run through a generic adapter, `CatalogueLongOnlyPlugin`. It declares a `BAR` input and an indicator `research_signal`, clamps the signal to $[0, 1]$ and emits `SET_TARGET` $\min(signal, w^*)$ or `EXIT_LONG`. The streaming pipeline does not yet compute `research_signal` (see Chapter 15).

6.9 Stock strategy parameters at a glance

Strategy key	Inputs	Entry / target	Exit
`RSI_MEAN_REVERSION`	RSI(14)	RSI crosses above 30	RSI crosses above 65
`SMA_CROSSOVER`	SMA(20), SMA(50)	fast crosses above slow	fast crosses below slow
`DONCHIAN_BREAKOUT`	Donchian(20), Donchian(10)	close $>$ prior 20-bar high	close $<$ prior 10-bar low
`VOLATILITY_TARGET_MOMENTUM`	momentum(20), realised vol(20)	$\text{sign}(\mu)\, 0.10/\sigma$, $\lvert w\rvert \le 0.20$	continuous re-targeting
`FIXED_WEIGHT_REBALANCE`	bar	constant w^*	change of w^*

CHAPTER 7

Automatic Options Strategies

The options engine selects, enters, holds and exits defined-risk option structures on an underlying (for example the NSE NIFTY 50 index) without human intervention. It is a port of the *Aryan Options* decision engines: payoff/POP, strategy selection, range entry and exit policy are carried over with thresholds and weights unchanged, and the RPS score, entry signal and dynamic exit are adapted to this runtime's data. This chapter explains the concepts, the full decision funnel with every formula and default, and how a chosen structure is executed leg by leg through IBKR.

7.1 Concepts

What the engine is trying to do

Most structures here are *short premium, defined risk*: sell options whose strikes lie outside the range the underlying is expected to stay in, and buy further-out options as protection. The trade earns the net credit if the underlying settles inside the short strikes; the long wings bound the worst case. The engine therefore asks four questions in order:

1. **Is the market range-bound?** (the RPS score)
2. **Is option premium rich relative to realised movement, and is a breach of this structure's boundaries unlikely?** (the IV/breach entry gate)
3. **Is the trade's risk bounded and its expectancy positive?** (POP, EV and loss filters)
4. **Which candidate structure has the best risk-adjusted quality?** (hard gates, TQS and the risk-adjusted winner)

Open interest never selects a strike

Strikes come from each template's fixed offsets from the at-the-money strike. Open interest is only an input to the RPS score and an independent confirmation level.

7.2 Structures (templates)

Templates are defined in `apps/strategies/option_templates.py`. An *offset* is an index shift on the ladder of priced strikes of that right; offset 0 is the priced strike nearest spot. Every leg has ratio 1.

Template key	Legs (side, right, offset)	Notes
`PUT_CREDIT_SPREAD`	BUY P −2; SELL P −1	Bullish/neutral; lower side at risk
`CALL_CREDIT_SPREAD`	SELL C +1; BUY C +2	Bearish/neutral; upper side at risk

Template key	Legs (side, right, offset)	Notes
IRON_CONDOR	BUY P −3; SELL P −2; SELL C +2; BUY C +3	Short strikes widened from ±1 to ±2 so that POP clears the 45% family floor
IRON_BUTTERFLY	BUY P −2; SELL P 0; SELL C 0; BUY C +2	Both shorts at the money
CASH_SECURED_PUT	SELL P −1	Collateral = strike × multiplier
DELTA_PUT_CREDIT_SPREAD	SELL P nearest $\Delta = -0.20$; BUY P nearest (short strike −100)	DTE 20–45, hold 10 days
LONG_ATM_STRADDLE	BUY C 0; BUY P 0	Long volatility
CALENDAR_CALL	BUY C 0 far expiry; SELL C 0 near expiry	Near DTE 1–24, far DTE 25–90
RPS_OPTIONS_AUTO_SELECTOR	Chooses among PCS, CCS, IC, IB	The default for the options API

Only four structures can currently be entered. The approved selector accepts only PUT_CREDIT_SPREAD, CALL_CREDIT_SPREAD, IRON_CONDOR and IRON_BUTTERFLY. The other four templates are seeded and supported downstream (collateral, calendar hedge coverage, plugin logic) but a fixed-template instance of them always ends in NO_ELIGIBLE_STRUCTURE.

Expiry. Candidate expiries satisfy $entry_min_dte \le (expiry - date) \le entry_max_dte$ (defaults 1 and 45); the nearest is used. After offsets are applied, every leg must resolve, legs must be distinct, and each leg must be liquid: price > 0, open interest ≥ 100, volume ≥ 1. A thin strike rejects the structure rather than shifting the offsets.

7.3 The decision funnel

7.4 Stage 0: preconditions and data

7.4.1 Entry window

The entry window reproduces the source's gate: with t the bar's window end in rps_market_timezone (default Asia/Kolkata), entry is allowed only when $t \ge open + delay$, where rps_market_open_time is 09:15 and rps_entry_delay_minutes is 0. The check runs *before* any IBKR request, so intraday timeframes spend no gateway calls on bars before the window. Timeframes 1m to 1d are accepted; the RPS score is timeframe-independent by construction (one observation per completed daily session), so the execution timeframe only decides how often the window is sampled.

7.4.2 Option chain and quotes from IBKR

OPTION_CHAIN_PROVIDER=IBKR is the only provider that can authorise an entry (the legacy Dhan provider has no contract multiplier or lot size and returns UNSUPPORTED_PROVIDER).

1. The backend requires a fresh underlying price, picks the chain definition (trading class equal to the symbol, then exchange, then SMART), and takes the option_strike_count (default 21, range 5–21) strikes nearest spot, both rights: up to 42 contracts in one OPTION_QUOTES command (45 s timeout).

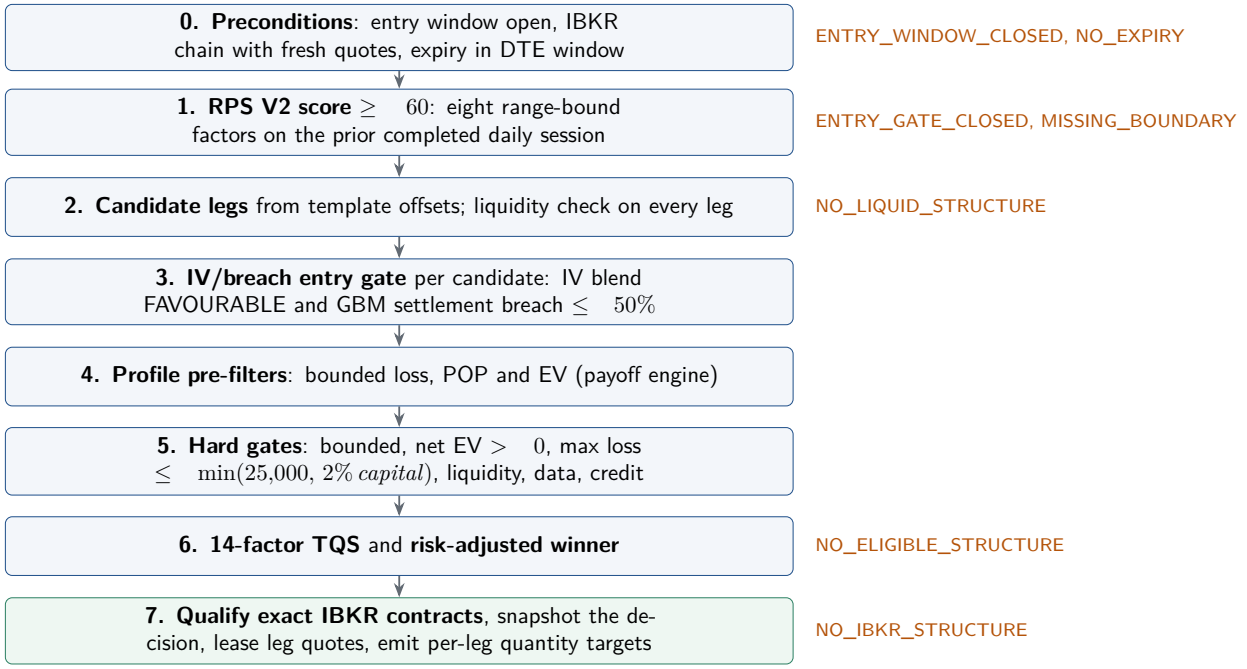


Figure 7.1: Order of the options entry decision. Each stage must pass; the refusal status on the right is recorded in the evaluation's evidence.

2. The gateway rejects a request whose mode differs from its trading mode, resolves contract details for all contracts concurrently, requests market data with generic ticks 100, 101, 104 and 106 (option volume, open interest, historical and implied volatility), and polls for up to 10 seconds until every ticker has bid, ask, volume, open interest and model Greeks stamped with the expected tick type.
3. The backend accepts a quote only if the returned mode equals the session mode, the timestamp is timezone-aware and $0 \le age \le 30$ s (live) or 930 s (paper: 15 minutes of delay plus 30 s), and the tick type is exactly 1 (live) or 3 (paper). Frozen (2) and delayed-frozen (4) quotes are always refused.
4. Chain rows require $bid \ge 0$, $ask > 0$, $ask \ge bid$ and non-null open interest and volume (missing open interest is never fabricated). The lot size is $\lceil min_size/increment \rceil \cdot increment$; the leg price is the bid/ask mid for the chain, and the *executable side* (ask to buy, bid to sell) for economics.

For the NIFTY contract tested on 19 September 2026, IBKR reported multiplier 1, minimum quantity 65, increment 65 and tick 0.05, so one exchange lot is 65 broker units. Multiplier and lot size are read from contract details, never hard-coded.

7.4.3 Daily features

The RPS model needs completed daily sessions, never the streaming warm-up buffer. With `REMOTE_FEATURE_DB_URL` configured, up to 500 sessions of `index_underlying_candle` rows are read from the remote feature database (strictly before the decision day) together with the latest `index_chain_feature` row (IV rank and percentile over 252 sessions, ATM IV, realised volatility, IV/RV, term structure, 25-delta skew). Without it, the model uses the strategy's own persisted daily IBKR bars, which only exist when the strategy runs at `1d`. A configured remote source that cannot be read *raises and refuses the entry*; it never silently reverts to a neutral score.

7.5 Stage 1: the RPS V2 score

RPS ("range probability score") measures how range-bound the underlying is, on a 0–100 scale.

7.5.1 Daily feature frame

Intraday or daily candles are grouped into sessions (open = first, high = max, low = min, close = last, volume = sum; zero volume, as reported for indices, becomes missing). With Wilder smoothing $W_n(x) = \mathrm{EWM}(\alpha = 1/n)$:

$$\mathrm{TR}_t = \max(H_t - L_t,\ |H_t - C_{t-1}|,\ |L_t - C_{t-1}|), \qquad \mathrm{ATR14}_t = W_{14}(\mathrm{TR}),\quad \mathrm{ATR50}_t = W_{50}(\mathrm{TR}),$$

$$r_t = \ln(C_t/C_{t-1}), \qquad \mathrm{HV}_n = \mathrm{std}(r, n)\sqrt{252},$$

$$\mathrm{BBW}_t = 4\,\sigma_{20}/\mathrm{MA}_{20}, \qquad z_{20} = (C_t - \mathrm{MA}_{20})/\sigma_{20},$$

$$\mathrm{DI}^{\pm} = 100\,W_{14}(\mathrm{DM}^{\pm})/W_{14}(\mathrm{TR}), \qquad \mathrm{ADX14} = W_{14}\left(100\,\frac{|\mathrm{DI}^{+} - \mathrm{DI}^{-}|}{\mathrm{DI}^{+} + \mathrm{DI}^{-}}\right),$$

$$\mathrm{vol_blend} = \mathrm{mean}\left(\tfrac{\mathrm{ATR14}}{C}, \tfrac{\mathrm{ATR50}}{C}, \mathrm{HV}_{20}, \mathrm{HV}_{60}\right), \qquad \mathrm{volume_ratio} = V_t/\overline{V}_{20}.$$

7.5.2 The range envelope

Every input is cut strictly before the decision session; the feature row used is that of the *prior completed* session. The support/resistance envelope is

$$S = C_{\mathrm{prev}} - 0.75 \cdot \mathrm{ATR14}_{\mathrm{prev}}, \qquad R = C_{\mathrm{prev}} + 0.75 \cdot \mathrm{ATR14}_{\mathrm{prev}}.$$

The multiplier must be exactly 0.75 (validated). Open-interest walls are computed separately: on each side of spot (puts at or below, calls at or above), open interest is winsorised to its 1st/99th percentile, the five largest strikes are taken, and the wall is their OI-weighted mean strike. If the put wall is not below the call wall, both are discarded.

7.5.3 Factors and weights

Each factor scores 0–100. interp is linear interpolation between the listed nodes; $\mathrm{invpct}(x, h)$ is the inverse percentile of x in its own trailing history h (last 252 valid values, at least 20), $\mathrm{clip}(100 - \mathrm{pct}, 1, 99)$, so a low value relative to history scores high.

Factor	Units	Weight	Score
`distance_sr`	18	21.18	$\mathrm{clip}(200 \cdot \min(s - S,\ R - s)/(R - S),\ 0,\ 100)$: 100 at mid-range, 0 at or beyond a boundary
`oi_structure`	17	20.00	The same range score using the OI put and call walls
`expected_move`	15	17.65	$x = \min(EM/(R - S), 3)$ with EM the ATM straddle mid; nodes $(0, 100), (0.5, 80), (1, 50), (1.5, 20), (3, 0)$
`volume_pressure`	10	11.76	$x = \min(\mathrm{volume_ratio}, 5)$; nodes $(0, 100), (0.5, 95), (1, 75), (1.5, 40), (2, 15), (5, 0)$
`atr_realized_vol`	10	11.76	$\mathrm{invpct}(\mathrm{vol_blend})$: calm relative to its history scores high
`trend_adx`	7	8.24	$\mathrm{invpct}(\min(\mathrm{ADX14}, 60))$: a weak trend scores high
`bollinger_compression`	4	4.71	$\mathrm{invpct}(\mathrm{BBW})$, needs at least 60 values

Factor	Units	Weight	Score
`mean_reversion`	4	4.71	$x = \min(\lvert z\rvert, 3)$ with $z = (s - \mathrm{MA}_{20})/\sigma_{20}$ of the prior session; nodes $(0, 100), (0.5, 90), (1, 70), (2, 30), (3, 0)$

The source weights total 85 units and are rescaled to 100 ($w_i = u_i \cdot 100/85$). The score renormalises over the set A of factors that could be measured:

$$\mathrm{RPS} = \frac{\sum_{i \in A} w_i \, s_i}{\sum_{i \in A} w_i}.$$

An unobservable factor is *excluded*, never scored as a fabricated neutral. The percentile factors need real history (about 80 sessions for all three). Entry requires RPS $\geq$ `rps_threshold` (60); otherwise the status is `ENTRY_GATE_CLOSED`.

7.6 Stage 3: the IV and breach entry gate

The gate runs *per candidate*, against that candidate's own boundaries.

7.6.1 Implied-volatility blend

Current IV σ is the mean IV of the strike(s) nearest spot at the candidate's expiry (IVs ≤ 0 or ≥ 5 are discarded; a missing same-expiry IV fails closed). Components:

Component	Weight	Score
IV rank	0.25	remote `iv_rank_252`, else neutral 50
IV percentile	0.20	remote `iv_percentile_252`, else neutral 50
$\mathrm{IV/RV} = \sigma/\mathrm{HV}_{20}$	0.30	$< 0.90 : 20$; $< 1.00 : 30$; $< 1.15 : 60$; $\leq 1.75 : 85$; $> 1.75 : 100$; missing: 50
Term ratio $\sigma_{\text{front}}/\sigma_{\text{next}}$	0.15	$[0.90, 1.10] : 70$; $\leq 1.20 : 55$; otherwise 25; missing: 50
25-delta skew $p = \lvert \mathrm{IV}_{P25} - \mathrm{IV}_{C25}\rvert \cdot 100$	0.10	$p \leq 0.5 : 70$; $\leq 1 : 60$; $\leq 2 : 50$; $\leq 3 : 35$; $> 3 : 20$; missing: 50

The raw blend $raw = \sum w_k c_k$ is normalised to its reachable span $[11.75, 92.5]$:

$$IVscore = \mathrm{clip}\left(100 \cdot \frac{raw - 11.75}{80.75},\, 0,\, 100\right),$$

and the IV condition is **FAVOURABLE** when a current IV exists and $IVscore \geq$ `rps_iv_score_threshold` (60), i.e. $raw \geq 60.2$.

> **Practical effect without the remote feature source**
>
> Without remote features, IV rank and percentile are neutral (50) and the term component is usually neutral, because only one expiry is fetched. Entry then effectively requires IV/RV ≥ 1.15 with a small skew, or IV/RV > 1.75.

7.6.2 Settlement breach probability

Boundaries are, in order of preference, the candidate's zero-cost **breakevens**, its short strikes, its wing strikes, or the RPS envelope. A single-sided spread has one boundary. The terminal price is simulated under deterministic geometric Brownian motion using the contract IV:

$$S_T = S_0 \exp\Big((\mu - \tfrac{1}{2}\sigma^2)\tau + \sigma\sqrt{\tau}\,Z\Big), \qquad \tau = \mathrm{DTE}/365, \quad \mu = r - q = 0.068 - 0.012,$$

with 10,000 paths and a seed derived from a SHA-256 of the decision inputs, so the same decision always gives the same probabilities. Then

$$P_{\uparrow} = \Pr(S_T > U), \qquad P_{\downarrow} = \Pr(S_T < L), \qquad P_{\text{either}} = P_{\uparrow} + P_{\downarrow},$$

the union being a sum because the two events are mutually exclusive at settlement.

7.6.3 Gate decision

Entry is allowed only if all of the following hold:

- data quality is OK (RPS, IV and breach all available);
- RPS ≥ 60 and the IV condition is **FAVOURABLE**;
- $P_{\uparrow} \leq 0.50$ and $P_{\downarrow} \leq 0.50$ (a side that does not apply counts as 0; an unavailable computation counts as 1);
- for two-sided structures only, $P_{\text{either}} \leq$ the two-sided ceiling (default min of the two one-sided ceilings, 0.50); a single-sided spread is judged only on its own side;
- $\mathit{range_confidence} = 0.5\,\mathrm{clip}(\mathrm{RPS}/100) + 0.5\,\mathrm{clip}\big(1 - (P_{\uparrow} + P_{\downarrow})/2\big) > 0$.

Because the breakevens are the default boundary, $P_{\text{either}} \leq 0.5$ acts as an implicit probability-of-profit floor of about 50% for every family.

7.7 Stage 4: payoff, POP and expected value

Each leg has a signed quantity $q = \pm\, \mathit{lots} \times \mathit{lot_size}$, multiplier m and an entry premium (ask for a buy, bid for a sell). The engine (`option_analytics/payoff.py`) works with the exact piecewise-linear expiry payoff:

$$\mathrm{PnL}(S_T) = -\,\mathit{cost} + \sum_{\text{legs}} q\,m\big(\mathrm{intrinsic}(S_T) - \mathit{premium}\big).$$

Breakevens and profitable intervals
P&L is evaluated at $\{0\} \cup \{\text{strikes}\}$; roots are found by linear interpolation within each segment, and the right tail by its slope $\sum_{\text{calls}} q\,m$.

Probability of profit
Risk-neutral lognormal: $\mathrm{POP} = \sum_{[a,b]} \big(F(b) - F(a)\big)$ over profitable intervals, with $F(K) = \Phi\Big(\dfrac{\ln K - \ln S - (r - q - \sigma^2/2)T}{\sigma\sqrt{T}}\Big)$ and $T = \mathrm{DTE}/365.25$.

Expected P&L
$\mathrm{EV} = -\mathit{cost} + \sum q\,m\,(\mathbb{E}[\text{payoff}] - \mathit{premium})$, with Black-style undiscounted expectations $\mathbb{E}[\text{call}] = FN(d_1) - KN(d_2)$, $\mathbb{E}[\text{put}] = KN(-d_2) - FN(-d_1)$, $F = Se^{(r-q)T}$, evaluated at a *forecast* volatility equal to realised HV_{20}. EV is therefore positive roughly when implied volatility exceeds realised volatility.

Max profit and max loss

From the extrema over $\{0\} \cup \{\text{strikes}\}$; unbounded when the call slope is positive (profit) or negative (loss). Values are net of the round-trip cost $rtc = round_trip_cost_per_lot \times legs \times lots$ (default 40 per leg per lot).

A candidate profile is discarded if it mixes expiries, has net short calls (unbounded upside loss), or shows a maximum loss of essentially zero (a sign of crossed or stale quotes). The pre-filters then reject if `rps_max_entry_loss` is set and exceeded, if POP is below `rps_min_pop` (clamped to the family floor; default 0), or if `rps_require_positive_ev` is set and EV ≤ 0.

7.8 Stages 5 and 6: gates, TQS and the winner

7.8.1 Hard gates

With *capital* the account's net liquidation, $net_ev = \text{EV} - rtc$:

Gate	Condition (any failure removes the candidate)
`GATE_BOUNDED`	defined risk (finite max profit and loss)
`GATE_EV`	$net_ev > 0$
`GATE_MAXLOSS`	$0 < max_loss \leq \min(25{,}000,\ 0.02 \cdot capital)$
`GATE_LIQ`	liquidity score $1 - penalty \geq 0.15$, with penalty $= \min(0.5,\ \overline{\text{spread/mid}} + 0.10\,[\text{thin OI}] + 0.10\,[\text{thin volume}])$
`GATE_DATA`	market-state data quality OK
`GATE_CREDIT`	credit families: wing width > 0, net credit > 0 and net credit $\geq 0.08\times$ wing width

Graded gates (POP $\geq$ family floor — iron butterfly 0.10, iron condor 0.45, credit spread 0.55, calendar 0.35; RPS ≥ 55; breach ≤ 0.50; event risk; cost $\leq 50\%$ of credit; EV ratio ≥ 0.005) are recorded as advisories and never reject.

7.8.2 Market-state routing

A routing table orders the four structures by the breach asymmetry and IV rank (for example $P_{\downarrow} + 0.10 < P_{\uparrow}$ puts the put credit spread first; IV rank ≥ 60 puts the iron condor first). Because the winner is an arg-max and routing never removes a structure unless event risk is extreme (not measured here), routing does not change the outcome in this runtime.

7.8.3 Trade Quality Score (TQS)

With $\text{up}(x,a,b) = \text{clip}((x-a)/(b-a))$, $\text{dn} = 1 - \text{up}$, $\text{logit}(x,m,k) = 1/(1+e^{-k(x-m)})$ and $\text{band}(x,a,b,c,d)$ a trapezoid (0 outside (a,d), 1 on $[b,c]$):

Factor	Weight	Formula
A1_RPS	0.10	$\text{up}(\text{RPS}, 55, 85)$
A2_BREACH	0.10	$\text{up}(1 - breach, 0.30, 0.85)$
A3_BANDFIT	0.10	$\text{band}((wing/2)/EM,\ 0.5, 0.8, 1.5, 3.5)$
B1_EVRISK	0.05	$\text{dn}(event_risk, 20, 90)$ — no event feed, excluded

Factor	Weight	Formula
B2_PIR	0.04	$\text{logit}(pir, 1.0, 4.0)$ — excluded
B3_EVGAP	0.03	$\text{up}(gap, 0, 3)$ — excluded
C1_IVRANK	0.05	$\text{up}(iv_rank, 10, 70)$; weight 0 when IV rank is not measured
C2_VRP	0.06	$\text{up}(100(\sigma_{\text{IV}} - \sigma_{\text{RV}}), -3, 6)$
C3_TERM	0.04	$\text{up}(slope, -3, 3)$; excluded when unavailable
D1_EV_RATIO	0.10	$\text{up}(net_ev/max_loss, 0, 0.15)$
D2_POP_ADJ	0.08	$\text{up}(\text{POP} \cdot max_profit/max_loss, 0.05, 0.55)$
D3_ROM	0.10	$\text{up}(net_ev/margin, 0, 0.20)$, margin = max loss
E1_LIQ	0.03	$\text{up}(liquidity_score, 0.15, 0.80)$
E2_COST	0.02	$\text{dn}(rtc/net_credit, 0.05, 0.50)$; excluded for debits

$$\text{TQS} = 100 \cdot \frac{\sum_{k \in \text{avail}} w_k \,\text{clip}(f_k)}{\sum_{k \in \text{avail}} w_k}, \qquad coverage = \frac{\sum_{\text{avail}} w_k}{\sum_{\text{all}} w_k}.$$

Unmeasured factors are excluded from numerator and denominator and listed in the evidence.

7.8.4 Choosing the winner

$$RAR = \frac{net_ev}{max_loss}, \qquad selection_score = RAR \cdot \Big(0.5 + 0.5\,\text{clip}\big(\tfrac{\text{TQS}-60}{20}\big)\Big).$$

Among candidates that pass every hard gate with $net_ev >$ `rps_tqs_min_net_pnl` (default 0), the winner maximises $(selection_score, \text{TQS}, RAR)$. Decision bands are also reported (TRADE_FULL $\geq$ 70, TRADE_HALF $\geq$ 60, WATCH $\geq$ 50, else NO_TRADE) but do not currently gate or scale the entry.

7.9 Stage 7: freezing the decision

For the winner, each leg is matched to an IBKR chain definition and its exact contract is qualified (conid, expiry, strike, right, multiplier, trading class must all match). An immutable `OptionSelectionSnapshot` is written with the observation time, provider, model version, ATR and OI boundaries, spot, entry score, candidate rankings, selected legs and full evidence (RPS observation, selection evidence, exit baseline, gateway mode, thresholds and the decision order). The snapshot is unique per (instance, version, bar id, bar version), so a retried job reuses the same decision, and while a structure is pending or held it is never re-selected.

A `STRATEGY` market-data lease (1-minute bars, 24 hours) is taken for every leg. The evaluation context is **READY** only when every leg's market state is execution-usable; otherwise the job waits in **WAITING_FOR_INPUT**. A later `market.quality` event for a leg promotes the *same* job to **PENDING** as soon as all legs are usable, without waiting for another daily bar.

7.10 The options plugin

The plugin (`apps/strategies/plugins/options.py`) uses the `MULTI_LEG_QUANTITY` execution model and supports underlyings of type `STK` or `IND`. Its only stream input is `BAR` with warm-up $\max(75,\ 5 \cdot atr_period + 5)$.

- **Entry**: `ENTER_OPTIONS`, next state **ENTRY_PENDING**, confidence = entry score/100. One quantity target per leg: $q = lots \times lot_size \times ratio \times (\pm 1)$, with metadata `hedge_first` (true for buy legs) and `lifecycle_locked`. With `one_entry_per_expiry` (default true) an expiry already entered is refused.
- **Holding**: `HOLD_OPTIONS`, restating the leg targets.
- **Exit**: `EXIT_OPTIONS`, a `FLAT` target per held leg, next state **EXIT_PENDING**.

7.11 Exits

> **Held to expiry by design**
>
> There is no hard stop and no range-breach exit. The worst case of a defined-risk structure is bounded at entry; closing on a transient breach would realise that worst case instead of letting it resolve. The range is still measured and reported on every evaluation.

The plugin exits for one of three reasons, in this precedence:

1. **Expiry**: $(nearest\ expiry - date) \le$ `exit_dte` (default 0, i.e. on expiry day).
2. **Hold period**: `hold_days` > 0 and the position is at least that old (default off; 10 for the delta put spread template).
3. **Soft exit** from the dynamic exit controller.

7.11.1 Dynamic soft exits

A baseline is frozen at entry: range bounds, the sum of leg volumes and the mean leg IV, and a peak P&L of 0. On every bar while holding (with fresh IBKR quotes for every leg; otherwise the position is simply held), the controller computes, with legs rebuilt from attributed positions at their average cost:

$$
\begin{aligned}
pnl &= \textstyle\sum (mid - \bar{c})\, q\, m, & peak &= \max(peak, pnl),\\
capture &= pnl / max_profit, & remaining &= \max(0, max_profit - pnl)/max_profit,\\
\Delta vol &= (V - V_0)/V_0, & \Delta iv &= (\sigma - \sigma_0)/\sigma_0,\\
P_{\text{giveback}} &= \Pr\left(\mathrm{PnL}_T < 0.75 \cdot peak\right), & ev_impr &= (\mathrm{EV}_{\text{hold}} - pnl)/\max(|pnl|, 1).
\end{aligned}
$$

Soft exit	Condition (defaults)
`SOFT_VOLUME_IV_EXPANSION`	$\Delta vol \ge 0.50$ **and** $\Delta iv \ge 0.20$ (OR if `dynamic_exit_require_volume_and_iv` is false)
`SOFT_PROFIT_GIVEBACK_RISK`	$capture \ge 0.70$ and $P_{\text{giveback}} \ge 0.40$
`SOFT_PROFIT_CAPTURE_RISK_REWARD`	$capture \ge 0.70$, $remaining \le 0.30$ and $ev_impr < 0.05$

All three bank an edge that already exists; none reacts to adverse movement alone.

7.12 Executing a structure

Option legs flow through the same target → rebalance → intent → risk → command pipeline as stocks (Chapter 8), with these option-specific rules.

7.12.1 Staging: protection first

The rebalancer assigns each leg a stage (rule `LONG_PROTECTION_BEFORE_SHORT_ENTRY;SHORT_COVER_BEFORE_LONG_EXIT`):

	Stage 0 (first)	Stage 1 (after stage 0 fills)
Entry	long (hedge) legs	short legs
Exit	buy back the short legs	sell the long legs

One order intent is created per leg, with `execution_priority` $= 10000 \cdot \textit{stage} + \textit{rank}$. In the `OPTION_HEDGES` phase, stage-1 intents are created *ineligible*. When every stage-0 order is terminal, the fill ratio $\sum \textit{filled} / \sum \textit{requested}$ is compared with 0.95: below it, the rebalance is **BLOCKED** and the shorts never go out; at or above it, the shorts become eligible and the phase moves to `OPTION_SHORTS`. A single-sided structure such as a cash-secured put starts directly in `OPTION_SHORTS`. Exact option quantities bypass the generic position sizer so that hedge ratios are never distorted.

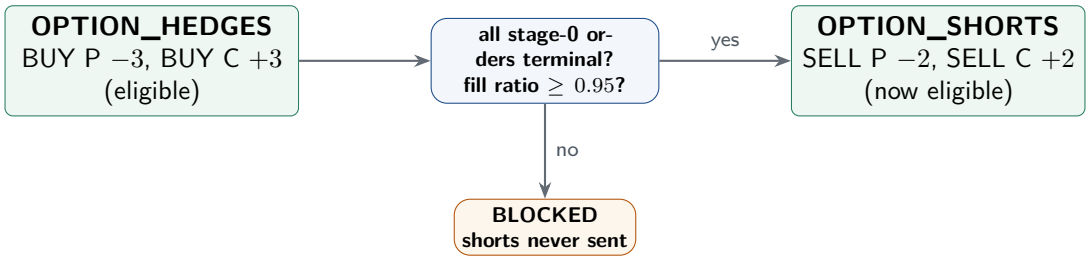


Figure 7.2: Entering an iron condor: protection is filled before any short leg is released.

7.12.2 Risk for option legs

- **Short legs.** A sell that exceeds the available long position is allowed only for an automatic (rebalance-origin) opening short whose leg is covered: by a cash-secured-put reserve (collateral = strike × multiplier) or by an *already filled*, strategy-attributed hedge — a same-expiry put with a lower strike for a short put, a call with a higher strike for a short call, or a later-expiry call with a strike at or below the short for a calendar. Collateral per unit is $|K_{\text{hedge}} - K_{\text{short}}| \times m$ for a vertical. The collateral must fit within *available cash* × (1 − *cash buffer*) and is reserved as an `OPTION_SHORT_COLLATERAL` capital reservation. Example: a 100-point put spread with multiplier 50 reserves 5,000.
- **Buy legs.** Cash check on notional plus fees, net of collateral already reserved for a short in the same contract.
- **Notional** includes the contract multiplier.
- **Bypass.** When `OPTION_PRETRADE_RISK_ENABLED=false`, option intents skip policy limits, reconciliation, quantity, freshness, capital and coverage checks (kill switches and gateway connectivity still apply). The local validation stack sets this because the available-position check otherwise refused every short leg. With the bypass, protection rests on staging and the 0.95 fill-ratio gate alone.

7.12.3 Order placement

Option orders are sent to the gateway's POST `options/orders/` endpoint with the full qualified identity (conid, expiration, strike, right, multiplier, trading class). The gateway re-qualifies the

contract and refuses the order if any field differs from the operator-selected contract. The order type comes from the strategy's order policy; with none it is a market order for the day. Fills apply the multiplier to cash and P&L (buying 1 contract at 100 with multiplier 50 and commission 1 debits 5,001).

7.13 Options parameters reference

Parameter	Default	Meaning
`lots`	1	Exchange lots per leg
`option_strike_count`	21	Strikes nearest spot fetched per right (5–21)
`round_trip_cost_per_lot`	40	Estimated cost per leg per lot, contract currency
`atr_multiplier`	0.75	Envelope multiplier (must be 0.75)
`rps_threshold`	60	Minimum RPS
`entry_min_dte` / `entry_max_dte`	1 / 45	Expiry window
`minimum_open_interest` / `minimum_volume`	100 / 1	Per-leg liquidity
`hold_days` / `exit_dte`	0 / 0	Lifecycle exits
`one_entry_per_expiry`	true	No re-entry into an entered expiry
`target_delta` / `strike_width`	−0.20 / 100	Delta put spread selection
`rps_market_open_time` / `rps_entry_delay_minutes`	09:15 / 0	Entry window
`rps_market_timezone`	Asia/Kolkata	Entry window zone
`rps_iv_breach_enabled`	true	IV/breach gate on
`rps_iv_score_threshold`	60	FAVOURABLE threshold
`rps_max_upper_breach_probability`	0.50	One-sided ceiling
`rps_max_lower_breach_probability`	0.50	One-sided ceiling
`rps_max_two_sided_breach_probability`	null	Two-sided ceiling (defaults to min of the above)
`rps_breach_simulation_paths`	10000	GBM paths
`rps_pop_risk_free_rate` / `rps_pop_dividend_yield`	0.068 / 0.012	Drift inputs
`rps_min_pop`, `rps_require_positive_ev`, `rps_max_entry_loss`	0, false, 0	Pre-filters (off)
`rps_tqs_threshold` / `rps_tqs_min_net_pnl`	60 / 0	TQS band threshold; minimum net EV
`dynamic_exit_enabled`	true	Soft exits on
`dynamic_exit_soft_max_giveback`	0.25	Floor = 0.75× peak
`dynamic_exit_soft_giveback_probability`	0.40	

Parameter	Default	Meaning
dynamic_exit_soft_profit_capture	0.70	
dynamic_exit_soft_volume_increase / dynamic_exit_soft_iv_increase	0.50 / 0.20	
dynamic_exit_soft_min_ev_improvement / dynamic_exit_soft_remaining_profit	0.05 / 0.30	

7.14 Creating an options strategy

```
POST /api/v1/options/strategy-instances/
{ "name": "NIFTY automatic options", "portfolio_id": 1, "ticker": "NIFTY",
  "parameters": { "lots": 1, "round_trip_cost_per_lot": 40 }, "activate": false }
```

The endpoint defaults to `RPS_OPTIONS_AUTO_SELECTOR` and daily bars. NSE/INR `NIFTY` resolves to IBKR's `NIFTY50` index. An `execution_mode` must not be sent: the portfolio's gateway session decides PAPER or LIVE, and within the gateway `IBC_TRADING_MODE` selects API port 4002 (paper) or 4001 (live). Activation uses the standard lifecycle, either with `activate: true` or POST `/strategy-instances/<id>/enable/`.

CHAPTER 8

Execution: From Target to Fill

This chapter follows a strategy target through the rest of the system: coordination into a portfolio snapshot, netting of several strategies, rebalance planning, position sizing, pre-trade risk, the order management system (OMS), broker command dispatch, and the return path of status updates and fills. The same pipeline serves stocks and options, and manual orders enter it at the order-intent stage.

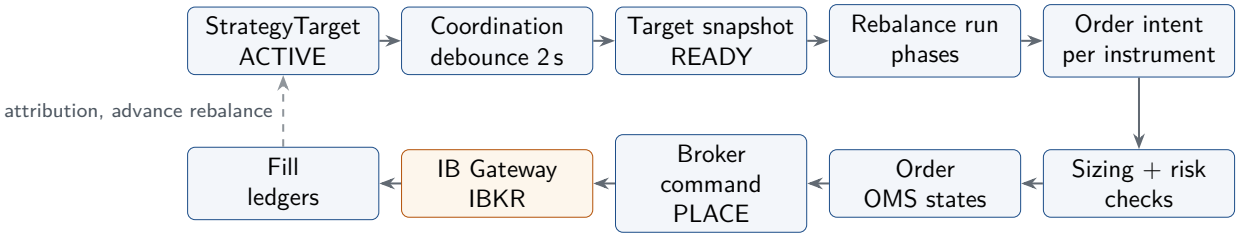


Figure 8.1: The execution chain. Each box is a durable database record with its own state machine.

8.1 Target coordination

8.1.1 Marking and debouncing

Every completed evaluation, pause or flatten calls `mark_portfolio_for_target_coordination`. There is exactly one `PortfolioTargetCoordination` row per portfolio:

- If no automatic rebalance is active, the row becomes **PENDING** with `debounce_until` = now +2 s. Several strategies firing on the same bar therefore collapse into one snapshot (five strategies at 0.02 each give one snapshot with net target 0.10).
- If an automatic rebalance is active and the new event is newer, the row becomes **ACTIVE** with `pending_recalculation` set; the new target is applied only at the rebalance's safe boundary. Older or duplicate events are ignored.

Every second, `coordinate_portfolio_targets` claims up to 50 due rows with `SKIP LOCKED` and coordinates each portfolio in its own transaction. A failure is isolated to that portfolio and retried with backoff $\min(300, 5 \cdot 2^{n-1})$ seconds.

8.1.2 Single flight

A partial unique constraint allows only one automatic rebalance per portfolio in **QUEUED**, **CALCULATING**, **INTENTS_CREATED** or **EXECUTING**. When a run finishes, `release_portfolio_coordination` re-arms the row immediately if a newer target arrived meanwhile. Rebalances for a portfolio are therefore strictly sequential and never interleave.

8.2 The portfolio target snapshot

`build_portfolio_target_snapshot` produces an immutable `PortfolioTargetSnapshot` under a portfolio row lock.

1. **Mode and capital.** The execution mode comes from the portfolio's session; NAV and available cash from the broker-synced account.
2. **Latest targets.** For each (instance, instrument), the most recent active target by signal time. For option strategies only the newest run that actually produced targets is used, keeping multi-leg structures all-or-nothing.
3. **Lifecycle behaviour.** Each strategy's lifecycle maps to a behaviour: active → *TARGET*; flatten requested → *FLATTEN*; paused, error, killed, disabled and activation states → the risk policy's configured behaviour (default *HOLD*: keep the attributed quantity).
4. **Reference prices** must be execution-usable (Section 3.6); a missing price rejects the snapshot with `MISSING_REFERENCE_PRICE`.
5. **Contribution checks.** A target is rejected if the strategy or target mode differs from the gateway mode, if its version is not current, if it is older than 900 s (`STALE_TARGET`, measured from run completion; lifecycle-locked option legs are exempt), or if it breaches the strategy risk policy (maximum weight, quantity, notional, shorting).
6. **Netting** (below).
7. **External holdings.** For exact-quantity instruments (options), any broker position not attributed to a strategy is added to the target, so a manual holding in the same contract is never liquidated.
8. **Projected exposure.** $projected = filled + \sum remaining\ of active\ orders + \sum reserved\ intents$. An open buy of 4 against a target of 10 produces a new intent of 6, not 10.
9. **Order policy** per instrument from the contributing strategy with the lowest (priority, id); default market order, day.
10. **Status READY**, or **REJECTED** if any input was rejected or NAV ≤ 0. A rejected snapshot cannot create a rebalance.

8.2.1 Netting several strategies

For strategy s contributing to instrument i, with capital share $\kappa_s = K_s/NAV$:

Target type	**Desired quantity** $q_{s,i}$	**Effective weight** $c_{s,i}$
WEIGHT	$c_{s,i}\,NAV/(P_i m_i)$	$w_{s,i}\,\kappa_s$
VALUE	$v_{s,i}/(P_i m_i)$	$v_{s,i}/NAV$
QUANTITY / FLAT	q^{target} (0 for FLAT)	$q\,P_i m_i/NAV$

The strategy trade delta is $\delta_{s,i} = q_{s,i} - a_{s,i}$ where a is the strategy's attributed position. The portfolio trades one net delta per instrument: $net_target_i = \sum_s c_{s,i}$ (and $\sum_s q_{s,i}$ for exact-quantity instruments). Opposite wishes cancel inside the netting.

When the resulting order intent carries net delta Δ_i, each contributing strategy is allocated

$$alloc_s = \begin{cases} \Delta_i\,\delta_{s,i}/\sum_s \delta_{s,i} & \text{if } \sum_s \delta_{s,i} \neq 0, \\ \Delta_i\,c_{s,i}/\sum_s c_{s,i} & \text{else if } \sum_s c_{s,i} \neq 0, \\ \Delta_i\,|c_{s,i}|/\sum_s |c_{s,i}| & \text{otherwise,} \end{cases}$$

stored as an `OrderIntentAttribution` (method `PRO_RATA_TARGET_DELTA`).

8.3 Rebalance planning

`plan_rebalance` converts a READY snapshot into a `RebalanceRun`, one `TargetPortfolioPosition` per instrument, and order intents.

8.3.1 Universe

For an automatic (strategy-target) run, only instruments with a net target are considered; unattributed broker holdings stay untouched, so an automatic strategy can never liquidate the whole account. Manual runs also include current positions (untargeted holdings get target 0).

8.3.2 Per-instrument computation

$$w_i^{\text{cur}} = current_i\, P_i m_i / NAV, \qquad drift_i = w_i - w_i^{\text{cur}},$$

$$q_i^{\text{target}} = \text{round_half_up}_{lot}\left(\frac{w_i\, NAV}{P_i m_i}\right) \text{ or the exact quantity}, \qquad \Delta_i = q_i^{\text{target}} - current_i.$$

A trade is suppressed, in this order, for: stale price (manual path), drift below 0.01 (non-exact instruments, strategy and other non-manual triggers), quantity below the minimum, notional below 10, unqualified contract (strict mode), portfolio maximum order quantity or notional, or cost exceeding benefit ($1 + 0.0005 \cdot notional \geq |drift| \cdot NAV$).

8.3.3 Ordering, cash and turnover

Candidates are sorted by (option stage, or sells before buys; then $-|drift|$; then cost). Buys are limited by cash: $buy_cash = cash(1 - 0.02)$ and $q^{\max} = \text{floor}_{lot}((buy_cash - 1)/(P\, m\, 1.0005))$; turnover is capped at 25% of NAV. With `sell_before_buy`, sell proceeds are added to the buy budget.

8.3.4 Phases

Phase	Meaning
`PREVIEW_COMPLETE`	Preview run: the plan is stored, no intents are created
`SELLS`	Sells are eligible; buys are created ineligible
`BUYS`	Buys unlocked after sells reached the 95% fill threshold; each buy is recomputed from actual cash and positions before release
`OPTION_HEDGES` / `OPTION_SHORTS`	Protection-first option staging (Section 7.12)
`COMPLETE` / `BLOCKED`	Terminal; blocked when a stage's fill ratio stayed below 0.95

Run status moves **CALCULATING** → **INTENTS_CREATED** → **EXECUTING** → **COMPLETED**, **PARTIALLY_COMPLETED** or **FAILED**. Each intent has key `rebalance:{run}:instrument:{id}:v1`, `origin=REBALANCE`, `requires_fresh_price=true`, a limit price (for LMT orders) of $P\,(1 \pm offset_bps/10^4)$, and a planned event on `portfolio.rebalance.planned.v1`.

8.4 Position sizing

For each non-option rebalance intent, `size_and_record` computes six candidate quantities (each floored to the lot) and approves the smallest:

Limit	Quantity
target	the requested trade delta
risk	$\text{floor}(0.005\,NAV/(\lvert P - stop\rvert\, m))$ when a stop is given (stop distance $< 0.1\%$ is invalid); unconstrained otherwise
weight	$\text{floor}(0.10\,NAV/(P\,m))$
liquidity	$\text{floor}(0.01 \cdot ADV)$, ADV from the fresh live `average_volume` indicator (within 300 s); 0 if missing
cash	$\text{floor}(cash/(P\,m))$ for buys
broker	broker maximum quantity; 0 for a sell without short availability

The binding constraint is recorded in a `PositionSizingDecision`. Risk later enforces $approved = \min(approved, sized)$ and rejects a sized quantity of zero. A missing ADV therefore rejects the order: the ADV requirement fails closed.

8.5 Pre-trade risk

`evaluate_intent` runs under row locks on the intent, the account and the risk policy. Every check writes a `RiskCheckResult` and a `risk.decisions.v1` event. Checks run in order and the first failure decides:

#	Check	Outcome	Condition
1	Kill switch	REJECTED	global setting, portfolio, account or strategy flag, or a matching `KillSwitch` row (GLOBAL, ACCOUNT, PORTFOLIO, INSTRUMENT, STRATEGY)
2	Option bypass	APPROVED	option intent with `OPTION_PRETRADE_RISK_ENABLED=false` (held if the gateway is disconnected)
3	Risk policy	REJECTED	policy disabled
4	Gateway	HELD	gateway not connected
5	Mode chain	REJECTED	intent, session and gateway modes differ; LIVE without `ALLOW_LIVE_TRADING`
6	Reconciliation	HELD	gateway not reconciled or unresolved material break
7	Tradable	REJECTED	instrument not tradable
8	Execution sequence	HELD	intent not yet eligible (waiting for the previous stage)
9	Order quantity	REJECTED	below minimum, or not a multiple of the lot size when fractional trading is unsupported

#	Check	Outcome	Condition
10	Market freshness	REJECTED	no fresh execution-usable price; conservative risk price: MKT market, LMT limit, STP $\max(stop, market)$, STP_LMT $\max(stop, limit)$
11	Currency	REJECTED	instrument currency differs from account base currency (no FX)
12	Position sizing	REJECTED / RESIZED	sized quantity zero, or smaller than requested
13	Policy limits	RESIZED / REJECTED	$\min(max\ qty,\ max\ notional/(P\,m))$; defaults 100,000,000 and 100,000
14	Available position (sell)	REJECTED / HELD	sell exceeds long position minus reserved sells, unless an option short is covered (Section 7.12)
15	Available cash (buy)	HELD	$notional + fees + reserved + committed > cash(1 - 0.02)$, fees $= 0.0005\,notional + 1$
16	Pre-trade	APPROVED / RESIZED	all passed

Capital reservations. An approved buy reserves $notional + fees$ as a `CapitalReservation`; concurrent intents cannot reserve the same cash. On fill the reservation becomes **CONSUMED** and is released at the next broker account summary (so cash is never counted twice in the gap before IBKR reflects the fill); on cancel, reject or expiry it is released at once.

HELD intents return to **PENDING** and are re-evaluated on the next beat (every second); **REJECTED** is terminal (**RISK_REJECTED**).

8.6 Paper and live

The execution mode is `PAPER` or `LIVE`, always derived from the portfolio's gateway session. Consistency is enforced at every hop:

1. The snapshot rejects strategy and target modes that differ from the gateway.
2. `plan_rebalance` requires the requested and snapshot modes to equal the portfolio mode.
3. Database check constraints restrict the mode of intents, commands, rebalances and snapshots.
4. Intent execution fails an intent whose mode no longer matches.
5. Risk requires intent mode = session mode = gateway health mode.
6. Commands derive their mode from the portfolio; reusing a key with a different mode is an idempotency conflict.
7. Final dispatch re-checks the four-way chain and the live gate.
8. Delayed market data is only usable for PAPER.

`ALLOW_LIVE_TRADING` (false by default) is an independent gate: without it, LIVE strategies cannot activate, LIVE manual orders return 403, and LIVE intents are rejected. Cancels are always allowed.

8.7 The order management system

8.7.1 Intent execution

Every second, `execute_order_intents` claims eligible **PENDING** intents without an order, by `execution_priority`, with `SKIP LOCKED`, recording an `OperationAttempt`. For each: check the mode, probe gateway health (an unavailable session fails the intent; a transient gateway error returns it to **PENDING**), run risk, and on approval create the `Order` (**CREATED** → **RISK_APPROVED** → **QUEUED**) and enqueue a PLACE command, all in one transaction.

8.7.2 Order status machine

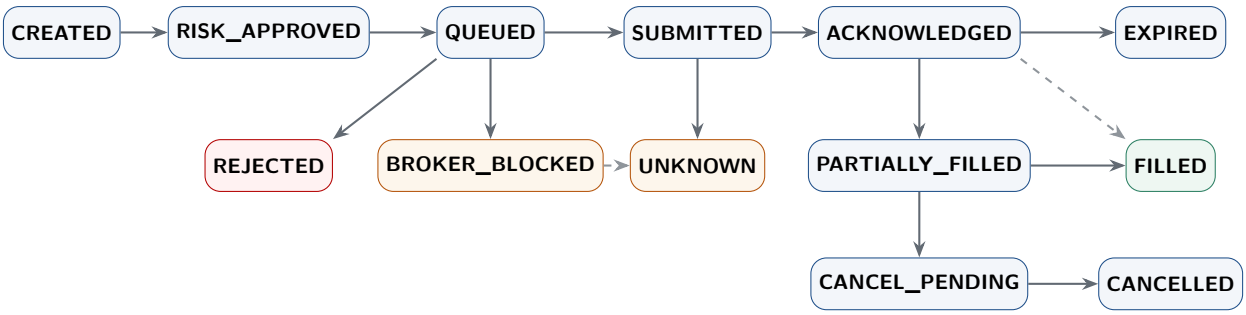


Figure 8.2: Principal order transitions (simplified). FILLED, REJECTED and EXPIRED are sinks; a cancelled order can still record late fills.

Transitions are validated against an explicit table and written as `OrderStatusHistory` rows with a unique `event_key`, which makes every transition idempotent: a repeated broker callback finds the existing row and does nothing. IBKR statuses map as follows: PendingSubmit/ApiPending → SUBMITTED; PreSubmitted/Submitted → ACKNOWLEDGED; PendingCancel → CANCEL_PENDING; ApiCancelled/Cancelled → CANCELLED; Inactive/ValidationError → REJECTED; Expired → EXPIRED. **FILLED is never set from a status message; only executions fill an order.**

8.8 Broker command dispatch

8.8.1 Command states

A `BrokerCommand` (PLACE, MODIFY or CANCEL) has status **PENDING**, **CLAIMED**, **SENDING**, **RETRY**, **UNCERTAIN**, **ACKNOWLEDGED** or **FAILED**. PLACE uses key `broker:place:{internal_id}`, so there is exactly one PLACE per order.

1. **Claim** (every second, dedicated worker): **PENDING**, or due **RETRY**/ **UNCERTAIN**, with `SKIP LOCKED`.
2. **Final checks**: health probe, four-way mode chain, connected; for non-cancel commands also the live gate, all kill switches, account reconciled, no material breaks, gateway reconciled. Any failure → **RETRY** with backoff; order and intent **BROKER_BLOCKED**.
3. **Sending marker**: **SENDING** with `sent_at`. From here on, the send may have happened.
4. **Send** with header `Idempotency-Key: session:{id}:{command key}` and *no HTTP-level retries.*
5. **Outcome**: a definitive 4xx rejection → **FAILED** (order **REJECTED**); a transport error after sending began → **UNCERTAIN** (order **UNKNOWN**); success → **ACKNOWLEDGED** (order **SUBMITTED**, broker order ids recorded).

8.8.2 Resolving uncertainty

Never resend blind

An **UNCERTAIN** command is never simply resent. When re-claimed, it asks the gateway for the order state by internal id. If the broker has the order, or the gateway has a matching command, the command is acknowledged (recovered). If the gateway proves *non-submission*, the command goes to **RETRY** and a real resend happens. Otherwise it stays **UNCERTAIN**, and execution readiness reports `UNRESOLVED_UNCERTAIN_ORDER`.

Recovery every 30 s: a command **CLAIMED** for over 120 s returns to **PENDING** (the worker died before sending); one **SENDING** for over 120 s becomes **UNCERTAIN**. Commands retry without an attempt limit; readiness raises `BROKER_COMMAND_STALE` after 60 s.

8.8.3 Manual orders, modifications and cancellations

POST `/orders/` (with `Idempotency-Key`) creates a `MANUAL` intent that joins the same pipeline; it wakes the intent worker on commit. `PATCH /orders/{id}/` modifies quantity (not below filled), limit, stop or time in force while the order is live. `POST /orders/{id}/cancel/` moves the order to **CANCEL_PENDING** and enqueues a CANCEL with the prior status, restored if IBKR rejects the cancel. IBKR code 201 surveillance warnings on manual orders move the intent to **CONFIRMATION_REQUIRED**; the operator can confirm (a new PLACE carrying IBKR's own override codes) or decline.

8.9 The return path: status, fills and ledgers

Every 5 seconds the backend pulls gateway events by cursor (Chapter 10):

- **Order events** update ids and status (matched by the internal id sent as IBKR's `orderRef`, then by permanent or order id). Unknown broker orders are imported with `origin=BROKER_IMPORT`.
- **Executions** call `apply_execution`, atomically and exactly once per execution id:
  - create the `Fill` (a duplicate with identical content is a no-op, except for a late commission correction, which adjusts the fill, the cash ledger and the attributions);
  - update filled quantity and VWAP; status **FILLED** when complete, otherwise **PARTIALLY_FILLED** (or stay **CANCEL_PENDING**/**CANCELLED**);
  - update the position with weighted-average cost: adding averages the cost; reducing realises $(p - \bar{c})\, q_{\text{closed}} \operatorname{dir} m$; crossing zero resets the cost to the fill price (4 at 100 then 6 at 110 gives 106; selling 4 at 130 from 10 at 100 realises 120);
  - write `PositionLedgerEntry` and `CashLedgerEntry` ($-\textit{signed qty} \cdot p \cdot m - \textit{commission}$), each with a unique key;
  - attribute to strategies: $\textit{alloc}_s \cdot \textit{fill qty}/\textit{order qty}$ added to each `StrategyAttributedPosition`, commission pro rata by $|\textit{alloc}_s|$;
  - advance the rebalance and settle the capital reservation.
- **Account summaries** update NAV, available cash, buying power and daily P&L, and release consumed reservations.
- **Disconnection** marks live orders **UNKNOWN** until the broker reports again.

8.10 Execution readiness

GET `/api/v1/execution/readiness/` returns 200 when automatic execution may proceed and 503 with named blockers otherwise. It is checked as a preflight on strategy activation; per-order safety comes from risk and final dispatch checks.

Blocker	Condition
`REQUIRED_FLINK_JOBS_MISSING / NOT_RUNNING`	`market-pipeline-v1` absent or not RUNNING
`FLINK_CHECKPOINT_MISSING / STALE`	no completed checkpoint, or older than 180 s
`FLINK_UNAVAILABLE`	Flink REST API unreachable (fail closed)
`KAFKA_DISABLED / KAFKA_UNAVAILABLE`	Kafka off, or connectivity heartbeat stale (> 60 s)
`REQUIRED_KAFKA_TOPICS_MISSING`	any of the 15 topics missing
`OUTBOX_PUBLISHER_UNHEALTHY`	publisher heartbeat stale (> 30 s) or failed outbox rows
`MARKET_RAW_PRODUCER_HEARTBEAT_STALE`	no raw market event within 30 s while strategies run
`MARKET_CONSUMER_HEARTBEAT_STALE`	persistence consumer heartbeat older than 30 s
`EXECUTION_WORKERS_ABSENT`	a worker role (evaluation, coordination, intents, broker commands) missing, stale (> 30 s) or unhealthy
`MARKET_DATA_STALE`	an enabled strategy's market state is not execution-usable or its subscription is inactive or silent for $\max(300,\ 2 \times bar + 60)$ s
`STRATEGY_JOB_BACKLOG_EXCEEDED`	more than 100 open evaluation jobs
`TARGET_COORDINATION_BACKLOG_EXCEEDED`	more than 100 portfolios awaiting coordination
`PENDING_INTENT_STALE`	oldest eligible pending intent older than 60 s
`BROKER_COMMAND_STALE`	oldest open broker command older than 60 s
`UNRESOLVED_UNCERTAIN_ORDER`	any command in UNCERTAIN
`GATEWAY_NOT_CONNECTED / NOT_RECONCILED`	session not connected (or checked more than 30 s ago, or mode mismatch) / not reconciled
`BROKER_RECONCILIATION_NOT_READY`	account not reconciled or unresolved material breaks

Worker heartbeats are coalesced: an idle, healthy worker whose result is unchanged rewrites its heartbeat at most every 10 s (well inside the 30 s staleness window); any real work or health transition writes immediately.

CHAPTER 9

Worked Walkthroughs

This chapter traces two complete automatic trades through the system. The numbers are illustrative (chosen to be easy to follow), but every step, record and rule is the one the code applies.

9.1 A stock trade: SMA crossover on AAPL

9.1.1 Setup

- Portfolio P bound to a connected, reconciled *paper* gateway session; account NAV 100,000 USD, available cash 100,000 USD.
- Strategy instance: `SMA_CROSSOVER`, instrument AAPL (qualified conid), timeframe `5m`, parameters fast 20, slow 50, target configuration `target_weight=0.10`, capital share 1.

9.1.2 Activation (t = 0)

1. POST `/strategy-instances/<id>/enable/` with an `Idempotency-Key`. Preflight passes: readiness is green (its own market-data staleness excused), the session is connected and reconciled, the contract is qualified. State **ACTIVATING**; the activation task is queued.
2. The worker registers four input requirements: `BAR` (warm-up 50), `sma` window 20 role fast, `sma` window 50 role slow, and the implicit `average_volume` window 20. Each gets an identity hash; an outbox event on `strategy.inputs.v1` carries all four. State **SUBSCRIBING**.
3. `reconcile_market_subscription` finds no subscription for (session, AAPL, 5m), mints a provider generation and sends `SUBSCRIBE_MARKET_DATA` with `historical_bars` $= 50 + 5 =$ 55.

9.1.3 Warm-up (t = seconds)

1. The gateway qualifies AAPL, sets delayed market data (paper session), fetches 55 historical 5-minute bars and emits them as `WARMUP` raw events, then starts delayed 5-second bars.
2. The backend's 5-second sync passes each event through the provider gate (generation matches) into the outbox; the publisher ships them to `market.raw.v1` within 2 s.
3. Flink normalises them (conid known from the registry), passes the 5-minute warm-up bars straight through as final bars (they already match the timeframe), and computes the three indicators on the shared LIVE/WARMUP history.
4. The persistence consumer writes 55 `MarketBar` and matching `IndicatorValue` rows. After the subscription completes, warm-up evidence exists (50 bars, all indicators non-null on the newest): state **READY_WAITING_FOR_LIVE_BAR**.

9.1.4 First live bars

Delayed 5-second bars are aggregated by Flink into 5-minute windows. When the watermark passes a window end, a final `LIVE` bar is emitted with its indicators. For each such bar:

1. `persist_bar` creates the bar and a `StrategyEvaluationJob` in **WAITING_FOR_INPUT**; as the four indicator values land, the same job becomes **PENDING**.
2. Within a second a worker claims it, re-validates every fact, and calls the plugin. Suppose the previous values were $f' = 199.8$, $s' = 200.1$ and the current $f = 200.3$, $s = 200.1$: the fast SMA crossed above the slow one and the strategy is flat, so the decision is `ENTER_LONG`, weight 0.10, next state **LONG**.
3. One transaction writes the run, the signal and a `WEIGHT` target of 0.10, emits a `strategy.targets.v1` event, and marks portfolio P for coordination (debounce 2 s).

9.1.5 Coordination, planning, sizing and risk

1. After 2 s the snapshot is built. The paper session's delayed price is execution-usable (source `ibkr_delayed`, fresh by ingress time); say $P = 200$. Effective weight $0.10 \times 1 = 0.10$; desired quantity $0.10 \times 100{,}000/200 = 50$ shares. Nothing is held or open, so projected quantity is 0.
2. The rebalance plan: drift 0.10 (above the 0.01 threshold), $\Delta = 50$, notional 10,000 (above the 10 minimum, below the 25% turnover cap), cost $1 + 5 = 6 \ll 0.10 \times 100{,}000$. One BUY intent for 50 shares, attributed 100% to the strategy.
3. Sizing: weight limit $\lfloor 0.10 \times 100{,}000/200 \rfloor = 50$; liquidity limit $\lfloor 0.01 \times ADV \rfloor$ (with an average volume of 500,000 shares per bar this is 5,000, not binding); cash limit 500. Approved 50, binding constraint `TARGET`.
4. Risk: kill switches clear, modes match (PAPER everywhere), reconciled, tradable, lot 1, fresh price, currency USD, within limits; cash check $10{,}000 + 6 \leq 98{,}000$; a capital reservation of 10,006 is written. Approved.

9.1.6 Dispatch and booking

1. The order (**QUEUED**) and a PLACE command are created. Within a second the broker-command worker runs the final checks, marks **SENDING**, and posts the order to the gateway with the order's internal UUID as IBKR `orderRef`. The gateway queues the command; its worker places the order at IBKR. Command **ACKNOWLEDGED**, order **SUBMITTED**.
2. IBKR reports PreSubmitted then Filled. The status events move the order to **ACKNOWLEDGED**; the execution snapshot (50 at 200.05, commission 1) calls `apply_execution`: fill created, order **FILLED**, position 50 at 200.05, cash ledger −10,003.50, strategy attributed position 50, rebalance **COMPLETED**, reservation consumed and released on the next account summary.

Later, when the fast SMA crosses below the slow one, the plugin emits `EXIT_LONG` with weight 0; the target becomes `FLAT`; the snapshot desires 0; a SELL of 50 goes through the same path.

9.2 An options trade: iron condor on NIFTY

9.2.1 Setup

- Portfolio bound to a paper session; account NAV 1,000,000 INR.
- Instance of `RPS_OPTIONS_AUTO_SELECTOR` on NIFTY (IBKR index `NIFTY50`, NSE, INR), daily bars, `lots=1`, `round_trip_cost_per_lot=40`, remote feature source configured.
- Contract details: multiplier 1, lot size 65 units, strikes every 50 points.

9.2.2 Decision on a daily bar

1. **Entry window.** Daily bars close after 09:15 IST, so the window is open.
2. **Chain.** Spot 25,000. The nearest expiry in the 1–45 DTE window is chosen; 21 strikes per right (42 contracts) are quoted in one command; paper quotes with tick type 3 and age up to 930 s are accepted.
3. **RPS.** Prior daily close 25,000 and ATR14 200 give $S = 24{,}850$ and $R = 25{,}150$. Spot is mid-range (distance factor 100); OI walls sit near 24,700 and 25,300; the ATM straddle implies an expected move below the envelope width; volume is unavailable for the index and excluded; say the renormalised RPS is $71 \geq 60$.
4. **Candidates.** Offsets on the 50-point ladder give: PCS (BUY 24,900P, SELL 24,950P); CCS (SELL 25,050C, BUY 25,100C); IC (BUY 24,850P, SELL 24,900P, SELL 25,100C, BUY 25,150C); IB (BUY 24,900P, SELL 25,000P, SELL 25,000C, BUY 25,100C). All legs pass liquidity.
5. **Iron condor economics** (executable side): buy 24,850P at 40, sell 24,900P at 52, sell 25,100C at 50, buy 25,150C at 38. Net credit 24 per unit, $24 \times 65 = 1{,}560$ INR. Round-trip cost $40 \times 4 \times 1 = 160$. Wing width 50, so

$$\textit{max loss} = (50 - 24) \times 65 + 160 = 1{,}850, \qquad \textit{max profit} = 1{,}560 - 160 = 1{,}400,$$

   with zero-cost breakevens 24,876 and 25,124.
6. **IV/breach gate.** Suppose IV/RV = 1.3 (score 85), remote IV rank 65 and percentile 70, skew 1.5 vol points (50), term neutral (50): $raw = 0.25 \cdot 65 + 0.20 \cdot 70 + 0.30 \cdot 85 + 0.15 \cdot 50 + 0.10 \cdot 50 = 68.25$, IV score 70 $\Rightarrow$ **FAVOURABLE**. GBM with the front IV gives, say, $P_{\downarrow} = 0.19$, $P_{\uparrow} = 0.18$, $P_{\text{either}} = 0.37 \leq 0.50$: passes.
7. **Gates.** Bounded; net EV positive when IV exceeds realised volatility; max loss $1{,}850 \leq \min(25{,}000,\ 20{,}000)$; liquidity score ≥ 0.5; credit $1{,}560 \geq 0.08 \times 50$.
8. **Winner.** Each surviving candidate's TQS and $RAR = net_ev/max_loss$ are computed; suppose the iron condor maximises the selection score.
9. **Freeze.** The four contracts are qualified exactly; an `OptionSelectionSnapshot` stores the full evidence; 24-hour quote leases open on the four legs. When all four have execution-usable quotes the plugin emits `ENTER_OPTIONS` with quantity targets +65, −65, −65, +65 (buy legs marked `hedge_first`); state **ENTRY_PENDING**.

9.2.3 Execution

1. The snapshot uses the exact quantities (no weight conversion; unattributed holdings in these contracts are preserved). The rebalance creates four intents: the two long wings in stage 0 (`OPTION_HEDGES`, eligible) and the two shorts in stage 1 (ineligible).
2. Risk approves the wings (cash for their premium). They are placed at IBKR with the full option identity and fill.
3. All stage-0 orders are terminal with fill ratio $1.0 \geq 0.95$: the shorts become eligible (phase `OPTION_SHORTS`). Each short passes the coverage check against its now-filled same-expiry wing; collateral $|24{,}900 - 24{,}850| \times 1 \times 65 = 3{,}250$ per side is reserved. The shorts are placed and fill. The rebalance completes.

9.2.4 Holding and exit

On each subsequent daily bar the plugin holds, restating the leg targets, and the dynamic exit controller evaluates the soft exits from live leg quotes. Suppose after a week the structure is worth

a profit of 1,050 (75% capture) and the probability of giving back more than a quarter of the peak is 0.45: the `SOFT_PROFIT_GIVEBACK_RISK` exit fires. The plugin emits `EXIT_OPTIONS` with a `FLAT` target on each leg; the rebalance buys back the two shorts first (stage 0) and only then sells the two wings (stage 1). Had no soft exit fired, the structure would have been closed on expiry day (`exit_dte=0`).

PART IV

Broker Connectivity

CHAPTER 10

Broker Connectivity

The backend never talks to the IBKR TWS API itself. Every IBKR login becomes a *managed session* backed by its own IB Gateway child container. The backend sends durable commands to the child over HTTP and pulls back an append-only event stream, which it projects into orders, fills, positions, accounts and market data. A periodic reconciliation compares broker truth with internal records.

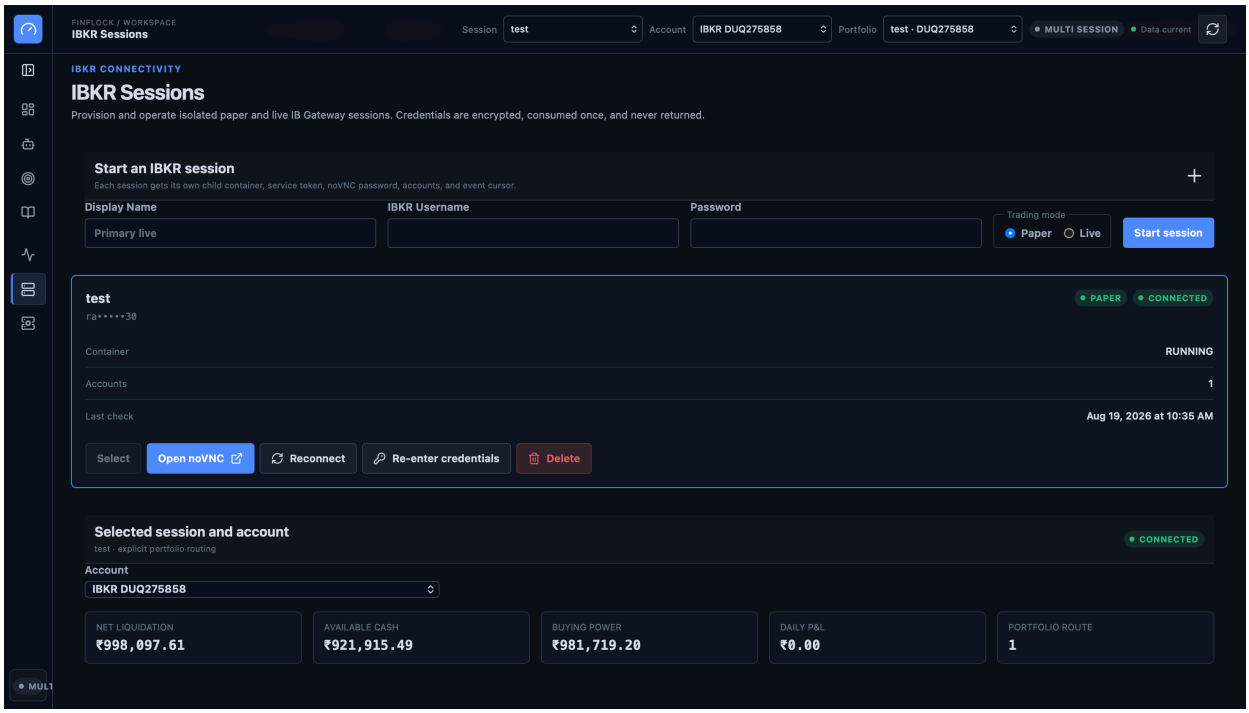


Figure 10.1: The IBKR sessions page: each session is a managed child container with its own login, mode, accounts and remote-desktop access for login and two-factor authentication.

10.1 Managed sessions

10.1.1 The session model

A `BrokerGatewaySession` (UUID primary key) holds a display name, a masked username hint (the raw username is never stored on the session), `mode` (`paper` or `live`), status, child container id and name, internal base URL, the encrypted per-session gateway token and noVNC password, `commands_enabled`, the last gateway health and QCH states, the last error, and a `lifecycle_version` bumped on every meaningful transition.

10.1.2 Provisioning through QCH

QCH is the hosting platform's app-scoped container broker, reached at `{QCH_API_HOST}/api/apps/{QCH_APP_ID}/containers` with a bearer service token (read from the environment on every call so a rotated token is never cached).

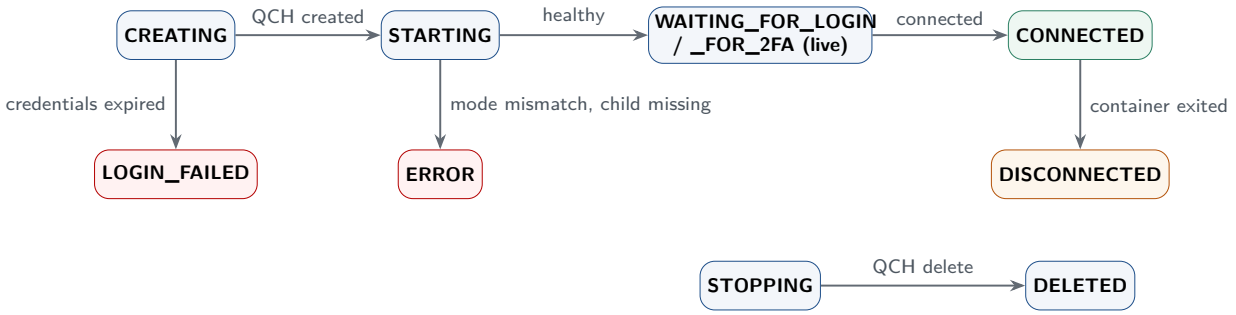


Figure 10.2: Gateway session lifecycle. Any active session can be deleted (STOPPING → DELETED); new credentials return a session to CREATING.

1. POST `/broker-sessions/` with display name, IBKR username and password, and mode. The browser can never supply an image or registry credentials (such fields are rejected). The backend generates a gateway token and noVNC password, stores the session and a *temporary* encrypted copy of the credentials (15-minute expiry), and queues provisioning. It returns 202 without any secret.
2. The provisioning task validates the configured image *before* decrypting any credentials, then creates the child with name `trading-engine-ibkr-<uuid hex>`. A 409 conflict adopts the existing container, and an ambiguous timeout looks the name up before retrying, so creation is idempotent.
3. Once QCH confirms the container, the temporary credentials are deleted: they are single-use.
4. The child environment contains the credentials, `IBC_TRADING_MODE`, the gateway token, the noVNC password, a fresh Django secret, remote desktop enabled (so a human can complete login or 2FA), and optionally per-session PostgreSQL settings.

Image pinning. `IBKR_GATEWAY_IMAGE` must be `docker.io/<ns>/<repo>@sha256:<digest>` (for production) or a fixed, non-`latest` tag (for controlled testing). Placeholders are rejected.

Secrets. Tokens, noVNC passwords and temporary credentials are Fernet-encrypted with `BROKER_SESSION_ENCRYPTION_KEY`. Without the QCH settings and key, managed-session endpoints return 503 but the rest of the backend works.

10.1.3 Monitoring

Every 5 seconds the monitor inspects each session: if QCH does not list the child, the child's own `/healthz` is asked, and a child that answers outvotes an empty listing (listing 404s were observed to flap). An exited container becomes **DISCONNECTED**. The authenticated `/api/v1/health/` decides the rest: a mode mismatch is an **ERROR**; not connected is **WAITING_FOR_LOGIN** (paper) or **WAITING_FOR_2FA** (live); connected triggers account synchronisation and event sync. `commands_enabled` equals connected. Transient QCH errors are recorded as "check deferred" without changing the status, so the UI does not flap.

10.1.4 Routing orders to the right session

`GatewayClient.for_portfolio(portfolio)` requires the portfolio to be bound to a session *and* the portfolio's account to be available on that session. Commands additionally require the session to be **CONNECTED**, command-enabled and reporting connected; this is re-checked on every command, so a session that disconnected after the client was built is refused. There is no global or static route.

10.1.5 Deleting a session

Deletion sets the kill switch on every bound portfolio, pauses their enabled strategies, deactivates their market subscriptions, marks linked accounts unreconciled, and deletes the child through QCH. Existing IBKR orders are *not* cancelled; the API response says so.

10.2 Inside the IB Gateway child

The child image (`gateway/IB_gateway`) is a self-contained linux/amd64 container:

Process	Port	Role
Xvfb, fluxbox	display :1	Virtual display; IB Gateway is a Swing application
IBC + IB Gateway	4002 / 4001	IBC automates the login dialog; paper uses API port 4002, live 4001
x11vnc, websockify	5900, 6080	Remote desktop for login and 2FA (managed children opt in)
gunicorn (Django)	8001	The gateway REST API
`broker_worker`	—	The single owner of the TWS connection (file lock), via `ib_async`
nginx	8080	Public front door: `/healthz`, `/readyz`, `/api/v1/`, `/novnc/`

IBC is configured atomically with the trading mode, `ReadOnlyApi=no`, primary session on conflict, a 2FA timeout with re-login, and a daily auto-restart time. `/healthz` is liveness only; `/readyz` requires the database, internal ports, the IB Gateway process, a listening API port, and a connected, reconciled broker. The child stores its state in SQLite by default, or in its own PostgreSQL database (`<IB_GATEWAY_POSTGRES_DB>_<session hex>`). Each session must have its own database, because every child numbers its session row 1 and would otherwise execute another session's commands.

10.2.1 Commands and events

Every write is a durable `GatewayCommand` keyed by an idempotency key (the backend sends `session:{id}:{key}`) and a canonical request hash; a reused key with a different request is a 409 conflict. The worker claims one command at a time with a 30-second lease, executes it through the adapter, and emits a `command.<type>.completed` or `command.failed` event. For an order command whose lease expires, the worker checks IBKR for the order by `orderRef` before doing anything; if the outcome cannot be confirmed, the command is marked **UNKNOWN** and never resubmitted.

Every broker observation is an append-only `GatewayEvent` with a monotonic id: periodic snapshots (accounts, account summary, positions per account, open and completed orders, executions), order status events, market data and errors, and `session.connected`/`session.disconnected`. Acknowledged events are compacted after a retention period.

10.2.2 Gateway API

Endpoint	Purpose
GET `health/`, `diagnostics/`, `session/`	Connection state, mode, connection generation; sanitised diagnostics

Endpoint	Purpose
GET `accounts/`, `positions/`, `open-orders/`, `executions/`	Latest snapshots
POST `contracts/search/`, `qualify/`, `option-chain/`	Contract lookup
POST `market-data/history/`, `schedule/`	Historical bars (up to 10 years daily, 90 days intraday)
POST `market-data/subscriptions/` (and `cancel/`)	Streaming subscriptions
POST `orders/`, PATCH `orders/<id>/`, POST `orders/<id>/cancel/`	Stock orders (MKT, LMT, STP, STP_LMT; DAY or GTC)
GET `orders/<id>/state/`	All knowledge about an order, including proof of non-submission
POST `options/orders/`, `options/quotes/`	Option orders (exact identity) and quotes
GET `events/?after=N`, POST `events/ack/`	Event feed (500 per page) and acknowledgement
POST `kill-switch/`, `session/reconnect/`	Operational controls

10.3 Event synchronisation

Every 5 seconds, `sync_events` reads events after the session's cursor. The child regenerates a `connection_generation` UUID on every (re)connect; when it changes, the cursor resets to 0 and all retained events are replayed, which is safe because every downstream write is idempotent. Each event is projected, the cursor advances, and the batch is acknowledged so the child can compact. A failing event stops the cursor (head-of-line) rather than being skipped.

Event	Projection
`snapshot.accounts`	Ensure `BrokerAccounts`, session mappings and a portfolio per (account, session) named "session · account"
`snapshot.account_summary`	NAV, available cash (AvailableFunds, else TotalCashValue, else CashBalance), buying power, daily P&L; release consumed reservations
`snapshot.positions`	Apply a *complete* per-account snapshot atomically: upsert present positions, zero missing ones; incomplete snapshots are audited but never applied; duplicates apply once
`snapshot.open_orders`, `completed_orders`	Update order ids and statuses
`snapshot.executions`	`apply_execution` for each (session-prefixed execution id)
`broker.order`	Order status event; import unknown broker orders
`market.raw`, `market.error`	Provider gate; fallback handling
`command.*.completed` / `command.failed`	Acknowledge or fail broker commands and subscriptions

Event	Projection
`session.disconnected`	Live orders → UNKNOWN; market subscriptions fail over (code 1100)

10.4 Reconciliation

Every 60 seconds each available account on a connected session is reconciled:

1. The account must resolve to exactly one session. Broker I/O (health, positions, executions) happens before the transaction.
2. **GATEWAY**: gateway not connected and reconciled → a critical material break.
3. **POSITION**: broker and internal quantities are summed per conid for the account's portfolios on that session; any difference → a critical material break.
4. **EXECUTION**: a broker execution missing internally → critical material break; an internal fill not at the broker → a non-material warning (IBKR only returns recent fills).
5. Prior material breaks of a category that is now clean are auto-resolved.
6. The run is **BLOCKED** if any unresolved material break remains, otherwise **COMPLETED**, and `BrokerAccount.is_reconciled` is set accordingly. This flag gates strategy activation, pre-trade risk and final dispatch.

Cash is projected from the broker, not reconciled.

10.5 Remote desktop (noVNC)

Live logins need a human for two-factor authentication. The backend proxies noVNC through its own ASGI router at `/api/v1/broker-sessions/<uuid>/novnc/`:

- The session API returns a URL with a short-lived HMAC access token (5 minutes) in the URL *fragment*, so it never reaches server logs. A connect page exchanges it for an HttpOnly, SameSite=Strict cookie.
- The upstream is always derived from the database (`http://<child>:8080/novnc/`), never from the client; host-like query parameters are rejected.
- The proxy performs the VNC password handshake itself (DES challenge-response with the stored password) and offers the browser "no authentication", so neither the password nor the challenge ever reaches the browser.

10.6 Accounts and portfolios

A `BrokerAccount` is global and unique by IBKR account id. A `TradingPortfolio` belongs to one account and is bound to one session; its execution mode is *derived* from the session's mode. It carries `cash_buffer_pct` (0.02), `margin_buffer_pct` (0.10), minimum notional (10), minimum quantity, minimum drift (0.001) and a kill switch. Positions are overwritten by complete broker snapshots; ledgers are append-only with unique keys.

PART V

Portfolio Intelligence

CHAPTER 11

Research and Recommendations

The research subsystem answers the question *"which stock, run by which strategy, suits this goal?"*. It turns two static catalogues — a universe of 500 US large-cap stocks with GICS classification and a universe of 97 strategy hypotheses in 10 families — into backtested, scored, goal-specific recommendations that the Portfolio Builder can attach to a user's plan.

11.1 Pipeline overview

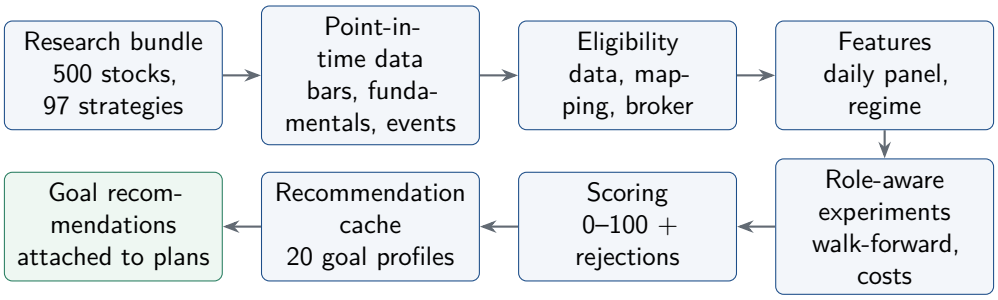


Figure 11.1: Research layers, from catalogue to recommendation.

> **Product constraints enforced everywhere**
>
> Long-only; exactly one primary strategy per stock; goals with a "NOW" horizon are cash-only; at least 756 daily bars of history; pair and basket strategies are research-only and never trade.

11.2 The research bundle

`research_bundle/` holds six JSON documents, each with a JSON Schema:

File	Content
`manifest.json`	Bundle name and date, counts, distributions, warnings (survivorship bias, unlicensed GICS transcription, strategies are hypotheses), and the size and SHA-256 of every file
`stock_universe.json`	500 stocks with GICS at four levels, issuer metadata (CIK, headquarters), eligibility thresholds (756 days of history, price $\geq$ \$5, 20-day median dollar volume $\geq$ \$25M, 240 of 252 trading days) and risk/timeframe priors
`strategy_universe.json`	97 strategies: family, scope, hypothesis, features and formulas (never evaluated), signal logic, parameter grid, sizing and caps, risk levels, goal timeframes, required metrics, known failure modes
`gics_taxonomy.json`	11 sectors, 25 industry groups, 74 industries, 163 sub-industries

File	Content
backtest_spec.json	Protocol daily_equity_walk_forward_v1: next-bar execution, costs (1 bp commission, spread, square-root impact, stress scenarios), caps, walk-forward validation, robustness tests, scoring weights, minimum score 65
compatibility_rules.json	Seven global rules and eight GICS overrides

Validation checks the manifest hashes, the schemas and semantic invariants (exact counts, unique codes and symbols, consistent GICS hierarchy). Import is idempotent per (bundle, version, manifest hash) and creates the dataset version, taxonomy, universe members (mapped to issuers and instruments), feature and strategy definitions, the protocol and rule set, and synchronises the implementation registry.

11.3 Strategy implementations

JSON formulas are specifications only. Each catalogue id is bound to an audited Python implementation in a hard-coded registry, with an implementation hash over its source code. The *role* of a strategy decides its backtest engine:

Role	Engine	Families and examples
EXECUTION (30)	single asset	buy-and-hold; 15 trend (SMA/EMA crosses, triple MA, Donchian, 200-day with slope, MACD, ADX, supertrend, Keltner, 10-month); 5 time-series momentum; 9 mean reversion (RSI, Bollinger, MA distance, stochastic, Williams, Connors, gap, VWAP proxy)
SELECTOR	cross-sectional	momentum rankings, reversal rankings, 16 composite factors (value, quality, profitability, investment, low volatility, low beta, dividends, revisions)
INCOME	cross-sectional	dividend and shareholder-yield composites
ALLOCATOR	allocator	equal weight, sector-neutral, inverse volatility, risk parity, minimum variance, maximum diversification, HRP, minimum CVaR, core blend
OVERLAY	overlay	volatility, drawdown, trend-volatility, ATR, correlation shock, market regime, liquidity stress scalers
EVENT	event	post-earnings drift, earnings gap, pre-earnings avoidance, turn of month, month end, ex-dividend, index change, split
PAIR_BASKET	pair/basket	eight peer-model labels; research only

EXECUTION strategies are also upserted as runnable StrategyDefinitions using the CatalogueLongOnlyPlugin runtime adapter.

11.4 Backtest engines

11.4.1 Single-asset engine

Positions lag signals by one bar ($pos_t = desired_{t-1}$); returns are open-to-open; exposure is bounded to $[0, 1]$ (long only). Each position change Δ costs

$$cost_t = |\Delta| \cdot \frac{commission + spread/2 + stress + impact \cdot \sqrt{participation} \cdot 10^4}{10^4},$$

with 1 bp commission, 5 bp spread and impact coefficient 0.1; stress runs add 25 and 50 bp. Metrics: total return, CAGR, annualised volatility, Sharpe and Sortino ($r_f = 0$), maximum drawdown, Calmar, drawdown duration, turnover, trade count, exposure, hit rate and profit factor.

11.4.2 Other engines

- **Cross-sectional**: rank a point-in-time panel, select the top quantile (default 20%) with retention of held names and an optional turnover cap; return = equal-weighted forward return after the decision date.
- **Allocator**: weights from the sample covariance (plus $10^{-8}I$) by the chosen method, capped per stock; optimisation methods use SLSQP.
- **Overlay**: multiply all weights by a scalar, e.g. τ/σ for volatility targeting, 0 beyond a drawdown trigger, regime multipliers CALM 1.0, NORMAL 0.9, STRESSED 0.5, CRISIS 0.25.
- **Pair/basket**: peers within sub-industry, bounded to $\pm k$ liquidity neighbours, correlation ≥ 0.5 over at least 60 overlapping returns.
- **Event**: only events available before the decision timestamp are used.

11.4.3 Point-in-time discipline

Bar reads filter provider and revision timestamps to the as-of time; fundamental restatements become public no earlier than their retrieval time (never backdated); unknown announcement times map conservatively to the next session open; Donchian channels use prior bars; scoped experiments require features available before the decision timestamp; and the last 126 bars of every single-asset run are held out.

11.5 Experiments

Experiments are created per role (one per stock for EXECUTION, one per stock with events for EVENT, one per universe otherwise) with an identity hash over dataset, protocol, implementation hash, feature version, instrument, parameter grid, dates and data version — so new experiments appear only when data or code change. Trials are the Cartesian product of the parameter grid (the baseline always included; a deterministic shuffle when over budget).

For single-asset experiments: walk-forward windows (train 252, purge 5, validation 63, embargo 5, test 63, at least 3 windows), four runs per trial (research span, full, +25 bp, +50 bp), window metrics, subperiod consistency, holdout metrics, capacity (252-day median dollar volume) and a deflated Sharpe ratio $\mathrm{DSR} = \Phi((\mathrm{SR} - E_{\max})/\mathrm{SE})$.

11.6 Scoring

Component	Weight	Normalisation (clamped to 0–100)
out-of-sample Sharpe	0.20	$50 + 20\,\mathrm{SR}$
Calmar	0.15	$50 + 15\,\mathrm{Calmar}$
drawdown fit	0.15	$100(1 - \mathrm{MaxDD}/0.40)$
regime consistency	0.15	$100\times$ share of positive windows
parameter stability	0.10	100 if Sharpe $\geq$ median of neighbours -0.5, else 0
cost resilience	0.10	$50 + 500\,R_{50\,\mathrm{bp}}$
turnover efficiency	0.05	$100 - 5\,\mathit{turnover}$
capacity	0.05	ADV / \$1M
diversification	0.05	50

A candidate is **eligible** if its score is at least 65 and it has no hard rejection: data quality failure, timestamp ambiguity, a negative result at 50 bp costs, drawdown above 40%, insufficient activity, unstable parameter neighbourhood, capacity failure, dependence on one subperiod ($> 60\%$ of gains), multiple testing failure (DSR < 0.5), or unprotected holdout. Scores expire after 7 days. Strategy readiness progresses **VALIDATED** → **BACKTESTED** → **SCORED** → **APPROVED_FOR_RECOMMENDATION** → **PAPER_VALIDATED** → **BUILDER_READY**.

11.7 Goals and the recommendation cache

A goal has a **timeframe** (NOW ≤30 days, HURRY 1–3 months, FAST 3–12 months, BUILD 1–3 years, GROW 3–7 years, COMPOUND 7+ years) and a **risk level** (1 Preservation to 5 Aggressive). The rules table fixes the maximum risk per timeframe (NOW 1, HURRY 2, FAST 3, BUILD 4, GROW and COMPOUND 5), giving 20 valid profiles, the minimum cash (the larger of a timeframe floor 1/0.70/0.40/0.20/0.05/0.02 and a risk floor 0.80/0.50/0.25/0.10/0.02), the maximum stock weight (0.05 to 0.25 by risk) and the stock count.

For each profile a daily cache snapshot is built:

1. **Tier 1 (full model)**: for each stock with a current stock role score, take its best eligible EXECUTION candidate (one strategy per stock), ranked by stock score plus execution score.
2. Fallbacks when research is incomplete: tier 2 copies a recent tier-1 snapshot; tier 3 uses a fixed price-only strategy per timeframe; tier 4 buy-and-hold; tier 5 the latest snapshot.
3. **Selection** diversifies one name per sector first, then fills by rank.
4. **Weights** from the best allocator (fallback inverse volatility), scaled to the investable share and capped by stock cap and capacity; then the top two overlays.

`/readyz` reports ready only when all 20 profiles have an unexpired snapshot (and exactly 97 implementations are registered).

11.8 Recommendation batches

POST `/portfolio-construction/plans/<id>/recommendations/` (idempotent, throttled) checks builder readiness, then asynchronously: reads the cached snapshot for every goal; qualifies the finalists as exact IBKR contracts, substituting lower-ranked names on failure; re-optimises

each goal's sleeves; and attaches instrument selections and strategy assignments to the plan, bumping its version once. It creates no orders, rebalances or strategy instances.

The sleeve optimiser solves, with SLSQP,

$$\min_{w} \; -(\mu - c)^{\top} w + \lambda\, w^{\top} \Sigma w + 0.01 \lVert w - w_0 \rVert_1 + 0.02 \lVert w \rVert^2 + \iota^{\top} w + \kappa \lVert w - w_{\mathrm{rec}} \rVert^2$$

subject to $\sum w = 1 - \mathit{min_cash}$, $0 \le w_i \le \min(\mathit{cap}, \mathit{capacity}_i)$, caps per strategy family (0.40), sector (0.35), industry (0.25) and sub-industry (0.20), a minimum number of sectors and a turnover limit, with $\lambda = 6 - \mathit{risk}$ and $\kappa = 2$.

CHAPTER 12

Portfolio Construction, Optimisation and Capital Flows

Three mechanisms decide *how much* capital each strategy and instrument should have: the goal-based **Portfolio Builder**, the **Markowitz optimiser**, and **capital-flow allocation** for deposits and withdrawals. All of them end in the same rebalance pipeline of Chapter 8.

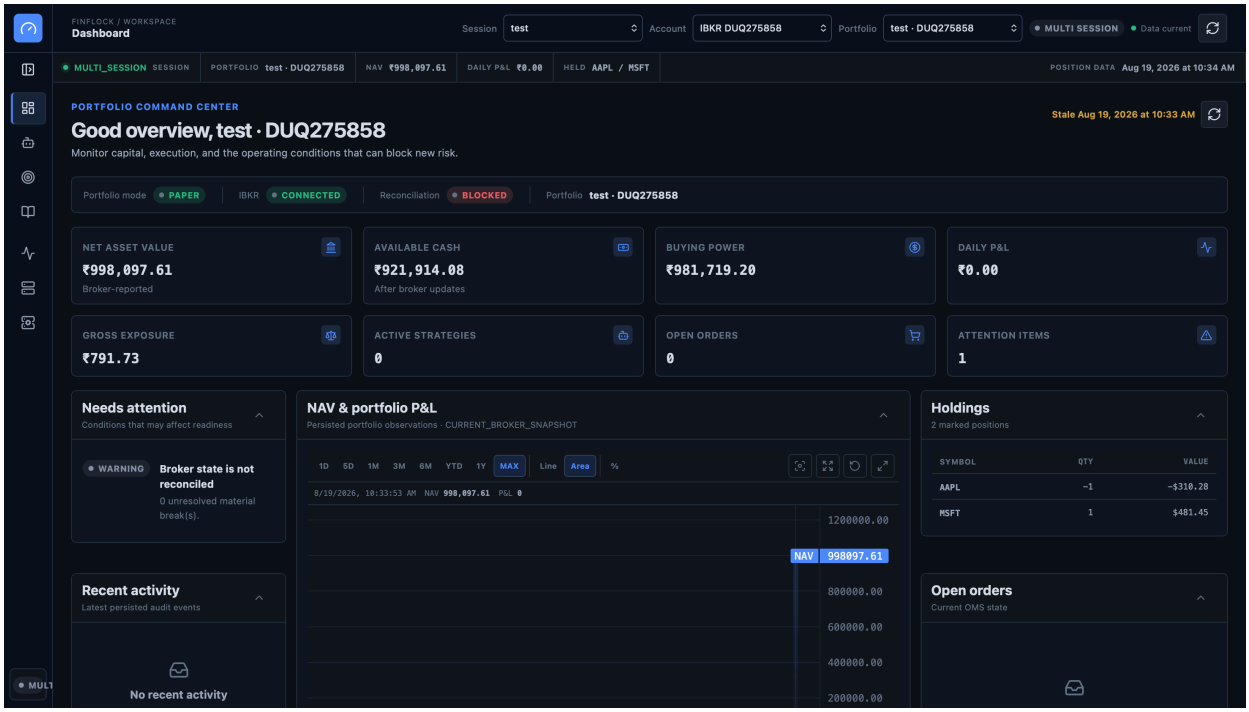


Figure 12.1: Portfolio overview. NAV, cash and exposures are broker-projected; positions are valued at the freshest trusted price.

12.1 The Portfolio Builder

12.1.1 Model

Model	Purpose
`PortfolioConstructionPlan`	One per portfolio; DRAFT, ACTIVE or PAUSED; a monotonically increasing version
`PortfolioGoalAllocation`	A goal: allocation weight, timeframe, risk level, source (manual optimiser or accepted recommendation)
`StrategyConstructionProfile`	Which goal timeframes and risk levels a strategy definition may serve, and whether users may select it
`GoalInstrumentSelection`	A stock chosen for a goal, with optional minimum and maximum weight

Model	Purpose
`GoalStrategyAssignment`	A strategy for that stock: definition, execution timeframe, parameters, share of the stock, policies, and the instance created for it
`PortfolioConstructionRun`	A calculation (QUEUED → CALCULATING → COMPLETED/FAILED) and its application status (NOT_APPLIED → QUEUED → APPLYING → ACTIVATING → APPLIED)
`PortfolioConstructionTarget`	Per-instrument target weight and value, return and risk contributions

12.1.2 Validation

A plan needs 1–10 enabled goals whose allocations sum to exactly 1. A strategy is eligible for a goal only if its plugin is available and enabled, it has an enabled construction profile (user-selectable unless system-generated), it supports the goal's timeframe and risk, the goal is not NOW, and it trades long `STK`. Enabled assignments per stock must sum to exactly 100%.

12.1.3 Calculation

For each goal: an accepted recommendation's stock weights are used verbatim (re-checked against caps and cash); a NOW goal is 100% cash; a single stock gets $\min(1 - \mathit{min_cash}, \mathit{cap})$; several stocks are optimised with Markowitz (minimum variance for risk ≤ 3, maximum Sharpe for 4–5). Portfolio weights are the goal-weighted sum $w_i = \sum_g a_g \, w_{g,i}$; each stock's weight is split across its assignments by share, and *strategy targets* are aggregated by identity (definition, instrument, timeframe, parameter hash, policies).

12.1.4 Preview and apply

A preview runs the calculation and a *preview* rebalance. Applying (idempotent, requires execution readiness):

1. Re-checks the run, the plan and unexpired recommendations.
2. Creates or reuses a `StrategyInstance` per strategy target (reusing only disabled or builder-owned instances; enabled user instances are never reused) with `target_configuration` carrying the target weight and the construction run id.
3. Queues an idempotent enable action per instance. Actual trading is driven by each strategy's own live targets, not by the builder: the application status tracks the instances through activation, warm-up, first evaluation and execution.

12.2 The Markowitz optimiser

A portfolio can define an optimisation universe (2–50 active tradable stocks, at least 60 observations) and a policy (minimum variance or maximum Sharpe, 252-day lookback, historical mean and sample covariance, target cash 0.02, weight bounds, maximum turnover, transaction-cost penalty, long only; the execution mode is derived from the session). With annualised $\mu = 252\,\bar{r}$ and $\Sigma = 252\,\widehat{\mathrm{Cov}}$ (symmetrised, with an eigenvalue floor of 10^{-8}):

$$\text{Minimum variance:} \quad \min_{w} \; w^\top \Sigma w + p\,\|w - w_0\|_1$$

$$\text{Maximum Sharpe:} \quad \min_{w} \; -\frac{\mu^\top w - r_f}{\sqrt{w^\top \Sigma w}} + p\,\|w - w_0\|_1$$

subject to $\sum w = 1 - \mathit{cash}$, $\ell_i \le w_i \le u_i$ and $\|w - w_0\|_1 + |\mathit{outside}| \le \mathit{max\ turnover}$, solved with SLSQP (1000 iterations, tolerance 10^{-12}) from a feasible water-filled start. Outputs include expected return, volatility, Sharpe, turnover and risk contributions $RC_i = w_i(\Sigma w)_i/\sigma_p$. Applying an optimisation run creates an *execution* rebalance directly.

12.3 Capital flows

POST `/allocations/flows/` records a deposit or withdrawal idempotently and allocates it, either by strategy allocation or, when an optimisation policy exists, by re-optimisation followed by an execution rebalance.

Deposit
: Investable = amount − the top-up needed to restore the cash buffer. It is distributed to strategies in proportion to their deficit $\max(\max(\mathit{target}, \mathit{min}) \cdot NAV' - \mathit{current}, 0)$, up to each strategy's maximum share and capacity, with leftovers by priority.

Withdrawal
: Funded from portfolio cash, then strategy idle cash, then strategy surplus above target, then position liquidation (proportional, or by conviction, liquidity, cost or priority). Anything left is *unfunded*. Pending withdrawals reserve cash so other operations cannot spend it.

Each allocation writes capital snapshots and decisions and adjusts `StrategyInstance.allocated_capital`, which the target snapshot uses as the strategy's capital share.

PART VI

Operations

CHAPTER 13

Platform and Operations

13.1 Technology stack

Area	Components
Web	Django 5.1, Django REST framework (function views returning JSON), django-cors-headers, Gunicorn with Uvicorn ASGI workers, httpx and websockets for the noVNC proxy
Tasks	Celery 5.4 on Redis; django-celery-results (errors only)
Data	PostgreSQL via psycopg 3
Streaming	confluent-kafka 2.8; Apache Flink 1.20.1 with PyFlink in a separate Python 3.11 environment and a shipped relocatable runtime
Numerics	numpy, scipy, pandas, pyarrow — imported lazily (about 158 MB saved per process that does not need them)
Other	jsonschema, prometheus-client, cryptography (Fernet), python-dotenv
Gateway child	Python 3.12, Django, `ib_async`, IBC 3.23, IB Gateway (stable channel, version verified at build), nginx, x11vnc, noVNC
Tests	pytest, pytest-django, responses

13.2 Scheduled work (Celery beat)

Entry	Task	Every	Queue
strategy-evaluation	`strategies.tasks.execute_strategy_evaluation_jobs`	1 s	`strategy_evaluation`
coordinate-portfolio-targets	`rebalancing.tasks.coordinate_portfolio_targets`	1 s	`target_coordination`
execute-order-intents	`execution.tasks.execute_order_intents`	1 s	`intent_execution`
dispatch-broker-commands	`execution.tasks.dispatch_broker_commands`	1 s	`broker_commands`
publish-outbox	`event_bus.tasks.publish_outbox_events`	2 s	default
sync-broker	`broker_gateway.tasks.sync_broker_events`	5 s	default
monitor-market-data-providers	`market_streams.tasks.monitor_market_data_providers`	5 s	default
restore-market-subscriptions	`market_streams.tasks.restore_active_market_subscriptions`	15 s	default
recover-* (evaluation, intents, commands)	lease and claim recovery	30 s	per path

Entry	Task	Every	Queue
warmup-timeouts	`market_streams.tasks.check_warmup_timeouts`	30 s	default
stream-health	`event_bus.tasks.check_stream_health`	30 s	default
reconcile	`reconciliation.tasks.run_scheduled_reconciliation`	60 s	default
instrument-registry	`instruments.tasks.publish_instrument_registry_snapshot`	60 s	default
recover-rebalances	`rebalancing.tasks.recover_incomplete_rebalances`	60 s	`target_coordination`
research-experiment-dispatch	`research.tasks.dispatch_research_experiments`	60 s	celery
research refresh (intraday, events)	research data tasks	1 h	`research_*`
Finnhub history and mappings	`market_data.tasks.*`	6 h	default
research daily pipeline, scoring, cache	research tasks	24 h	`research_*`
compact-operational-records	retention (30 days)	24 h	default

The order-path intervals are configurable through `*_INTERVAL_SECONDS` environment variables; each is the maximum latency added at that hop.

13.3 API conventions

- All endpoints live under `/api/v1/` and are also mounted under `APP_BASE_PATH` (`/trading_eng_backend` on QFS), so the service works whether or not the proxy strips the prefix. Trailing slashes are optional (middleware appends them internally, preserving request bodies).
- Every JSON response uses the envelope `{ok, data, error, meta}`; errors are `{code, message, details}` with upper-snake codes. Lists paginate with `limit`/`offset` (core) or `page`/`page_size` (research).
- Asynchronous acceptance returns 202; conflicts 409 (`IDEMPOTENCY_CONFLICT`, `RETRY_NOT_ALLOWED`); risk or price problems 422; throttling 429; unavailable dependencies 503; gateway command timeouts 504.
- **Idempotency.** State-changing endpoints require an `Idempotency-Key` header. The server stores a canonical SHA-256 request hash (keys sorted, numbers normalised so 1, 1.0 and 1.00 are equal); a replay with the same key and payload returns the original result, a different payload returns 409. A failed operation is re-run only with `Idempotency-Retry: true` and only if it was marked retryable.
- **Throttling.** Expensive operations (optimisation, construction preview, recommendations, Finnhub configuration) use a fixed-window counter in the Django cache.

The complete endpoint catalogue is in Appendix A.

13.4 Health, readiness and observability

Endpoint	Meaning
GET `/healthz`	Process liveness (container health check)
GET `/readyz`	Database reachable and, if the recommendation system is enabled, a complete active research universe, 97 implementations and 20 warm cache profiles
GET `/api/v1/system/`	Mode (multi-session), live-trading flag, broker deployment readiness, global kill switch, count of material reconciliation breaks
GET `/api/v1/execution/readiness/`	Fail-closed automatic-execution readiness (Section 8.10); 503 with blockers
GET `/api/v1/execution/diagnostics/`	The same payload, always 200
GET `/api/v1/execution/flink-diagnostics/`	Flink jobs, exceptions, checkpoints and a probable-cause classification
GET `/api/v1/streaming/health/`	Data-path health: Kafka, consumer heartbeat, Flink, per-strategy stream status, outbox and dead letters, provider states
GET `/metrics`	Prometheus gauges: outbox pending, dead letters, stale instruments, subscriptions by provider, provider transitions, Finnhub counters, research and recommendation freshness

13.5 Dashboard analytics

- **Summary**: positions valued at $q \times price \times m$, where the price is a fresh broker position price if available, else the freshest execution-usable market price; gross and net exposure; NAV, cash, buying power and daily P&L from the broker; counts of active strategies, open orders and positions; and an attention list (gateway disconnected, unreconciled, kill switches, stale instruments, blocked strategies, held or rejected risk checks).
- **NAV series**: reconstructed from *current holdings* times historical bar closes plus available cash, anchored so the last point equals broker NAV. It is an estimate, not a historical NAV record.

13.6 Security model

- Broker credentials are used once to provision a child and then deleted; tokens and passwords are encrypted at rest; no secret is ever returned by the API; the noVNC password never reaches the browser.
- Staff login (`/auth/session/`) sets a Django session and reports `is_admin` to the frontend. **API endpoints do not enforce authentication at the application layer**; access control relies on the network and the platform proxy. CSRF protection applies to non-exempt unsafe endpoints; several order and strategy endpoints are CSRF-exempt.
- Live trading needs `ALLOW_LIVE_TRADING`; any kill switch (global, account, portfolio, instrument, strategy) stops new orders while still allowing cancels.

13.7 Configuration

Settings load the process environment first, then optional `.env` files (never overriding the environment). The main groups (secret values omitted; see Appendix B for names and defaults):

- Web: `PORT`, `APP_BASE_PATH`, `PUBLIC_BASE_URL`, `ALLOWED_HOSTS`, CORS and CSRF origins.
- Data stores: `DATABASE_URL`, Redis and Celery URLs, `CLEANUP_DATABASE_ON_START`.
- Order path cadences and limits; strategy evaluation retries; target coordination debounce and age.
- Kafka and Flink: bootstrap servers, required topics and jobs, runtime archive, bootstrap timeouts.
- Broker: gateway timeouts, QCH, gateway image, session encryption key, child database.
- Policy: `ALLOW_LIVE_TRADING`, `GLOBAL_KILL_SWITCH`, `OPTION_PRETRADE_RISK_ENABLED`.
- Market data: staleness, leases, warm-up, option chain provider, remote feature database, Finnhub fallback.
- Research: enable flags, universe, recommendation sizes, lookbacks, parallelism.

Destructive startup flag

When `CLEANUP_DATABASE_ON_START=true`, *every* container start cancels the Flink job, empties the Kafka topics and deletes all database rows before re-seeding. The committed example environment file sets it to true and is copied into the image. Production deployments should set it explicitly to false.

13.8 Local validation stack

`docker-compose.options.yml` starts PostgreSQL 16, Redis 7, a single-node Kafka 3.9 (with a durable volume), a Flink 1.20.1 JobManager and TaskManager, a paper IB Gateway built from `gateway/IB_gateway` (health-checked on `/readyz`, not `/healthz`), and the backend with live trading disabled. Docker needs at least 12 GiB of memory, or the TaskManager is killed once the pipeline starts. Ports: backend 18000, Flink UI 18081, gateway and noVNC 18082 (all bound to localhost).

13.9 Testing

The suite (`tests/`, 54 modules, about 537 tests) runs on in-memory SQLite with Kafka disabled (`config/test_settings.py`). Helpers bind a portfolio to a connected, reconciled fake session without external I/O. Coverage spans the API contract and health; broker sessions, the gateway client, sync and command dispatch; OMS, risk and reconciliation; readiness, Flink bootstrap and diagnostics, event bus and streaming determinism; Finnhub fallback and remote features; the strategy framework, lifecycle, evaluation jobs, deletion and reporting; stock and option strategies; target coordination, rebalancing, sizing, allocation, optimisation and construction; research and recommendations; an end-to-end automatic PAPER path; and one Docker-based integration test (marker `docker_integration`). The gateway image has its own test suite under `gateway/IB_gateway/tests`.

CHAPTER 14

Data Model

All state lives in PostgreSQL. Primary keys are big integers except for gateway sessions (UUID). Deletion is almost always `PROTECT`, so financial history can never be removed by cascading deletes.

14.1 The relational spine

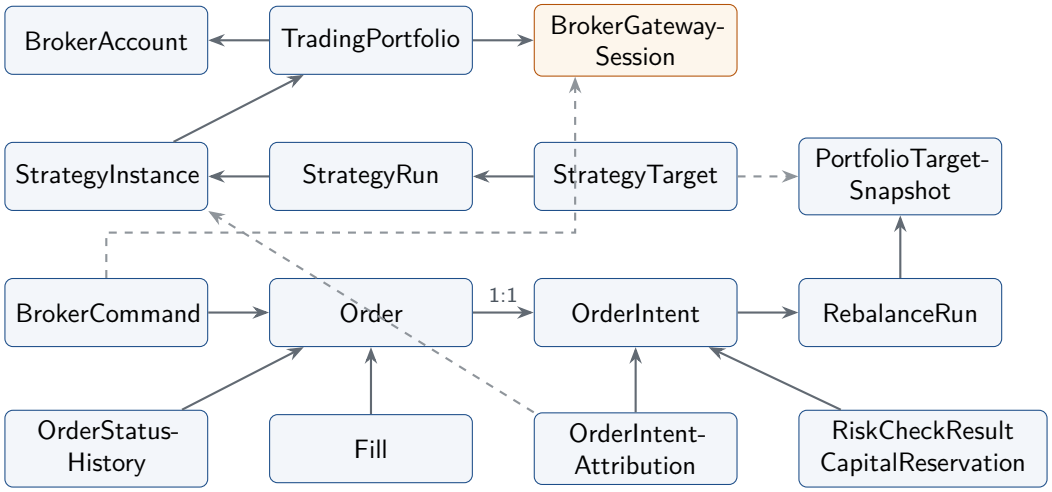


Figure 14.1: The main relationships. Solid arrows are foreign keys (child → parent); dashed arrows are logical links (targets feed snapshots; attribution links intents back to strategies; commands route through a session).

14.2 Operational events and history

The repository includes an ER diagram of the audit, execution-history and event-streaming tables (`docs/operational-events-er-diagram.svg`), reproduced in Figure 14.2.

14.3 Models by application

App	Models
`accounts`	`BrokerAccount`
`broker_gateway`	`BrokerGatewaySession`, `BrokerGatewaySessionSecret` (temporary credentials), `BrokerSessionAccount`, `BrokerSyncCursor`, `BrokerPositionSnapshot`
`instruments`	`Issuer`, `Instrument`, `BrokerContract`, `OptionContract`, `InstrumentProviderMapping`
`portfolios`	`TradingPortfolio`, `PortfolioPosition`, `CashLedgerEntry`, `PositionLedgerEntry`

App	Models
strategies	StrategyDefinition, OrderPolicy, StrategyRiskPolicy, StrategyInstance, StrategyVersion, StrategyInputRequirement, StrategyInputBinding, StrategyWarmupReadiness, StrategyRun, StrategySignal, StrategyTarget, OptionSelectionSnapshot, StrategyAttributedPosition, StrategyAllocation, StrategyAction
market_streams	MarketDataSubscription, MarketDataConsumerLease, MarketDataProviderTransition, MarketBar, IndicatorValue, StrategyEvaluationJob, InstrumentMarketState
market_data	MarketDataProviderConfiguration, InstrumentPriceHistory, MarketDataFetchRun
allocation, rebalancing	PortfolioFlow, AllocationRun, StrategyCapitalSnapshot, AllocationDecision, RebalancePolicy, PortfolioTargetSnapshot, RebalanceRun, PortfolioTargetCoordination, TargetPortfolioPosition, OrderIntentAttribution
oms	OrderIntent, Order, OrderStatusHistory
execution	BrokerCommand, Fill
risk	KillSwitch, RiskCheckResult, PreTradeRiskPolicy, CapitalReservation
position_sizing	PositionSizingPolicy, PositionSizingDecision
reconciliation	ReconciliationRun, ReconciliationBreak
audit	AuditEvent, OutboxEvent, OperationAttempt
event_bus	ConsumedEvent, DeadLetterEvent, ReplayRequest, StreamHealthMetric
portfolio_ optimization	PortfolioUniverse, PortfolioUniverseInstrument, PortfolioOptimizationPolicy, PortfolioOptimizationRun, OptimizedPortfolioTarget
portfolio_ construction	PortfolioConstructionPlan, PortfolioGoalAllocation, StrategyConstructionProfile, GoalInstrumentSelection, GoalStrategyAssignment, PortfolioConstructionRun, PortfolioConstructionTarget
research (36 models)	dataset and taxonomy (ResearchDatasetVersion, BacktestProtocolVersion, CompatibilityRuleSet, GICSTaxonomyNode, InstrumentClassification); universe and eligibility; strategy catalogue and implementations; point-in-time data stores; experiments and trials; candidate and role scores; feature and regime snapshots; recommendation policies, runs, sleeves, acceptances, cache snapshots and batches

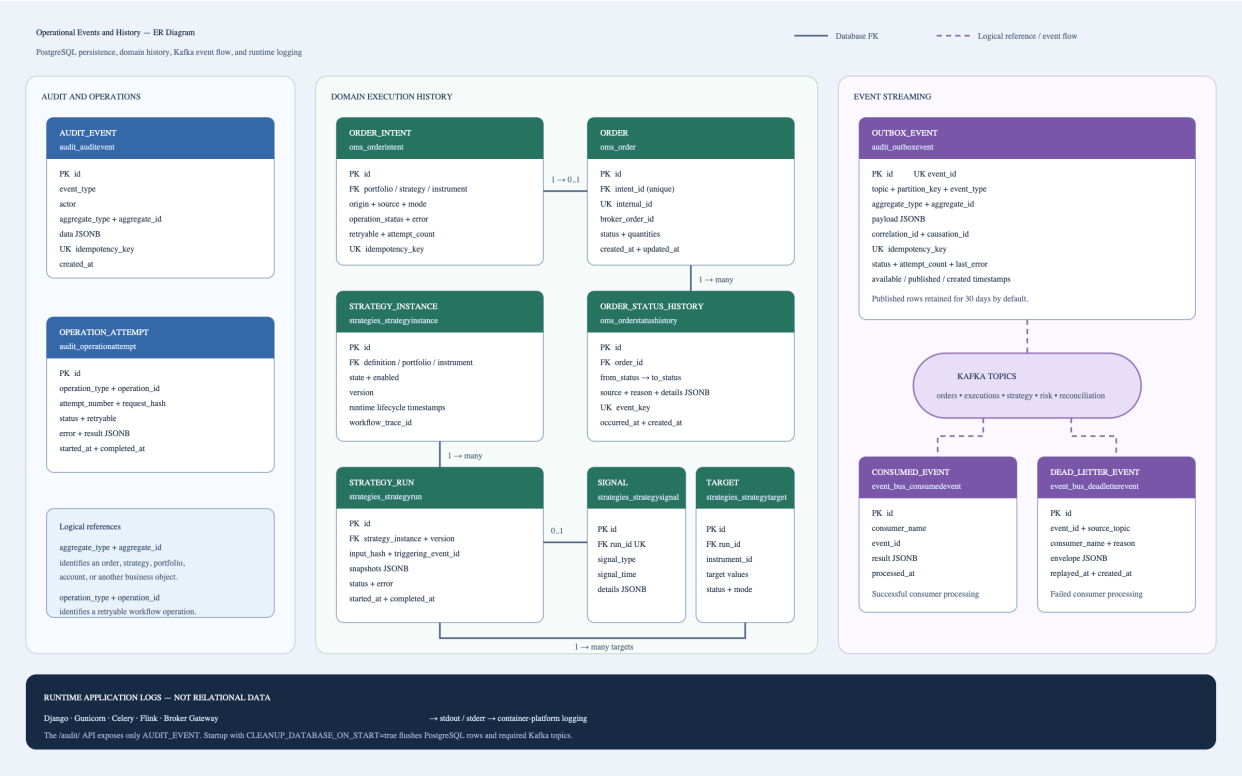


Figure 14.2: Operational events and history: audit and operation attempts (left), domain execution history (centre) and event streaming through the outbox, Kafka, consumed events and dead letters (right). Runtime logs go to stdout and the platform's logging, not to the database.

14.4 Uniqueness guarantees that make the system idempotent

Record	Unique key
`OrderIntent`, `BrokerCommand`, `RebalanceRun`, `PortfolioFlow`	`idempotency_key` (plus a stored request hash)
`StrategyRun`	idempotency key per (instance, version, bar, bar version)
`StrategyEvaluationJob`	(instance, version, bar id, bar version)
`MarketBar` / `IndicatorValue`	(bar id, version) / `source_key`
`OrderStatusHistory`	`event_key`
`Fill`	`execution_id` (session-prefixed)
Ledger entries	`position:{execution}` / `cash:{execution}`
`ConsumedEvent`	(consumer name, event id)
`OutboxEvent`, `AuditEvent`	`idempotency_key`
`PortfolioTargetSnapshot`, `PositionSizingDecision`, `CapitalReservation`	idempotency key
`RebalanceRun` (partial)	one active automatic run per portfolio

CHAPTER 15

Known Limitations and Review Notes

This chapter lists behaviours found while reading the code for this document that differ from what a reader might expect, or from other documentation. They are recorded so they can be confirmed, fixed or accepted deliberately. None of them was changed as part of writing this document.

15.1 Options

1. **Only four structures can be entered.** Cash-secured put, delta put credit spread, long ATM straddle and calendar call are seeded and selectable, but the approved selector accepts only PCS, CCS, IC and IB, so fixed-template instances of the other four never enter (`option_analytics/runtime.py`).
2. **Decision bands are not enforced.** The TRADE_FULL, TRADE_HALF, WATCH and NO_TRADE bands and their size multipliers are computed but not applied; a WATCH-band winner that passes the hard gates is entered at full size.
3. **POP is not a hard gate.** The family POP floors are advisory; the effective floor comes from the breach gate (about 50% inside the breakevens).
4. **Term structure is rarely measured.** Only the nearest expiry's chain is fetched (two for calendars), so the term component is usually neutral and C3_TERM drops out. Where a slope exists it is in decimal IV units but scored on a vol-point scale.
5. **GATE_CREDIT unit mismatch** (carried over from the source on purpose): net credit in currency is compared with wing width in points, so the gate is nearly always satisfied.
6. **Liquidity gate edge case**: a liquidity penalty of exactly zero is read as 1.0, which fails GATE_LIQ.
7. **Routing is a no-op** in this runtime, and the market-state breach figures come from whichever candidate is first eligible.
8. **Remote feature outages are terminal.** `RemoteFeatureUnavailable` is classified as a plugin logic error, which moves the instance to ERROR — also while a position is held, which stops soft exits and the expiry exit until an operator intervenes.
9. **Order pricing**: option orders default to market orders; limit offsets apply to the stream reference price rather than the bid/ask used for selection; no rounding to the minimum tick.
10. **Daily timing**: at `1d` the bar arrives after the close, so the entry window always passes and orders are submitted outside market hours; intraday timeframes without the remote feature source have no daily history and return `MISSING_BOUNDARY`.
11. **Year fractions differ**: breach uses DTE/365, POP and EV DTE/365.25, realised volatility $\sqrt{252}$.
12. **Paper tick type**: the backend accepts only tick type 3 for paper, so a paper gateway configured with `IBKR_MARKET_DATA_TYPE=LIVE` would have every option quote rejected, contrary to `docs/automatic-options.md`.
13. **Risk bypass**: with `OPTION_PRETRADE_RISK_ENABLED=false` short-leg hedge verification and collateral reservation are skipped entirely.
14. **Stale descriptions**: the auto-selector's seeded description still mentions a weekly ATR and a minimum-risk winner; `OptionSelectionSnapshot.atr_reference` stores spot, not the

prior close.

15. **Validation status**: no paper or live order has yet been placed under the ported decision rules; the last paper validation lacked NSE option and index market-data permissions (IBKR errors 354, 10090, 420).

15.2 Stock strategies and streaming

1. RSI is a simple-average (Cutler) RSI, not Wilder's; realised volatility is annualised with $\sqrt{252}$ on every timeframe.
2. RSI, SMA and Donchian accept SHORT/BOTH directions but only trade long; volatility-target momentum with direction SHORT does not suppress long weights.
3. The implicit `average_volume`(20) input raises the minimum warm-up of every strategy to about 20 bars.
4. The catalogue plugin's `research_signal` indicator is not implemented in Flink; an activated catalogue strategy would raise inside the indicator operator.
5. Bar-aggregation state never evicts fired buckets, so it grows for the life of the job.
6. A bar closes only when a later event at least 30 s past the window end arrives; the last bar before a quiet period is emitted late.
7. `instrument.registry.v1` is described as compacted but no topic configuration sets it; the 60-second re-publication is what keeps it recoverable. No per-topic retention is configured.
8. `strategy.targets.v1` and the other audit topics have no consumer in this repository.

15.3 Execution

1. Strategy attribution is based on the pre-risk intent delta while fills are scaled by the (possibly resized) order quantity, so attributed positions can drift from broker positions after a resize.
2. Several policy fields are not enforced on the automatic path: `OrderPolicy` collar, replace and market-order flags; `PositionSizingPolicy` exposure limits; `RebalancePolicy.portfolio_drift_threshold`.
3. There is no price-band, FX conversion or margin/buying-power check; cash is AvailableFunds less the buffer.
4. HELD intents are re-evaluated every second, writing new risk-check, outbox and attempt rows each time.
5. Broker commands retry without an attempt cap (readiness reports them after 60 s).
6. A coordination row in ERROR (rejected snapshot) is retried only when the next target event arrives.

15.4 Broker connectivity

1. Deleting a session does not cancel open IBKR orders.
2. Reconciliation compares positions and executions, not cash, orders or account values.
3. A failing gateway event blocks the sync cursor until it is fixed.
4. With the default SQLite storage, child state is lost when the container is recreated (QCH attaches no volume); per-session PostgreSQL is the durable option.

15.5 Research

1. The single-asset engine credits the $\text{open}_{t-1} \rightarrow \text{open}_t$ return to a signal formed with bar $t-1$ data, which overlaps information the signal already saw; market impact is effectively zero because a weight change is divided by dollar volume.
2. The walk-forward does not refit parameters; the scored Sharpe covers the full research span; the deflated Sharpe mixes annualised and per-observation quantities and is lenient.
3. Some validation flags and scores are constants; several grid parameters and some strategy variants are label-only (identical results), which makes the stability test trivially pass.
4. Compatibility rules and the protocol configuration are stored but not enforced by code.

15.6 Platform

1. No endpoint enforces authentication at the application layer; several order and strategy write endpoints are CSRF-exempt.
2. There is no `CACHES` setting, so throttling counters and research locks are per process.
3. `Idempotency-Retry` is not in the CORS allowed headers.
4. The committed `.env.example` is copied into the image as `.env` and enables `CLEANUP_DATABASE_ON_START` and `ALLOW_LIVE_TRADING`; it also contains values that look like real credentials and keys, which should be rotated and removed from version control.
5. `/readyz` hard-codes exactly 97 research implementations.

APPENDIX A

REST API Catalogue

All paths are relative to `/api/v1/` (also served under `APP_BASE_PATH`). "IK" marks endpoints that require an `Idempotency-Key` header.

Method	Path	Purpose	IK
System, authentication, dashboards			
GET	`system/`	Mode, live flag, broker deployment, kill switch, breaks	
GET/POST/DEL	`auth/session/`	Session status, staff login, logout	
GET	`dashboard/summary/`, `portfolios/series/`	Portfolio summary; NAV, P&L and exposure series	
Broker sessions and gateway			
GET/POST	`broker-sessions/`	List; create a managed session (202)	
GET/DEL	`broker-sessions/<uuid>/`	Detail; delete (does not cancel IBKR orders)	
POST	`broker-sessions/<uuid>/reconnect/`, `credentials/`	Reconnect; replace credentials	
GET	`broker-sessions/<uuid>/accounts/`	Account mappings	
GET/POST	`gateway/`	Connectivity summary; reconnect	
Reference data			
GET	`accounts/`, `portfolios/`, `positions/`, `instruments/`	Lists	
GET	`instruments/search/`	Broker contract search	
GET/POST	`instruments/resolve/`	Resolve and qualify an instrument	
POST	`instruments/options/chain/`, `instruments/options/resolve/`	Option chain; qualify an option	
Orders, risk, reconciliation, audit			
GET/POST	`orders/`	List (filters, pagination); manual order	yes
GET	`orders/<id>/detail/`	Order, history, diagnostics, risk, fills, attribution	
PATCH	`orders/<id>/`	Modify quantity, prices, time in force	yes
POST	`orders/<id>/cancel/`	Cancel	yes
GET/POST/DEL	`orders/manual-quote/`	Manual quote market-data lease	
GET	`orders/intents/<id>/status/`	Intent status	

Method	Path	Purpose	IK
POST	`orders/intents/<id>/confirmation/`	Confirm or decline an IBKR warning	yes
GET	`executions/`, `reconciliation/`, `audit/`, `rebalances/`	Lists	
GET/POST	`risk/`	Kill switches and recent risk checks; upsert a kill switch	yes
Strategies			
GET	`strategy-definitions/[<key>/]`, `strategy-policies/`	Definitions and policies	
GET/POST	`strategy-instances/`	List; create (optionally activate)	opt.
GET/PATCH/DEL	`strategy-instances/<id>/`	Detail; update; guarded delete	opt.
POST	`strategy-instances/<id>/enable/`, `pause/`, `flatten/`	Lifecycle actions	yes
GET	`strategy-instances/<id>/state/`, `signals/`, `runs/`, `targets/`, `ledger/`, `pnl/`, `execution-timeline/`, `chart/`	Read models	
all	`options/strategy-instances/[<id>/]`	Options-only variant (auto selector, daily)	opt.
Streaming and execution diagnostics			
GET	`streaming/health/`, `topics/`, `consumer-lag/`, `dead-letter/`	Streaming health	
POST / GET	`streaming/replay/`, `streaming/replay/<id>/`	Replay a topic range; status	yes
GET	`execution/readiness/`, `diagnostics/`, `flink-diagnostics/`	Readiness and diagnostics	
Allocation, rebalancing, sizing			
GET	`allocations/policies/`, `allocations/runs/[<id>/]`	Allocations	
POST	`allocations/flows/`	Deposit or withdrawal	yes
GET	`rebalancing/policies/`, `rebalancing/runs/[<id>/]`	Rebalances	
POST	`rebalancing/preview/`, `rebalancing/run/`	Plan a preview or execution rebalance	yes
POST / GET	`position-sizing/preview/`, `position-sizing/decisions/<id>/`	Sizing	yes
Market data providers			
GET	`data-providers/finnhub/`	Status and fallback flags	
POST	`data-providers/finnhub/configure/`, `test/`	Configure key; test connection	yes
GET/POST	`data-providers/finnhub/mappings/[<id>/]`	Mappings; manual verification	yes
Portfolio optimisation			

Method	Path	Purpose	IK
GET/POST	`portfolio-universe/`, `portfolio-optimization/policies/`	Universe and policy	
POST	`portfolio-optimization/preview/`, `run/`	Preview; apply	yes
GET	`portfolio-optimization/runs/[<id>/]`	Runs	
Portfolio construction (Portfolio Builder)			
GET/POST/PATCH	`portfolio-construction/plans/[<id>/]`, `.../goals/`	Plans and goals	
POST	`portfolio-construction/plans/<id>/recommendations/`	Generate recommendations (202)	yes
GET	`portfolio-construction/recommendation-batches/<id>/`, `readiness/`	Batch status; builder readiness	
various	`.../goals/<id>/instruments/`, `.../assignments/`	Stock selections and strategy assignments	
POST	`portfolio-construction/preview/`, `runs/<id>/apply/`	Preview; apply	yes
Research (read-only)			
GET	`research/dataset-versions/`, `universes/`, `strategies/`, `readiness/`, `candidate-scores/`, `experiments/<id>/`	Research data	

Outside `/api/v1/`: GET `/`, `/healthz`, `/readyz`, `/metrics`, and the noVNC proxy at `/api/v1/broker-sessions/<uuid>/novnc/`.

APPENDIX B

Environment Variables

Names and non-secret defaults only. Values marked "secret" must come from the deployment's secret store.

Variable	Default	Purpose
Web and data stores		
PORT / APP_BASE_PATH / PUBLIC_BASE_URL	8000 / "" / —	Serving and public URLs
DJANGO_SECRET_KEY, DATABASE_URL	secret	Django key; PostgreSQL DSN
REDIS_URL, CELERY_BROKER_URL	secret	Redis
CLEANUP_DATABASE_ON_START	false (code)	Destructive reset at start
Order path		
STRATEGY_EVALUATION_INTERVAL_SECONDS	1	Evaluation poll
TARGET_COORDINATION_INTERVAL_SECONDS	1	Coordination poll
ORDER_INTENT_INTERVAL_SECONDS	1	Intent poll
BROKER_COMMAND_DISPATCH_INTERVAL_SECONDS	1	Command poll
BROKER_SYNC_INTERVAL_SECONDS	5	Gateway event sync
OUTBOX_PUBLISH_INTERVAL_SECONDS	2	Outbox publisher
STRATEGY_EVALUATION_MAX_ATTEMPTS	5	Job retries
PORTFOLIO_TARGET_MAX_AGE_SECONDS	900	Target staleness
PORTFOLIO_TARGET_COORDINATION_DEBOUNCE_SECONDS	2	Debounce
EXECUTION_AVERAGE_VOLUME_WINDOW / EXECUTION_REGISTER_ADV_INPUT	20 / true	ADV input
EXECUTION_ACTIVATION_PREFLIGHT_ENABLED	true	Readiness preflight
PENDING_INTENT_MAX_AGE_SECONDS / BROKER_COMMAND_MAX_AGE_SECONDS	60 / 60	Readiness ages
EXECUTION_WORKER_HEARTBEAT_STALE_SECONDS	30	Worker liveness
Streaming		
KAFKA_ENABLED / KAFKA_BOOTSTRAP_SERVERS	false / localhost:9092	Kafka
KAFKA_TOPIC_PARTITIONS	3	Topic creation
EXECUTION_REQUIRED_KAFKA_TOPICS	15 topics	Readiness
FLINK_REST_URL / EXECUTION_REQUIRED_FLINK_JOBS	— / market-pipeline-v1	Flink
FLINK_CHECKPOINT_STALE_SECONDS	180	Readiness
FLINK_PYTHON_RUNTIME_MODE	archive	Worker Python
Broker		

Variable	Default	Purpose
QCH_APP_ID, QCH_API_HOST, QCH_SERVICE_TOKEN	secret	Container broker
IBKR_GATEWAY_IMAGE	—	Pinned child image
BROKER_SESSION_ENCRYPTION_KEY	secret	Fernet key
BROKER_CREDENTIAL_TTL_SECONDS	900	Temporary credentials
GATEWAY_HTTP_TIMEOUT_SECONDS	10	HTTP timeout
GATEWAY_COMMAND_TIMEOUT_DEFAULT_SECONDS	20	Command wait
IB_GATEWAY_POSTGRES_HOST (and _PORT, _DB, _USER, _PASSWORD)	—	Per-session child database
LOCAL_PAPER_GATEWAY_URL, LOCAL_PAPER_GATEWAY_SERVICE_TOKEN	—	Static local paper gateway
Policy		
ALLOW_LIVE_TRADING	false	Live gate
GLOBAL_KILL_SWITCH	false	Stop all new orders
OPTION_PRETRADE_RISK_ENABLED	true	Option risk checks
Market data		
MARKET_PRICE_STALE_SECONDS	300	Price staleness
WARMUP_SAFETY_BARS / WARMUP_TIMEOUT_SECONDS	5 / 300	Warm-up
MANUAL_MARKET_DATA_LEASE_SECONDS	120	Manual ticket lease
OPTION_CHAIN_PROVIDER	IBKR	Option data source
OPTION_STRATEGY_MARKET_DATA_LEASE_SECONDS	86400	Leg quote lease
REMOTE_FEATURE_DB_URL / REMOTE_FEATURE_SESSIONS	secret / 500	RPS feature source
MARKET_DATA_FALLBACK_ENABLED and the three FINNHUB_*_FALLBACK_ENABLED flags	false	Finnhub fallback
FINNHUB_API_KEY, FINNHUB_ENCRYPTION_KEY	secret	Finnhub credentials
Research		
RESEARCH_ENABLED / RECOMMENDATION_SYSTEM_ENABLED	true / true	Switches
RECOMMENDATION_MAX_STOCKS / RECOMMENDATION_MIN_STOCKS	20 / 5	Recommendation size
RESEARCH_MINIMUM_DAILY_BARS	756	History requirement
RESEARCH_MAX_PARALLEL_DATA_TASKS / ..._BACKTEST_TASKS	8 / 8	Worker threads

APPENDIX C

Status Glossary

Entity	States
Strategy instance	DISABLED, ACTIVATING, SUBSCRIBING, WARMING_UP, READY_WAITING_FOR_LIVE_BAR, FLAT, LONG, SHORT, ENTRY_PENDING, PARTIALLY_LONG, EXIT_PENDING, PARTIALLY_SHORT, PAUSED, FLATTEN_REQUESTED, KILLED, BLOCKED, ERROR
Evaluation job	WAITING_FOR_INPUT, PENDING, CLAIMED, RUNNING, RETRY, COMPLETED, FAILED
Market subscription	PENDING, SUBSCRIBING, ACTIVE, DEGRADED, ERROR, CANCELLING, INACTIVE; provider IBKR, FINNHUB, NONE; fallback PRIMARY, FAILING_OVER, FALLBACK, RECOVERING, FAILED
Market state	FRESH, STALE, UNAVAILABLE; data type LIVE or DELAYED; processing mode LIVE, WARMUP, REPLAY, BACKFILL
Target coordination	IDLE, PENDING, CLAIMED, ACTIVE, ERROR
Rebalance run	QUEUED, CALCULATING, PLANNED, INTENTS_CREATED, EXECUTING, COMPLETED, PARTIALLY_COMPLETED, FAILED; phases PLANNING, PREVIEW_COMPLETE, SELLS, BUYS, OPTION_HEDGES, OPTION_SHORTS, COMPLETE, BLOCKED
Order intent	PENDING, CLAIMED, RISK_APPROVED, QUEUED, RISK_REJECTED, FAILED, BROKER_BLOCKED, BROKER_REJECTED, UNKNOWN, CONFIRMATION_REQUIRED, CONFIRMING, USER_CANCELLED, COMPLETED
Order	CREATED, RISK_APPROVED, QUEUED, BROKER_BLOCKED, SUBMITTED, ACKNOWLEDGED, PARTIALLY_FILLED, FILLED, CANCEL_PENDING, CANCELLED, REJECTED, EXPIRED, UNKNOWN
Broker command	PENDING, CLAIMED, SENDING, RETRY, UNCERTAIN, ACKNOWLEDGED, FAILED
Risk check	APPROVED, RESIZED, HELD, REJECTED
Capital reservation	ACTIVE, CONSUMED, RELEASED
Gateway session	CREATING, STARTING, WAITING_FOR_LOGIN, WAITING_FOR_2FA, CONNECTED, DISCONNECTED, LOGIN_FAILED, ERROR, STOPPING, DELETED
Reconciliation run	RUNNING, COMPLETED, BLOCKED; breaks CRITICAL or WARNING, material or not
Outbox event	PENDING, PUBLISHING, PUBLISHED, FAILED